Draft version December 16, 2024
Typeset using LaTeX **twocolumn** style in AASTeX631

A Binary Ballad: Tracking Beta Aurigae's Radial Velocities

114995854, 114380964, and 115282553[1]

[1] *Department of Physics and Astronomy, Stony Brook University*

(Dated: 15th of December, 2024)

ABSTRACT

This report details a spectroscopic survey of the binary star system Beta Aurigae in hopes of unveiling their radial velocity, using the 14-inch telescope at Mt.Stony Brook Observatory on November 13th, 2024. The experiment aimed to track radial velocity values of the star's orbits right after hitting their peak speeds via measuring the Doppler shifts of certain spectral absorption lines. Through data analysis and reduction, which included wavelength correction, flat-field, bias, and dark calibration, and Gaussian fitting/correction, we resulted in multiple measured radial velocity values for different spectral lines: Hα, Hβ, and Hγ ranging from 198.24 km/s - 243.69 km/s

1. INTRODUCTION

1.1. *Scientific Motivation*

Binary star systems are critical entities for studying stellar masses, orbital mechanics, and stellar evolution. The various and immense number of combinations that they provide, such as star types, orbiting distances, and composition, provide a multitude of unique case studies that help us better understand stellar dynamics, composition, and interactions with each other. Beta Aurigae is a binary system that is both eclipsing and spectroscopic, meaning that their orbital radii are too small to resolve separately and can only even then be resolved via spectroscopy and that the stars orbit each other such that one star will periodically block our view of the other star from Earth's frame of reference(Tatum (2004)).

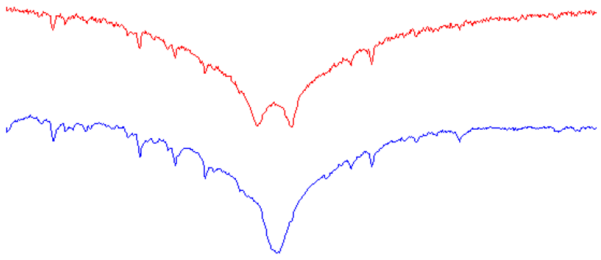
Beta Aurigae, particularly notable for its observable spectroscopic features, having a coupled apparent magnitude of 1.9 and only 81 light years away from Earth, provides direct insights into the system's dynamics (Unknown (2007)). By analyzing Doppler shifts in spectral lines such as Hα, Hβ, and Hγ, we can derive the radial velocities of the binary components (Staff Writers (2020)). Allowing us to better understand Binary interactions, movements, and behavior. Tracking radial velocities can lead to further projects such as mass ratio calculations, roche lobe predictions, and more beyond the relative scope of this experimentation as they require more intricate radial velocity curves and mapping, and we strictly calculated a specific value range right after the peak velocity when we obtained our data (Tauris & van den Heuvel (2017)).

Spectroscopic imaging of the stars absorption spectrum allows us to image different spectral lines that can ultimately give insight into orbital dynamics. Beta Aurigae's absorption spectra are known to contain Hα, Hβ, and Hγ, lines that allows us to observe Doppler shifts binary systems. (Unknown (2007)).

1.2. *Experimental Goals*

In this lab we aimed to observe Doppler shifts constituted by minor pixel position shiftings in our data allowing us to calculate radial velocity values, through spectroscopic imaging method. This method entails taking various series of spectroscopic images in different wavelength ranges of the binary system, aiming to capture specified absorption lines of beta Aurigae. And construct wavelength maps in order to track the pixel shifts from the different data sets taken to identify Doppler

Figure 1. Literature Hα Spectral Lines



example of a spectrum with double

Halpha absorption lines (but single telluric lines)

shifts. This spectroscopy method allows us to bypass the struggles of detecting the dynamics of spectroscopic binary systems in trying to resolve the stars and obtain information about their kinematics.

Here is a theoretical radial Velocity curve as a function of time generated by one of our lab members. The peaks represents the max and min velocity magnitudes, with X=0 representing the peak right after we assumed to have taken our data after. The red data values represent radial velocities values at those specified times helping visualize the velocity changes in comparison to time(Tatum (2004)). The next section, Observations and Data, will entail the experimental design, data acquisition, unique complications/issues, and data descriptions. We will then detail our data reduction, analysis, and troubleshooting in section three, and will complete the papers with our results and their implications and discussions in section 4.

Figure 2. Theoretical Radial Velocity Curve

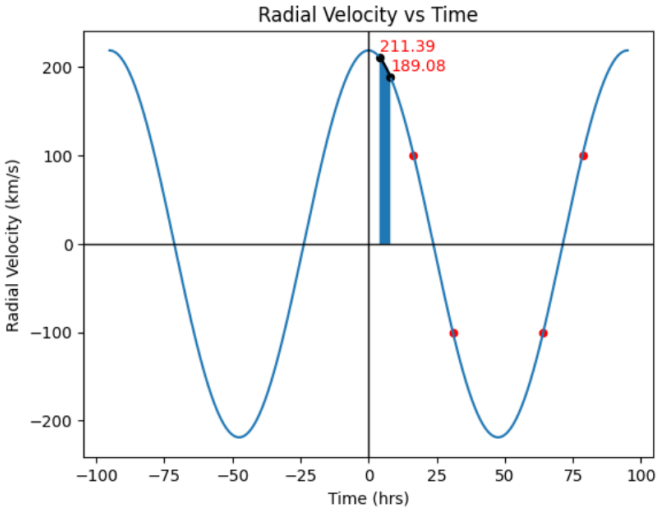


Table 1. Predicted Radial Velocities.

Measurement Set	Time (hrs)	Expected Velocity
Hα set 1	5.300	205.694 km/s
Hα set 2	7.933	189.564 km/s
Hβ, γ set 1	6.767	197.450 km/s
Hβ, γ set 2	7.033	195.749 km/s

2. OBSERVATIONS AND DATA

2.1. *Experimental Setup*

The experimental setup entailed primarily calculating and plotting a time dependent theoretical radial velocities curve, defining maximums and minimums in velocity magnitude, and selecting times right after a maximum, when we observe the largest slope gradient, in order to obtain data that has the largest pixel shift. Our goal was to obtain a few pixel shift where one pixel is correspondent to one Angstrom, which assisted in conversions to Doppler shift values.

Due to weather conditions and scheduling constraints, we conducted our experiment on the 13th of November. We created Finder charts (slightly larger than the Telescope's F.O.V of 23.69') and Star Alt Plots to help properly locate the target in the telescope during calibration. Since we had various data sets to take, we took our calibration files: flats and wavelength calibration (using Ne and Ar spectrum lamps) at the end of each science images set, and darks at the complete end. We needed two different arc lamps because we took our data sets in two different wavelength ranges: 300-400 nm and 600-700 nm, in order to capture Hβ and Hγ, and Hα along with predicted atmospheric telluric O2 lines respectively(Atomic Spectra Database (2024)).

Our duration at the Mt. Stony Brook observatory lasted from 10:00 pm to 3:30 am. The first hours consisted of setting up the telescope, which included hooking up our high resolution spectrograph DADOS spectrograph (with 900 I/m grating) and CCD camera - we used a ST402ME - to our telescope, a 14-inch Meade LX200-ACF telescope which rests on a Mesu-200 German Equatorial Mount (Sibirrer & Contributors (2024)).We operated the CCD Camera and Telescope from a windows machine that has CCD Soft, Cartes De Ciel, and other relevant applications. Once our setup was connected and ready to go, we begun by calibrating our wavelength spectrum's and locating the star.

Once we completed our wavelength calibrations and located our star, we had to manually align our star in our slit finder, as we worked in the "middle" slit of the 3 slits within the camera.

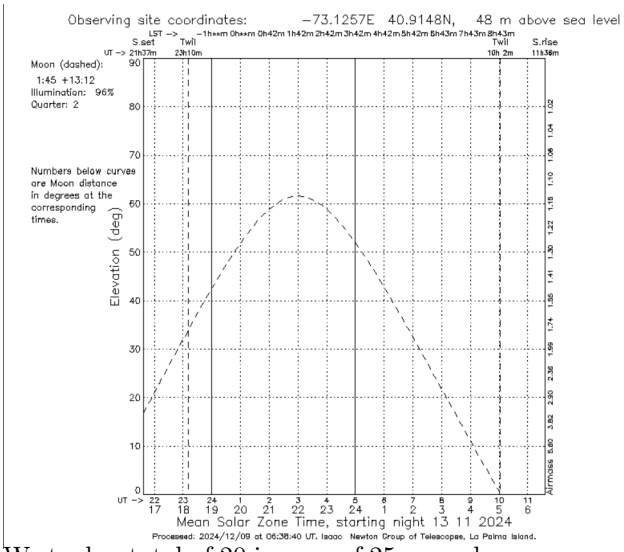
2.2. *Experimental Design and Data-taking*

We will detail the experimental plan and process for the data taking in this section. Our program involves taking various series of spectroscopic images of 3 main spectral lines: Hβ, Hγ, and Hα along with telluric O2 lines. Our spectrograph has a slit of 100 nm, so The Hβ and Hγ were captured in the same set as they are close in wavelength range: 486 nm and 434 nm, and Hα was taken in the same set as the telluric O2 lines as they were both at around 656.3 nm and 687 nm -695 nm. Our flat fields and wavelength calibration images were taken at the end of each set, we used a Neon arc lamp for the 600-700 nm range and an Argon arc lamp for the

400-500 nm range in order to calibrate our wavelengths for the purpose of properly correlating pixel position to wavelengths on the spectrum. Darks were taken all at the end for the various exposure times.

Our duration lasted a total of 6 hours starting at around 10 pm and ending at 4 am. The moon was not too high but there was high illumination indicated by the star alt plot (Figure 6). But due to the positioning of the moon versus Beta Aurigae that night, we did not encounter any issues due to lunar illumination. As for weather conditions, we experienced essentially 0 cloud coverage visible to the naked eye. Forecasts also predicted 0 cloud coverage during our hours of recording.

Figure 3. Transit information



We took a total of 20 images of 25 second exposures per science image set, with 10 wavelength calibration images of 15 seconds for Neon and 10 seconds for Argon. With the appropriate flats and darks for each taken set based on the exposure times. The filter here is irrelevant since we will not be working with visible light and using the spectroscopic camera.

Figure 4. Summary Of Observation Frames

Frames	Total # of Exposures	Exposure time (s)	AutoDark
Hα	20	25 s	Off
Hβ	20	25 s	Off
Hγ	20	25 s	Off
Dark Frames	50	60, 25, 15, 10, 3 s	Off
Flat Frames	15	60 and 3 s	Off
Neon arc Lamp	20	15 s	Off
Argon arc Lamp	5	10 s	Off

Our recorded calibration files were flat fields, wavelength calibration, and dark frames. These encompass our needed files to correct any read out noise, pixel sensitivity, vignetting, and of course dust/dirt on the telescope. The dark frames already incorporate the bias frames into them as they are just a measure of electric noise which is a part of the read out noise. And the wavelength calibration allows us to map pixel correlations to wavelength positioning on the Neon and Argon spectrum.

3. DATA ANALYSIS

In this section we detail our overall data analysis of the obtained data, including reduction, wavelength calibration, Gaussian fitting, and Doppler shift calculations to obtain our radial velocity values

3.1. *Data Reduction*

We begin our data analysis by primarily correcting the images from intrinsic factors that all telescope users have to deal with. Factors like readout and electrical noise, vignetting, dust/dirt, and pixel sensitivity are just a few variables that our images pick up, but can be corrected by calibration-al images. Dark frames, images taken at certain exposures with a closed shutter, allow us to measure overall readout noise: statistical electric noise (also known as bias frames - 0 exposure images) and noise due to thermal fluctuations. Flat fields are images taken at even illumination sources, maintaining same exposures as our science images, in order to capture the effects of vignetting, pixel variability, and dust/dirt on the telescope and camera.

We took a total of 50 dark frames for our different exposures times across flat fields, wavelength calibrations, and science images, and 15 flat field frames for the purpose of calibration. It is important for the dark frames and flat fields to be uniform in exposures in order to make sure that we don't over or under correct the pixel variations of the flat fields. We started off by taking the mean combine of our dark frames by stacking our 10 images in a 3D array, and finding the average pixel value at each position. We then take the same mean combine for our 10 flat field images, and then we normalize our flat fields. Normalization makes them universally applicable, independent of exposure times or brightness levels. We accomplish this by dividing out the median value of the difference of the master flats and darks, from their difference.

$$Flat_{Master_{norm}} = \frac{Flat_{Master} - Dark_{Master}}{Median[Flat_{Master} - Dark_{Master}]} \quad (1)$$

In order to isolate our target star and data, we defined cuts along the 25 μm middle slit. (we will address the

target signal correction in a later section). We then match our calibration spectra and flat fields to the same frame cuts.

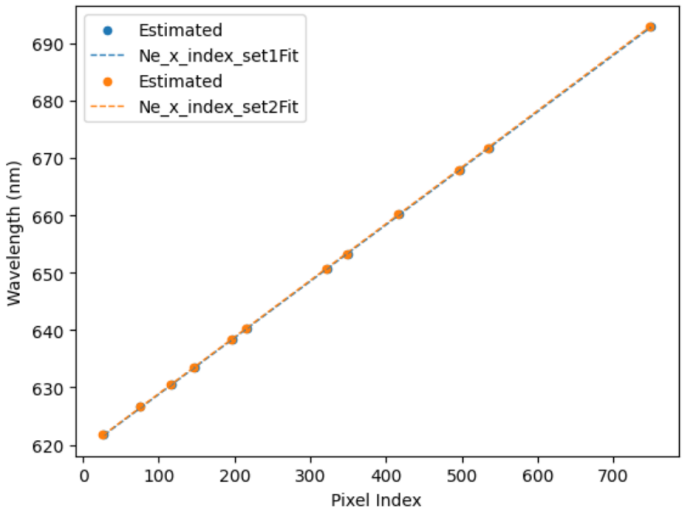
We also repeated the mean combines process for both the wavelength calibration and our science images, we then finally calibrated our science images and calibration spectra using the master dark frames and polynomial fitting for flat fields. This is where you fit a curve to the median pixel value for each column versus pixel position and normalize by dividing that out to account for flat field correction. We applied this polynomial fitting instead of flat field image division for our flat corrections as it seemed to give a better correction for background gradients and illumination.

$$Science_{calibrated} = \frac{Science - Dark_{Master}}{Flat_{Master_{norm}}} \quad (2)$$

3.2. *Wavelength Calibration and Target Spectra*

We then proceeded to cross reference our calibration spectra with the known spectra from a couple websites we found: NIST emission wavelength database and Atomic Spectra website. We picked various pixel positions on our spectra that lined up with the proper emission wavelengths and then we fit a linear function of pixel positions versus wavelength in order to derive a wavelength calibration solution and whether our calibration spectra is accurate or not. The provided images 5 and 6 show that our derivations were pretty accurate as we obtained a fairly linear relationship between the wavelengths and pixel position, so our calibration files can be used accordingly.

Figure 5. Neon Wavelength calibration solution



Once we calibrated our wavelength spectra, we proceeded to tackle the aforementioned issue of isolating

Figure 6. Argon Wavelength calibration solution

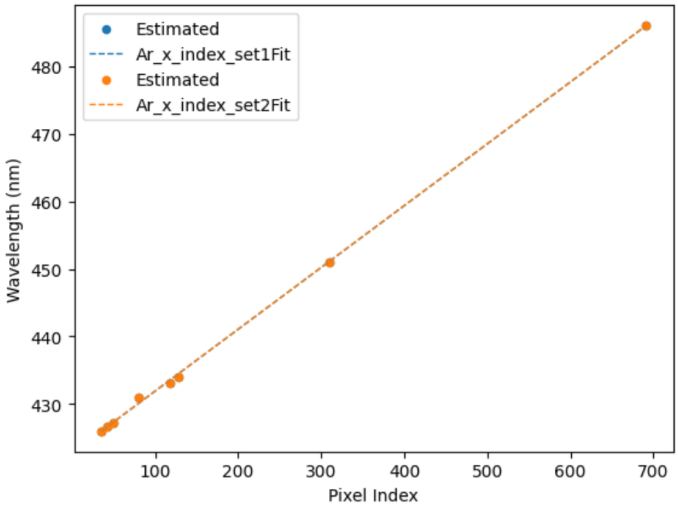
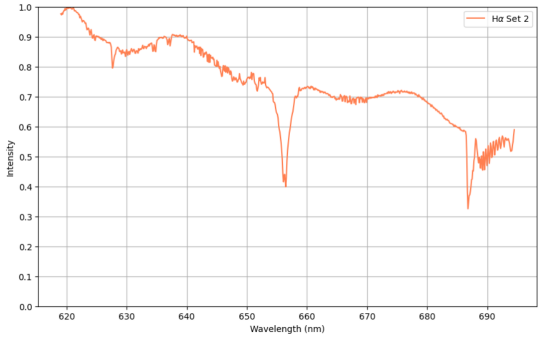


Figure 7. calibrated Hα spectrum



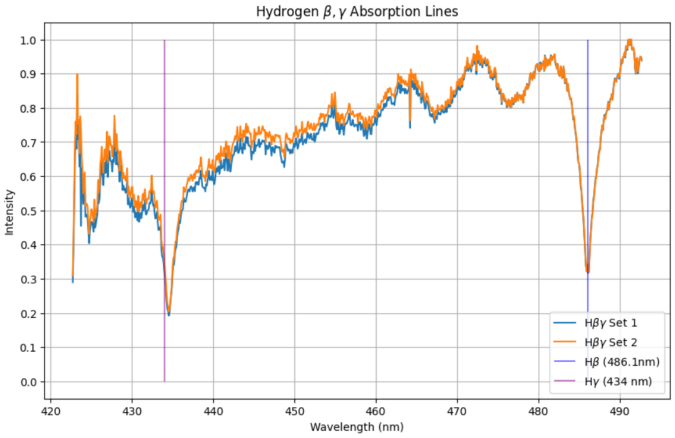
our target signal from intrinsic background noise. We did this via fitting a Gaussian to the data skew by finding the mean and standard deviation of each column and isolating the points that are within or above a certain threshold we identified relative to our standard deviation. We then moved our data points that fell outside of this range into a new array for background signal and again found the mean and standard deviation in order to exclude outliers. And calculated our final signal as:

$$S' = S - N\mu_{\text{background}} \quad (3)$$

where N is the # of pixels in the target signal. We then calibrate our science spectra images using this fitting, and then subtracting the background noise from our target signal. And below is the calibrated and science spectra plotted simultaneously to view the proper correlation and post correction for intrinsic errors and noise, of our second Hα set and a stacked image of both sets of the Hβ and Hγ lines.

3.3. *Wavelength Alignment and Gaussian Fitting*

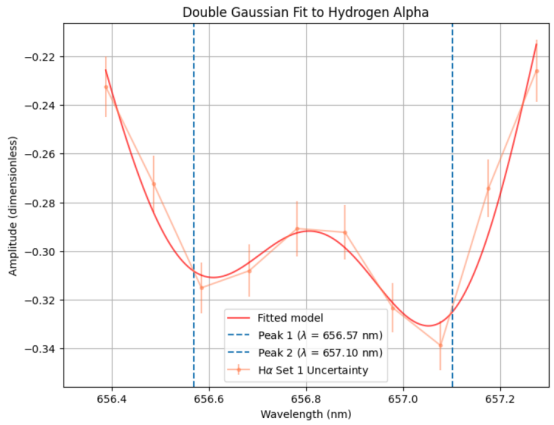
Figure 8. stacked calibrated Hβ and Hγ spectrum



As visible in figure 8, our expected dip for the Hα line in comparison to the known Hα line is slightly off, so we had to align them manually. We used the telluric O2 lines in order to adjust our spectrum as we know that because they are atmospheric absorption lines produced by the Earth, they do not experience any Doppler shifts so we can use them as a referential point in order to manually adjust the Hα spectrum to match the known wavelength positions

Once we matched up the peaks and our Hα line was much closer to the expected wavelength, we could then fit a Gaussian to the entire spectrum in order to observe the proper centroid of the spectra which assists us in Doppler shift calculations and can also assist in correcting for large outliers and errors. Due to the nature of our data we observed a doublet in our Hα absorption, and so we enforced a double Gaussian system to fit the data in order to preserve double dip. We infer that the dip is due to picking up Hα absorptions from both stars.

Figure 9. zoomed in calibrated Hα spectrum



This is a zoomed in snapshot of the doublet after a double gaussian was fit to the data and as you can see the dips are smoothly shaped as expected and match pretty closely to known values for Hα represented by the dotted blue lines.

3.4. *Doppler Shifts and Statistical Testing*

After obtaining our fitted data, with wavelength values reasonably close to our expected data values, we were finally able to calculate our Doppler shift values using this equation:

$$v = c \cdot \frac{\Delta\lambda}{\lambda_{\text{rest}}} \tag{4}$$

We obtained our rest and shifted wavelengths from our fitted graphs. Here are our resulting raw Doppler shifts from the gaussian graphs for Hβ, Hγ, and Hα

Table 2. Raw Doppler Shifted Velocities

Hα set 1	243.69 km/s
Hα set 2	201.53 km/s
Hβ set 1	204.15 km/s
Hβ set 2	198.24 km/s
Hγ set 1	228.66 km/s
Hγ set 2	222.03 km/s

In order to to better understand the validity and nature of our Gaussian fits to our calibrated final spectrum, and therefore the validity of our Doppler shift calculations. We performed a chi-squared test; a statistical test in which you compare the spectral line profile to the our fitted Gaussian model, and observe our reduced chi-squared value on how much it varies from 1 indicating consistencies and uncertainties in our data.

Figure 10. Chi Squared test for Hα

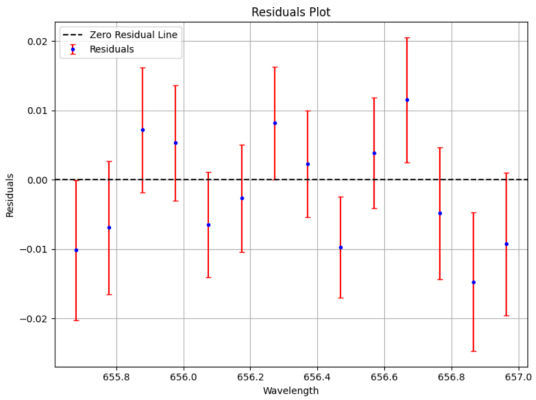


Figure 11. Chi Squared test for Hγ set 1

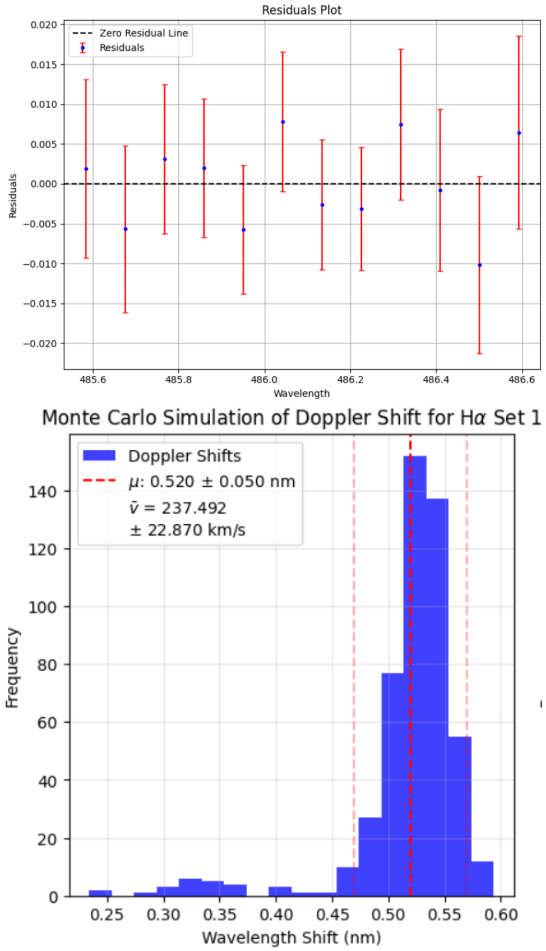


Figure 12. Monte Carlo Simulation Graphs

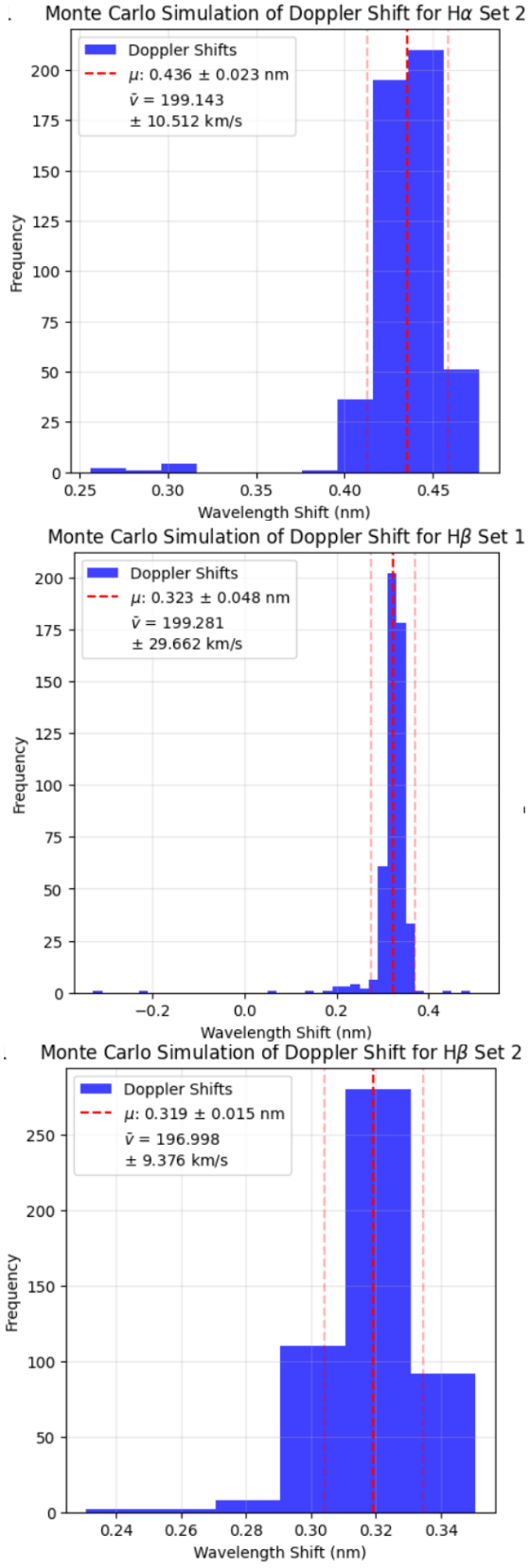


Table 3. Chi Squared Test Results

Data sets	Hα set 1	Hα set 2	Hβ/γ Set 1	Hβ/γ set2
χ^2	6.15	11.43	3.80	7.59
Reduced χ^2	1.54	1.43	0.63	1.27
P-Value	0.19	0.18	0.70	0.27

Here are a few visual result of our tests for our Hα set 1, and Hγ set 1. And the summary of our data values from all our set tests. As seen by the resulting statistical values, the sets overall are fairly efficient and the Gaussian's fit well to the model. The 'Hbg' set 1 encompasses the best results compared to the other sets. With the highest P-value as well as the most fit reduced χ^2 being the closest to a value of 1.

Once we finished our Chi squared testing, to obtain better results of our Doppler shift velocities, we performed a Monte Carlo simulation on the data set fitting and value calculations for the Doppler shift values. Monte Carlo testing is a statistical method used to assess uncertainties or explore variability in a system

Figure 13. Chi Squared Test Results

	Mean λ Shift	λ Shift uncertainty	Mean Doppler Shift	Doppler Shift Uncertainty
Hα Set 1	0.520	± 0.050	237.492	± 22.870
Hα Set 2	0.436	± 0.023	199.143	± 10.512
Hβ, γ Set 1	0.323	± 0.048	199.281	± 29.662
Hβ, γ Set 2	0.319	± 0.015	196.998	± 9.376

by performing repeated random sampling. Gives uncertainties as well as evaluates how the noise and error propagates throughout the overall analysis(Bradley & Cutcomb (1977)). Each data set was ran 501 times, due to the nature of our data's complex noise and to ensure large sample size with robust results. making it odd also allows us to establish and work with median results for the test values. Running the Monte Carlo Simulation gave us overall much better results, summarized in the table above:

4. RESULTS

After correction, calibration, alignment, and analyzing the spectroscopic data taken for Beta Aurigae, we were successfully able to derive Doppler shifts and velocities for the $H\beta, H\gamma, H\alpha$ lines. They were derived by fitting Gaussian [and double Gaussian for $H\alpha$] to the absorption line spectra. The Doppler velocities exhibit clear evidence of dual stars present in Beta Aurigae, reflecting the motion of its stellar components relative to Earth. These velocities are consistent with expectations from the radial velocity curve seen in table 1.

Table 4. Doppler Shift Velocities for Different Spectral Lines

Spectral Line	Doppler Shift Velocity (km/s)
Hα Set 1	243.69
Hα Set 2	201.53
Hβ Set 1	204.15
Hβ Set 2	198.24
Hγ Set 1	228.66
Hγ Set 2	222.03

Additionally, chi-squared testing and Monte Carlo simulations were performed to validate the results: Reduced χ^2 values ranged between 0.63 and 1.54, indicating that the Gaussian fits were consistent with the observed data within the uncertainties. The Hβ and Hγ set 1 results showed the best fit, with reduced χ^2 values closest to 1 at 0.63. After running 501 iterations of the Monte Carlo simulation, we obtained reasonable intervals for the Doppler shift velocities, showing low variability due to noise or fitting errors. This strengthens the reliability of our derived velocities.

5. DISCUSSION

The results of this experiment provide meaningful insights into the orbital dynamics of the Beta Aurigae binary system. By using spectroscopic imaging and Doppler shift calculations, we were able to: observe double Gaussian profiles for the $H\alpha$ line strongly suggest the presence of two stars contributing to the absorption spectrum. This supports the classification of Beta Aurigae as a spectroscopic binary. The Doppler shifts obtained for $H\alpha$, $H\beta$, and $H\gamma$ lines highlight the relative motion of the binary components. These velocities reflect their orbital motion and provide a foundation for further analysis, such as constructing a complete radial velocity curve and determining orbital parameters. We were also able to observe tulleric O2 which allowed us to examine atmospheric absorption lines due to the Earth, unaffected by Doppler shifts, allowing us to align the $H\alpha$ spectrum with the telluric O2 lines. Serving as an effective method for addressing wavelength calibration.

To build upon this work, future studies should aim to observe Beta Aurigae over its entire orbital cycle. Doing so would allow for the construction of a more detailed radial velocity curve, facilitating accurate mass and orbital measurements for the two stellar components. Additionally, incorporating high-resolution spectroscopy across a broader wavelength range could provide further insights into the chemical compositions, temperature profiles, and rotational velocities of the stars. Advanced modeling techniques, such as Roche lobe analysis, could also reveal whether mass transfer processes are occurring within the system (Tauris & van den Heuvel (2017)).

6. CONCLUSION

In this work, we conducted a spectroscopic analysis of the binary star system Beta Aurigae to determine its radial velocities through Doppler shift measurements. By leveraging data collected using the 14-inch telescope at the Mt. Stony Brook Observatory. We successfully measured wavelength shifts in the prominent Hα, Hβ, and Hγ absorption lines, as well as telluric O2 features. These observations allowed us to infer the radial veloci-

ties of the binary components as they moved along their orbits, providing valuable insight into the system's dynamics.

Our analysis pipeline involved extensive calibration and data reduction procedures to account for systematic errors, background noise, and instrumental artifacts. Dark frames, flat-field normalization, and wavelength calibration using Neon and Argon arc lamps were steps in producing accurate spectra. The alignment of the spectra with telluric O2 lines ensured proper referencing to stationary atmospheric absorption features, correcting for any residual wavelength shifts. Furthermore, Gaussian and double-Gaussian fitting methods were applied to the spectral line profiles to accurately determine their centroids, which were then used to compute Doppler shifts.

chi-squared testing demonstrated that our Gaussian fits were consistent with the observed spectral line profiles, with reduced chi-squared values close to 1 and reasonable p-values, further validating the reliability of our measurements. As well as Monte Carlo simulations, were employed to quantify uncertainties in the derived Doppler velocities. This statistical approach allowed us to evaluate the robustness of our results against noise and error propagation.

The results from this study, summarized in tables 1 and 4, revealed clear Doppler shift velocities for the Hα, Hβ, and Hγ lines, with values ranging between approximately 198 km/s and 243 km/s, depending on the spectral set. The double-Gaussian fits for the Hα line provided additional evidence of the binary nature of Beta Aurigae, as both stellar components contributed to the absorption features. These radial velocity values correspond to significant orbital motion within the system, consistent with the expectations for an eclipsing spectroscopic binary.

The analysis of Beta Aurigae provides key insights into the orbital behavior of binary star systems, which are fundamental to understanding stellar evolution and dynamics. Radial velocity measurements can serve as a foundation for calculating essential orbital parameters such as the mass ratio, semi-major axes, and orbital inclination. Additionally, extending this study to cover a full orbital period could allow for the creation of a complete radial velocity curve, enabling precise determination of the stellar masses through Kepler dynamics(Tauris & van den Heuvel (2017)).

Our approach demonstrates the power of spectroscopy in overcoming the limitations of direct imaging for closely orbiting binaries. By carefully calibrating observational data and employing robust statistical methods such as Monte Carlo simulations and chi-squared tests, we were able to extract meaningful results despite the inherent noise and challenges in our dataset(Bradley & Cutcomb (1977)).

Through this experiment, we successfully demonstrated the application of spectroscopic imaging and Doppler shift analysis to study a binary star system. Beta Aurigae's spectroscopic features provided a rich dataset for understanding its dynamics, and our methods effectively revealed its radial velocity behavior. The results not only validate our approach but also lay the groundwork for future investigations into the stellar and orbital properties of binary systems.

REFERENCES

Atomic Spectra Database. 2024, Atomic Spectra Database, https://www.atomic-spectra.net/

Bradley, D. R., & Cutcomb, S. 1977, Behavior Research Methods Instrumentation, 9, 193, doi: 10.3758/BF03214499

Sibirrer, T., & Contributors, P. 2024, Observing Equipment, https://github.com/sibirrer/PHY517_AST443/wiki/Observing-Equipment

Staff Writers. 2020, Color-Shifting Stars: The Radial-Velocity Method, https://www.planetary.org/articles/color-shifting-stars-the-radial-velocity-method

Tatum, J. 2004, Celestial Mechanics, http://astrowww.phys.uvic.ca/~tatum/celmechs.html

Tauris, T. M., & van den Heuvel, E. P. 2017, Physics of Binary Star Evolution – from Stars to X-ray Binaries and Gravitational Wave Sources (Cambridge: Cambridge University Press)

Unknown, A. 2007. https://arxiv.org/abs/astro-ph/0703634

Import Statements

In [1]:

```
#matplotlib ipympl
import numpy as np
from astropy.io import fits
import matplotlib.pyplot as plt
from matplotlib.ticker import MaxNLocator
from scipy.optimize import curve_fit
from scipy.stats import chi2
from IPython.display import clear_output
from IPython.display import Image
import os
from os import path
import pandas as pd
from scipy.signal import find_peaks
import glob
import math
import mplcursors
import time
```

Functions

In [2]:

```
def fits_data_to_3d_array(file_list):
    '''
    Takes in a list of fits files and converts data into a 3d array
    Parameters:
        file_list (list): List containing paths of FITS files to convert.
    Returns:
        final_array: 3d array of data from each fits file.
    '''
    for i, file in enumerate(file_list):
        with fits.open(file) as hdu:
            if i==0:  #If first iteration, create array with correct shape of data
                final_array = np.zeros((len(file_list), *hdu[0].data.shape))
            final_array[i,:,:] = hdu[0].data
    return final_array

#Creates a median combine of a 3d array
def median_combine(frame_array_3d):
    master_frame = np.median(frame_array_3d,axis=0) #takes median along the 'file_index' axis
    return master_frame

def save_array_to_fits_file(array, new_file_name):
    '''
    Saves a 2d array of pixel data to a FITS file.
    Parameters:
    array(2d): 2d array containing pixel values.
    new_file_name (path): location, name of FITS file to be saved.
    '''
    hdu = fits.PrimaryHDU(data = array)
    hdul = fits.HDUList([hdu])
    hdul.writeto(new_file_name,overwrite=True)


def glob_files(folder, specifier, asterisks='both', show=False):
    asterisk = "*"
    if asterisks == 'both':
        files = glob.glob(os.path.join(folder, asterisk + specifier + asterisk))
    elif asterisks == 'left':
        files = glob.glob(os.path.join(folder, asterisk + specifier))
    elif asterisks == 'right':
        files = glob.glob(os.path.join(folder, specifier + asterisk))
    if show:
        print(f'Number of Files: {len(files)}')
        for file in files:
            print(file)
    return files

def cut_pixel_data_array(pixel_data_array, y_pixel_min, y_pixel_max, print_shape=False):
    '''
    Returns an array of pixel data, cut to a specified vertical (y-pixel) range.
    Parameters:
        pixel_data_array (2d array): Array of pixel data to create a cut from.
        y_pixel_min (int): Lower bound to start cut.
        y_pixel_max (int): Upper bound to end cut.
    Returns:
        final_array: 3d array of data from each fits file.
    '''
    lower_index = (-1) + y_pixel_min
    upper_index = (-1) + y_pixel_max
    pixel_data_array_cut = pixel_data_array[lower_index:upper_index,:]
    if print_shape: print(pixel_data_array_cut.shape)
    return pixel_data_array_cut

def display_2d_array(pixel_data_array, figsize=(7,3), lower_percentile=1, upper_percentile=99,title='2D Array', cmap='g
    '''
    Displays a 2d array of pixel data. Useful to see the image contained in a FITS file,
    or to verify successful array data handling.
    Parameters:
        pixel_data_array (2d array): Array of pixel data to display.
        lower_percentile (int): Adjusts the pixel scale lower bound; default = 1
        upper_percentile (int): Adjusts the pixel scale upper bound; default = 99
    '''
    fig, ax = plt.subplots(figsize=figsize)
    minimum = np.percentile(pixel_data_array,lower_percentile)
    maximum = np.percentile(pixel_data_array,upper_percentile)
    img = ax.imshow(pixel_data_array, cmap=cmap, origin = 'lower', vmin=minimum, vmax=maximum)
    cbar = plt.colorbar(img, ax=ax, orientation='horizontal', pad=0.1)
    cbar.set_label('Pixel Counts/Intensity')
    if overlay_x != []:
        plt.scatter(overlay_x,overlay_y,color='r', s=5)
    plt.title(title)

def darks_correction(mcombine_array, darks_mcombine_array):
    return mcombine_array - darks_mcombine_array

def darks_correction_uncertianty(mcombine_unc_array, darks_mcombine_unc_array):
    '''
    Can also be used for multiplication/division of two quantities w uncertainties
    '''
    return np.sqrt(mcombine_unc_array**2 + darks_mcombine_unc_array**2)

def flat_correction_uncertainty(sci,sci_uncert,flat,flat_uncert):
    radical = ((sci*flat_uncert) / flat**2)**2 + (sci_uncert/flat)**2
    return np.sqrt(radical)

def quadrature_unc(uncertainties_array, axis=0):
    return np.sqrt(np.nansum(np.square(uncertainties_array),axis=axis))

def calibrate_fits_files(fits_file_list=[], output_3d_only=False, output_mcombine_only=True, output_darks_corrected=Tru
                         mcombine_array=None, darks_array=None, display_array=False):
    '''
    Function is multifunctional. It can:
    1) Output a 3d array
    2) Output a median combine (2d)
    3) Output a darks corrected array (2d)

    fits_file_list([]) : List containing paths of FITS files to convert.
    output_3d_only(True/False) : Output only 3d array
    output_mcombine_only(True/False) : Output only mcombine array
    output_darks_corrected(True/False) : Output only darks corrected array
    mcombine_array :
    darks_array :
    display_array(True/False) :
    '''
    if output_3d_only:
        array_3d = fits_data_to_3d_array(fits_file_list)
        return array_3d
    elif output_mcombine_only:
        array_3d = fits_data_to_3d_array(fits_file_list)
        array_mcombine = median_combine(array_3d)
        array_std = np.std(array_3d, axis=0) / np.sqrt(np.shape(array_3d)[0])
        if display_array:
            display_2d_array(array_mcombine)
        return array_mcombine, array_std
    elif output_darks_corrected:
        final_array = darks_correction(mcombine_array - darks_array)
        if display_array:
            display_2d_array(final_array)
        return final_array

def mad(data, axis=0):
    """
    Calculate the Median Absolute Deviation (MAD) of the given data along the specified axis.
    Parameters:
    - data: 2D array of data
    - axis: Axis along which to compute the MAD (default is 0 for columns)
    Returns:
    - mad_values: MAD for each column (1D array)
    """
    medians = np.median(data, axis=axis)
    #absolute deviations from the median
    abs_devs = np.abs(data - medians[np.newaxis, :])
    #median of the absolute deviations
    mad_values = np.median(abs_devs, axis=axis)
    return mad_values

def median_uncertainty(data, axis=0):
    """
    Calculate the uncertainty of the median for each column in the data, using the MAD.
    Parameters:
    - data: 2D array of data
    - axis: Axis along which to compute the median uncertainty (default is 0 for columns)
    Returns:
    - uncertainty: Uncertainty of the median for each column (1D array)
    """
    mad_values = mad(data, axis=axis)
    uncertainty = 1.4826 * mad_values / np.sqrt(data.shape[0])
    return uncertainty

def normalize_to_unity(array_2d, ctype='sum', axis=0, array_uncert=[], return_uncert=False):
    if ctype == 'sum':
        factor = np.max(np.nansum(array_2d, axis=axis))
        if return_uncert:
            return quadrature_unc(array_uncert, axis=0) / factor
        else:
            return np.nansum(array_2d, axis=axis) / factor
    elif ctype == 'mean':
        factor = np.max(np.mean(array_2d, axis=axis))
        if return_uncert:
            sigma = np.std(array_uncert, axis=0) / np.sqrt(array_uncert.shape[0])
            return  sigma / factor
        else:
            return np.mean(array_2d, axis=axis) / factor
    elif ctype == 'median':
        factor = np.max(np.median(array_2d, axis=axis))
        if return_uncert:
            return median_uncertainty(array_uncert) / factor
        else:
            return np.median(array_2d, axis=axis) / factor

def normalize_to_unity_1d(array_1d, array_uncert_1d):
    '''
    Returns:
    - normalized_array, normalized_uncert_array
    '''
    factor = np.nanmax(array_1d)
    normalized_array = array_1d / factor
    normalized_uncert_array = array_uncert_1d / factor
    return normalized_array, normalized_uncert_array

def display_cut_spectrum(pixel_data_array, figsize=(12,5), lower_percentile=1, upper_percentile=99, title='2D Array', c
    '''
    Displays a 2d array of pixel data. Useful to see the image contained in a FITS file,
    or to verify successful array data handling.
    Parameters:
        pixel_data_array (2d array): Array of pixel data to display.
        lower_percentile (int): Adjusts the pixel scale lower bound; default = 1
        upper_percentile (int): Adjusts the pixel scale upper bound; default = 99
    '''
    fig, ax = plt.subplots(figsize=figsize)
    minimum = np.percentile(pixel_data_array,lower_percentile)
    maximum = np.percentile(pixel_data_array,upper_percentile)
    img = ax.imshow(pixel_data_array, cmap=cmap, origin = 'lower', vmin=minimum, vmax=maximum)
    #seismic RdGy gist_gray
    cbar = plt.colorbar(img, ax=ax, orientation='horizontal', pad=0.1, fraction=.05, aspect=50)
    cbar.set_label('Pixel Counts/Intensity')
    fig.tight_layout
    plt.title(title)
    plt.plot()

def cut_pixel_data_array_3d(pixel_data_array, y_pixel_min, y_pixel_max):
    '''
    Returns an array of pixel data, cut to a specified vertical (y-pixel) range.
    Parameters:
        pixel_data_array (2d array): Array of pixel data to create a cut from.
        y_pixel_min (int): Lower bound to start cut.
        y_pixel_max (int): Upper bound to end cut.
    Returns:
        final_array: 3d array of data from each fits file.
    '''
    lower_index = (-1) + y_pixel_min
    upper_index = (-1) + y_pixel_max
    pixel_data_array_cut = pixel_data_array[:, lower_index:upper_index, :]
    return pixel_data_array_cut

def plot_spectrum(spectrum_label, index_set, invert_spectrum=False, find_peaks=False, plot_Habg_balmer=[False, False, F
                  xlim=[], ylim=[], legend_loc='upper right', peaks_height=(0, 0.35), dy=0):
    '''
    spectrum_label : Ne_set1 Ne_set2 Ha_set1 Ha_set2 Ar_set1 Ar_set2 Hbg_set1 Hbg_set2
    index_set : Ne_x_index_set1 Ne_x_index_set2 Ar_x_index_set1 Ar_x_index_set2
    '''
    if invert_spectrum:
        c = -1.0
        d = 1.0
    else:
        c = 1.0
        d = 0.0
    spectrum_index = np.arange(len(normalized_spectra[spectrum_label]))  # pixel index
    spectrum_wavelength_index = pixel_to_wavelength_index(spectrum_index, *calibration_stats[index_set])  # pixel index

    if find_peaks:
        # Find Peaks
        peaks, _ = find_peaks(normalized_spectra[spectrum_label], height=peaks_height)
        # Annotate peaks
        for peak in peaks:
            plt.annotate(f'{spectrum_wavelength_index[peak]:.1f} nm',
                         (spectrum_wavelength_index[peak], normalized_spectra[spectrum_label][peak]),
                         xytext=(0, 4), textcoords='offset points', ha='center', fontsize=7, color='red')

    plt.plot(spectrum_wavelength_index, c*normalized_spectra[spectrum_label]+d+dy, label=spectrum_label + ' Calibrated
    plt.ylabel('Intensity')
    plt.xlabel('Wavelength (nm)')
    plt.yticks(np.arange(0.0, 1.1, 0.1))

    if xlim != []:
        plt.xlim(xlim)
    if ylim != []:
        plt.ylim(ylim)

    # Plot Balmer Lines:
    if plot_Habg_balmer[0]:
        plt.vlines(x=656.3, ymin=0, ymax=1, color='r', label="Hα 656.3 nm")
    if plot_Habg_balmer[1]:
        plt.vlines(x=486.1, ymin=0, ymax=1, color='cyan', label="Hβ 486.1 nm")
    if plot_Habg_balmer[2]:
        plt.vlines(x=434, ymin=0, ymax=1, color='blue', label="Hγ 434 nm")

    plt.legend(loc=legend_loc)
    plt.grid(True)
```

Creating Calibrated Science Images

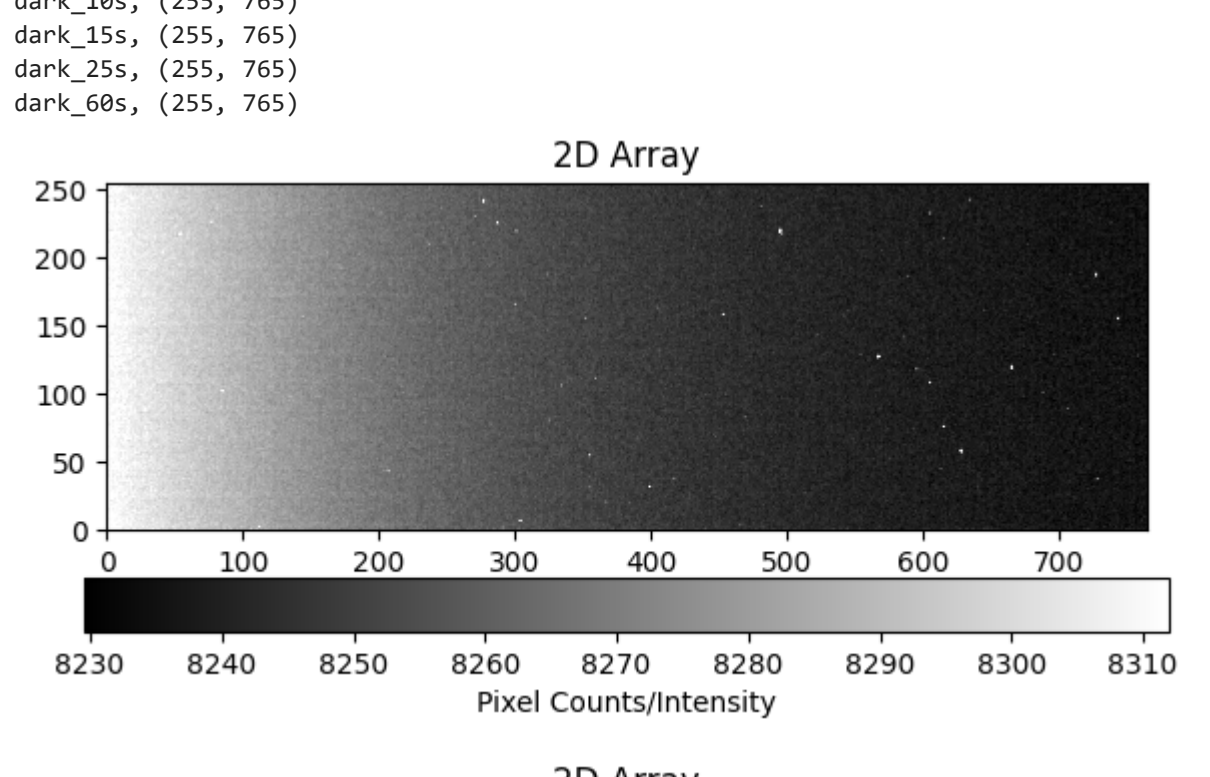
Creating Median Combine Dicts

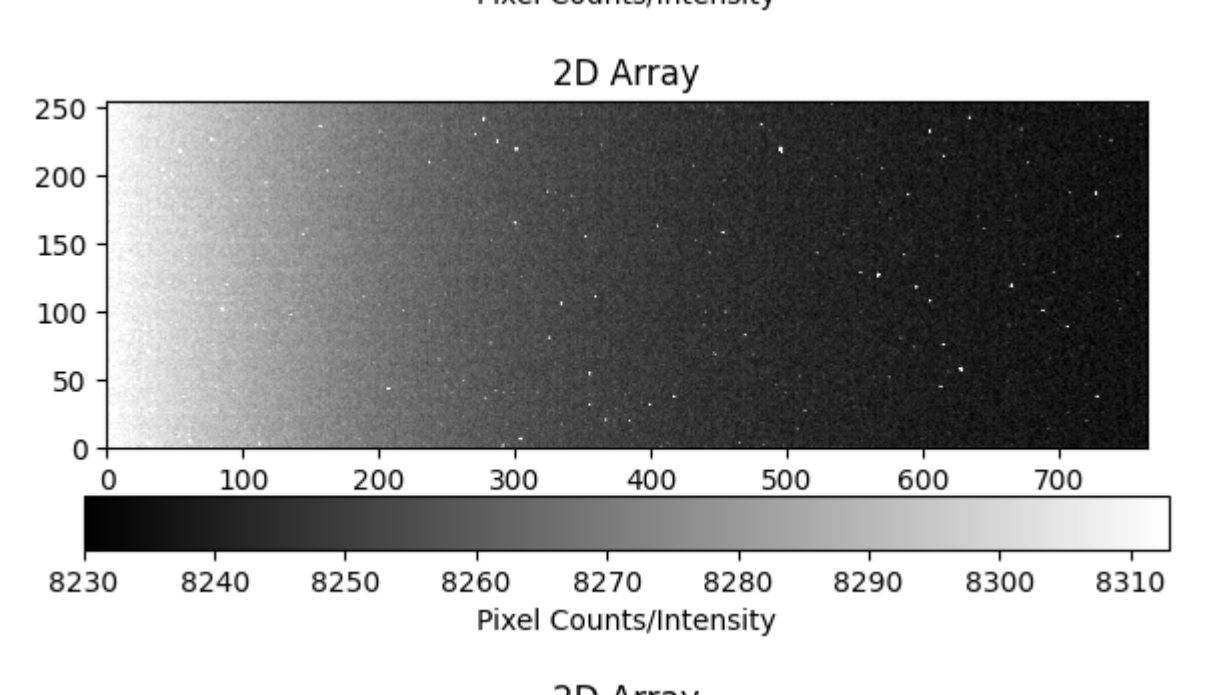
Darks

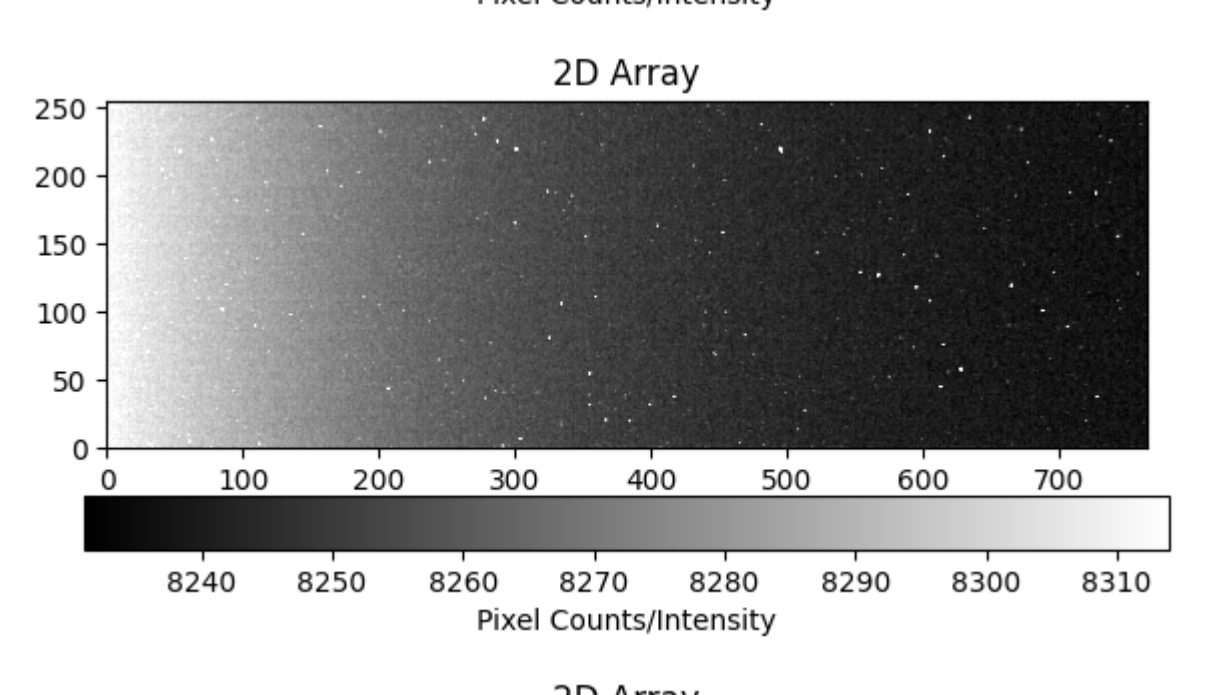
In [3]:

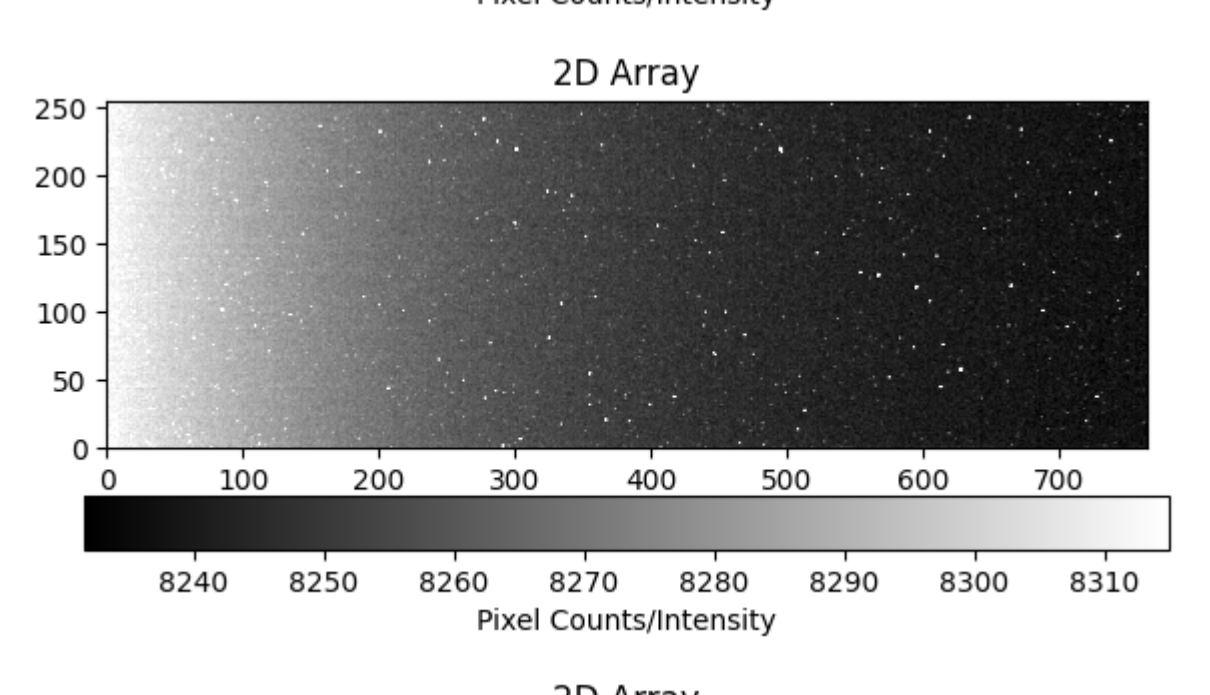
```
show_files = True
show_arrays = True
darks_exposure_times = ['dark_3s', 'dark_10s', 'dark_15s', 'dark_25s', 'dark_60s']
darks_mcombines = {}
darks_mcombines_unc = {}

for exposure in darks_exposure_times:
    fits_files = glob_files('night_2', exposure, show=show_files)
    darks_mcombines[exposure], darks_mcombines_unc[exposure] = calibrate_fits_files(fits_file_list=fits_files, display_
    #display_2d_array(darks_mcombines_unc[exposure])
    #plt.title(f'{exposure} uncertainty')
    #print(exposure, np.max(darks_mcombines_unc[exposure]))

for i in darks_mcombines:
    print(f"Avg pixel value in {i}: {np.mean(darks_mcombines[i])}")

print('Darks Median Combines')
for key, value in darks_mcombines.items():
    print(f"{key}, {np.shape(value)}")
```

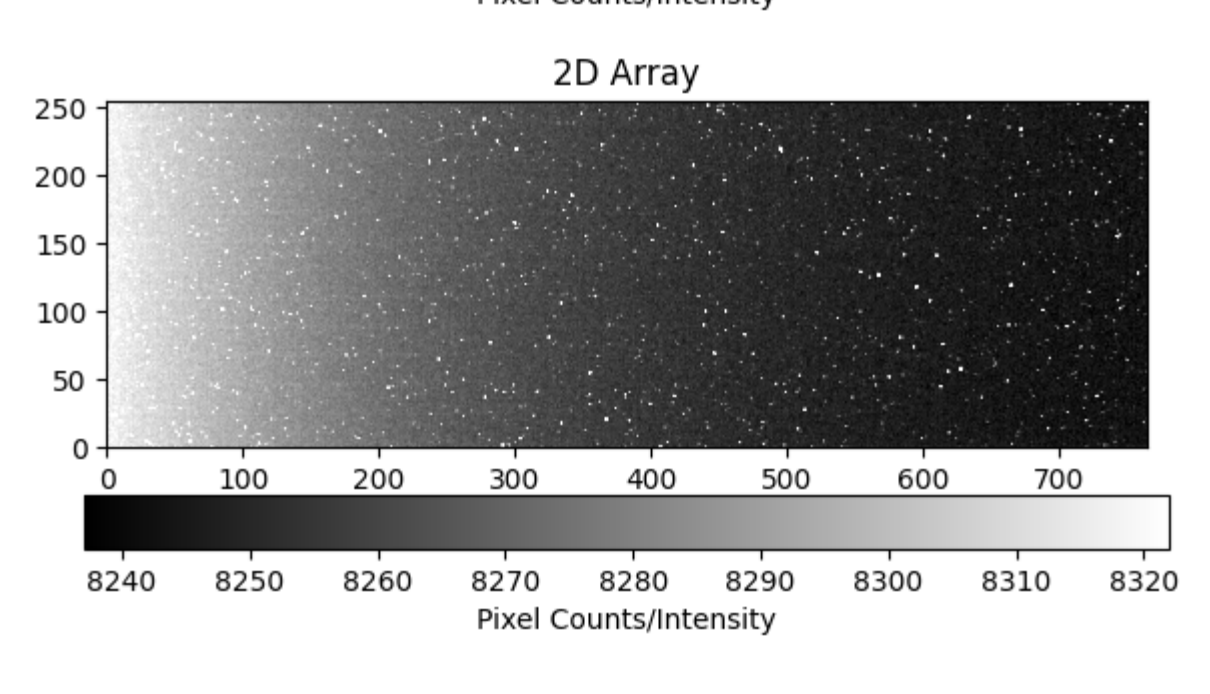
```
Number of Files: 10
night_2\dark_3s.00000000.DARK.FIT
night_2\dark_3s.00000001.DARK.FIT
night_2\dark_3s.00000002.DARK.FIT
night_2\dark_3s.00000003.DARK.FIT
night_2\dark_3s.00000004.DARK.FIT
night_2\dark_3s.00000005.DARK.FIT
night_2\dark_3s.00000006.DARK.FIT
night_2\dark_3s.00000007.DARK.FIT
night_2\dark_3s.00000008.DARK.FIT
night_2\dark_3s.00000009.DARK.FIT
Number of Files: 10
night_2\dark_10s.00000000.DARK.FIT
night_2\dark_10s.00000001.DARK.FIT
night_2\dark_10s.00000002.DARK.FIT
night_2\dark_10s.00000003.DARK.FIT
night_2\dark_10s.00000004.DARK.FIT
night_2\dark_10s.00000005.DARK.FIT
night_2\dark_10s.00000006.DARK.FIT
night_2\dark_10s.00000007.DARK.FIT
night_2\dark_10s.00000008.DARK.FIT
night_2\dark_10s.00000009.DARK.FIT
Number of Files: 10
night_2\dark_15s.00000000.DARK.FIT
night_2\dark_15s.00000001.DARK.FIT
night_2\dark_15s.00000002.DARK.FIT
night_2\dark_15s.00000003.DARK.FIT
night_2\dark_15s.00000004.DARK.FIT
night_2\dark_15s.00000005.DARK.FIT
night_2\dark_15s.00000006.DARK.FIT
night_2\dark_15s.00000007.DARK.FIT
night_2\dark_15s.00000008.DARK.FIT
night_2\dark_15s.00000009.DARK.FIT
Number of Files: 10
night_2\dark_25s.00000000.DARK.FIT
night_2\dark_25s.00000001.DARK.FIT
night_2\dark_25s.00000002.DARK.FIT
night_2\dark_25s.00000003.DARK.FIT
night_2\dark_25s.00000004.DARK.FIT
night_2\dark_25s.00000005.DARK.FIT
night_2\dark_25s.00000006.DARK.FIT
night_2\dark_25s.00000007.DARK.FIT
night_2\dark_25s.00000008.DARK.FIT
night_2\dark_25s.00000009.DARK.FIT
Number of Files: 10
night_2\dark_60s.00000000.DARK.FIT
night_2\dark_60s.00000001.DARK.FIT
night_2\dark_60s.00000002.DARK.FIT
night_2\dark_60s.00000003.DARK.FIT
night_2\dark_60s.00000004.DARK.FIT
night_2\dark_60s.00000005.DARK.FIT
night_2\dark_60s.00000006.DARK.FIT
night_2\dark_60s.00000007.DARK.FIT
night_2\dark_60s.00000008.DARK.FIT
night_2\dark_60s.00000009.DARK.FIT
Avg pixel value in dark_3s: 8257.340351146995
Avg pixel value in dark_10s: 8258.111749327181
Avg pixel value in dark_15s: 8258.985743944637
Avg pixel value in dark_25s: 8259.849114443163
Avg pixel value in dark_60s: 8266.209042675893
Darks Median Combines
dark_3s, (255, 765)
dark_10s, (255, 765)
dark_15s, (255, 765)
dark_25s, (255, 765)
dark_60s, (255, 765)
```









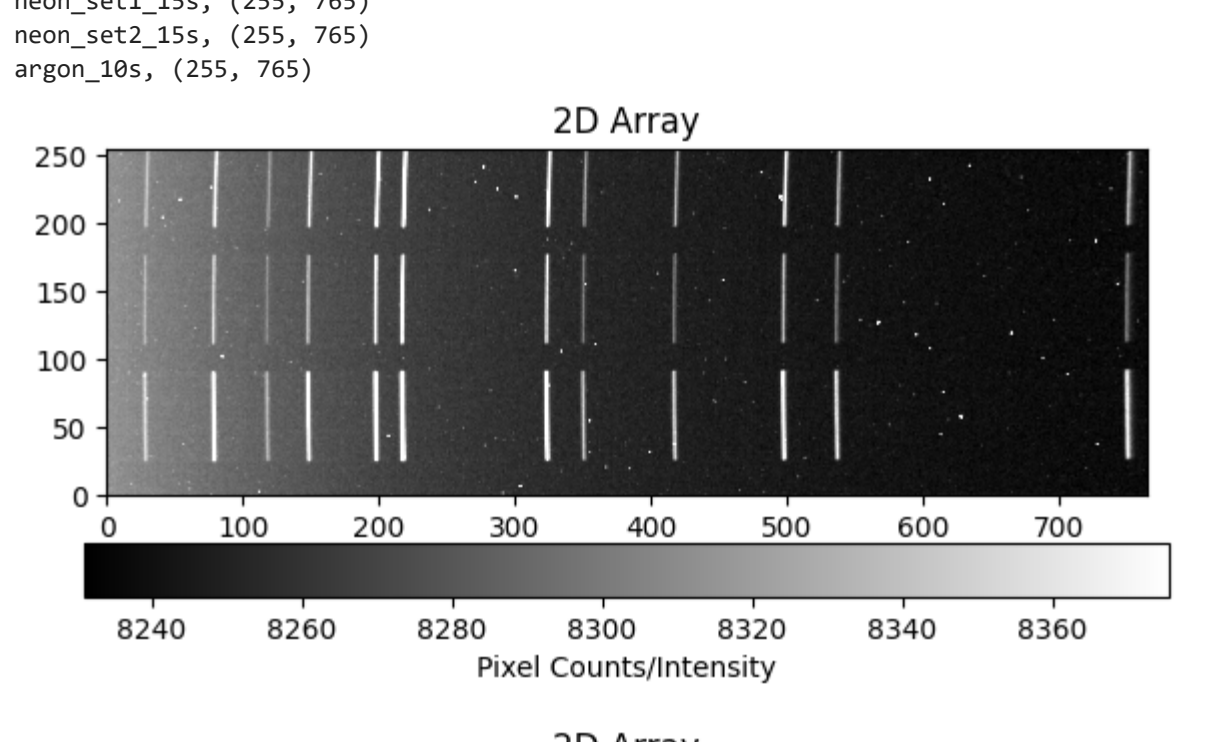


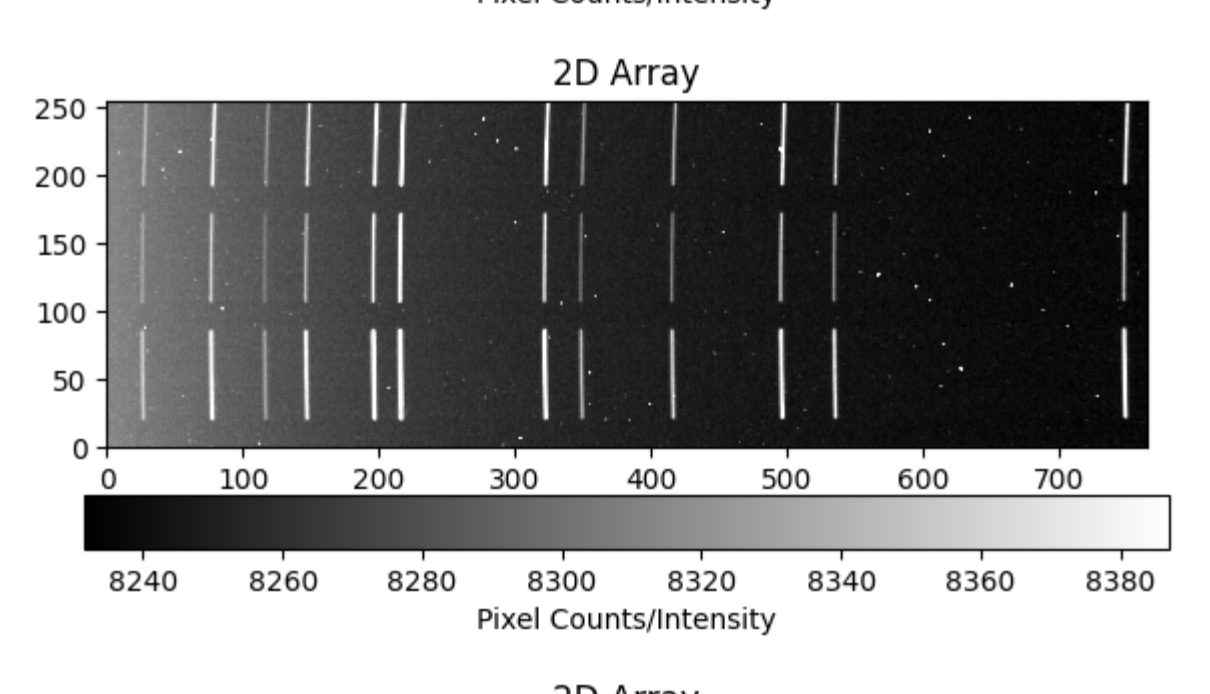
Calibration Spectra

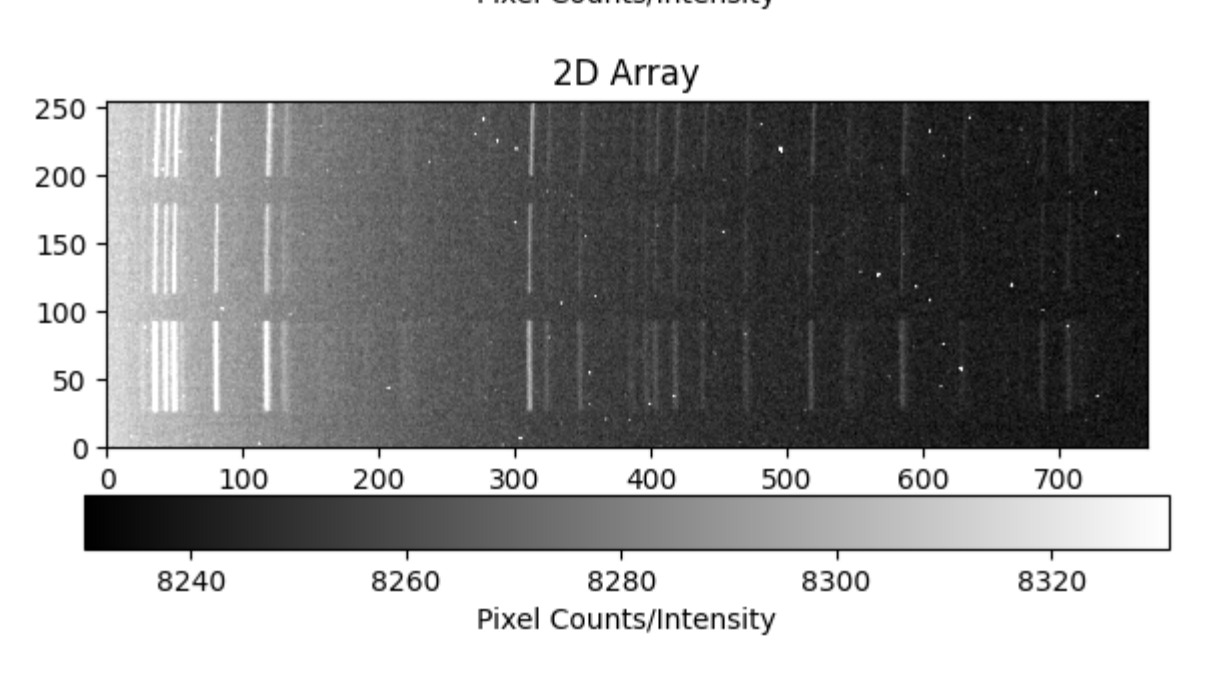
In [4]:

```
show_files = True
show_arrays = True
spectra = ['ha_neon_calibration_spectrum_set1', 'ha_neon_calibration_spectrum_set2_15s', 'hbg_argon_calibration_spectru
spectra_calib = {}
spectra_calib_unc = {}

for spectrum in spectra:
    fits_files = glob_files('night_2', spectrum, show=show_files)
    spectra_calib[spectrum], spectra_calib_unc[spectrum] = calibrate_fits_files(fits_file_list=fits_files, display_arra
    #display_2d_array(spectra_calib_unc[spectrum])
new_keys = {
    'ha_neon_calibration_spectrum_set1': 'neon_set1_15s',
    'ha_neon_calibration_spectrum_set2_15s': 'neon_set2_15s',
    'hbg_argon_calibration_spectrum_sets1and2': 'argon_10s'
}
for dictionary in [spectra_calib, spectra_calib_unc]:
    updated_dict = {new_keys.get(old_key, old_key): value for old_key, value in dictionary.items()}
    dictionary.clear()
    dictionary.update(updated_dict)

print('Spectra Median Combines')
for key, value in spectra_calib.items():
    print(f"{key}, {np.shape(value)}")
```

```
Number of Files: 10
night_2\ha_neon_calibration_spectrum_set1_15s.00000000.FIT
night_2\ha_neon_calibration_spectrum_set1_15s.00000001.FIT
night_2\ha_neon_calibration_spectrum_set1_15s.00000002.FIT
night_2\ha_neon_calibration_spectrum_set1_15s.00000003.FIT
night_2\ha_neon_calibration_spectrum_set1_15s.00000004.FIT
night_2\ha_neon_calibration_spectrum_set1_15s.00000005.FIT
night_2\ha_neon_calibration_spectrum_set1_15s.00000006.FIT
night_2\ha_neon_calibration_spectrum_set1_15s.00000007.FIT
night_2\ha_neon_calibration_spectrum_set1_15s.00000008.FIT
night_2\ha_neon_calibration_spectrum_set1_15s.00000009.FIT
Number of Files: 10
night_2\ha_neon_calibration_spectrum_set2_15s.00000000.FIT
night_2\ha_neon_calibration_spectrum_set2_15s.00000001.FIT
night_2\ha_neon_calibration_spectrum_set2_15s.00000002.FIT
night_2\ha_neon_calibration_spectrum_set2_15s.00000003.FIT
night_2\ha_neon_calibration_spectrum_set2_15s.00000004.FIT
night_2\ha_neon_calibration_spectrum_set2_15s.00000005.FIT
night_2\ha_neon_calibration_spectrum_set2_15s.00000006.FIT
night_2\ha_neon_calibration_spectrum_set2_15s.00000007.FIT
night_2\ha_neon_calibration_spectrum_set2_15s.00000008.FIT
night_2\ha_neon_calibration_spectrum_set2_15s.00000009.FIT
Number of Files: 5
night_2\hbg_argon_calibration_spectrum_sets1and2_10s.00000000.FIT
night_2\hbg_argon_calibration_spectrum_sets1and2_10s.00000001.FIT
night_2\hbg_argon_calibration_spectrum_sets1and2_10s.00000002.FIT
night_2\hbg_argon_calibration_spectrum_sets1and2_10s.00000003.FIT
night_2\hbg_argon_calibration_spectrum_sets1and2_10s.00000004.FIT
Spectra Median Combines
neon_set1_15s, (255, 765)
neon_set2_15s, (255, 765)
argon_10s, (255, 765)
```





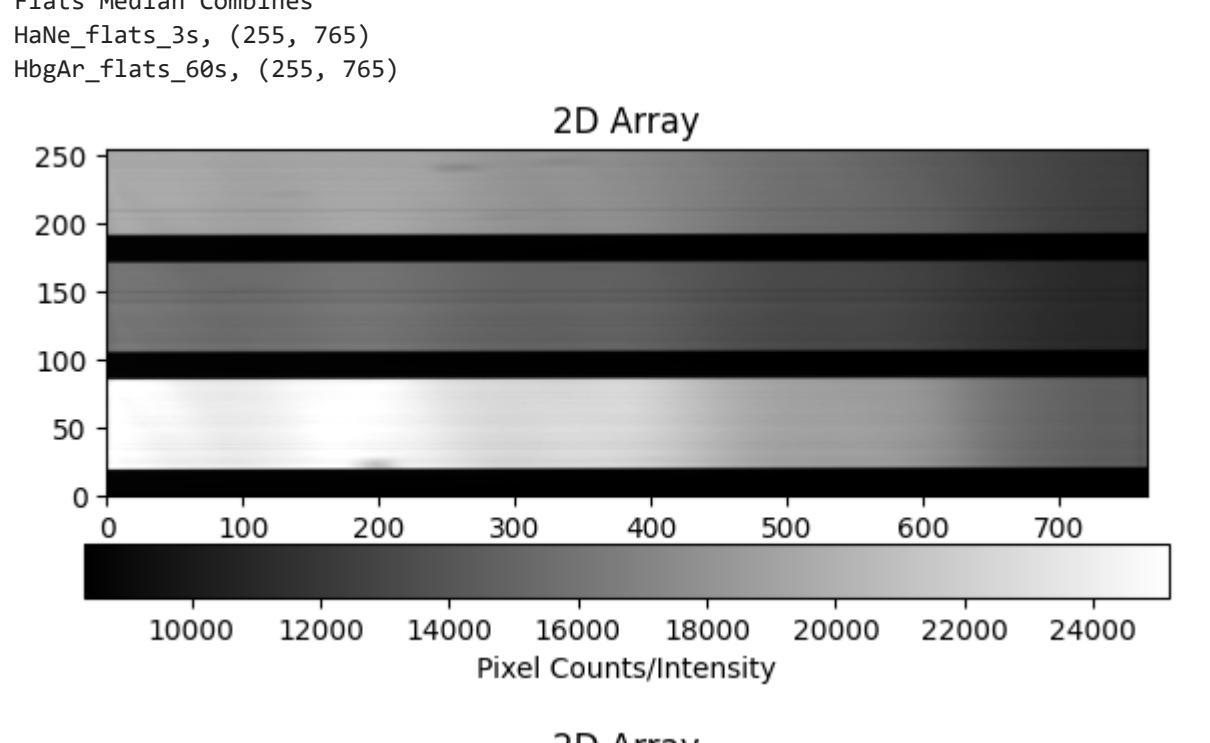


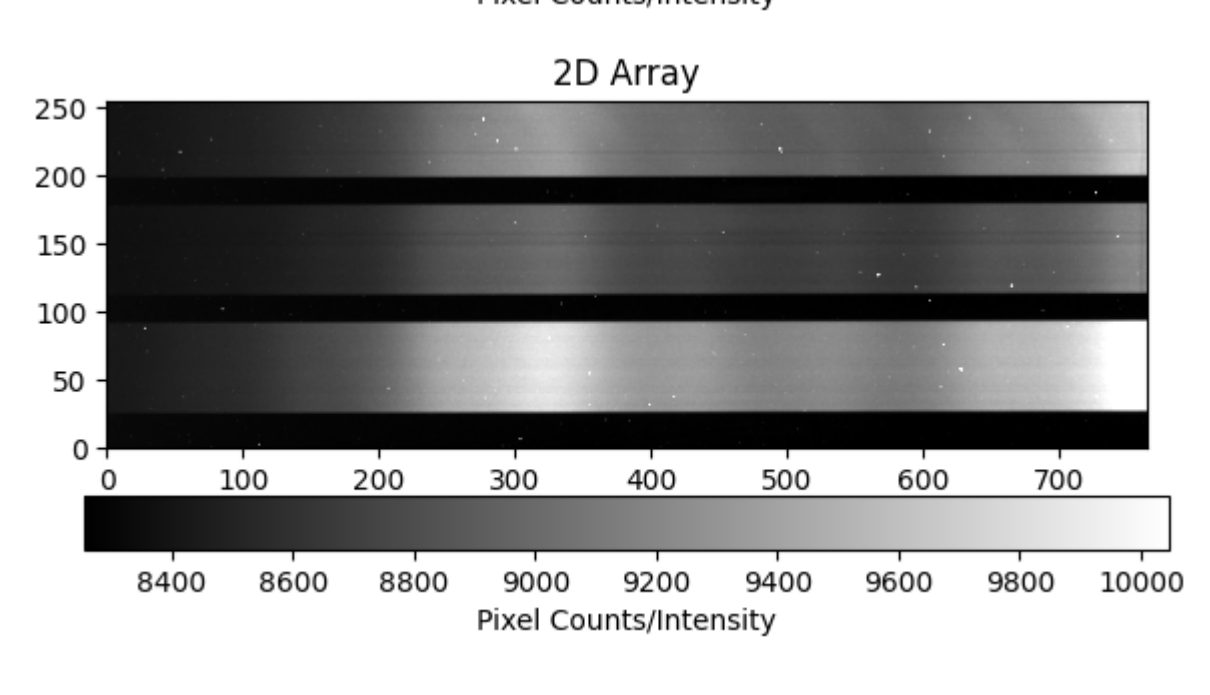
Flats

In [5]:

```
show_files = True
show_arrays = True
flats_names = ['Ha_Neon_flats_3s', 'hbg_flat_spectrum_sets1and2']
flats_mcombines = {}
flats_mcombines_unc = {}

for name in flats_names:
    fits_files = glob_files('night_2', name, show=show_files)
    flats_mcombines[name], flats_mcombines_unc[name] = calibrate_fits_files(fits_file_list=fits_files, display_array=sh
    #display_2d_array(flats_mcombines_unc[name])

new_keys = {
    'Ha_Neon_flats_3s': 'HaNe_flats_3s',
    'hbg_flat_spectrum_sets1and2': 'HbgAr_flats_60s',
}
for dictionary in [flats_mcombines, flats_mcombines_unc]:
    updated_dict = {new_keys.get(old_key, old_key): value for old_key, value in dictionary.items()}
    dictionary.clear()
    dictionary.update(updated_dict)

print('Flats Median Combines')
for key, value in flats_mcombines.items():
    print(f"{key}, {np.shape(value)}")
```

```
Number of Files: 10
night_2\Ha_Neon_flats_3s.00000000.FIT
night_2\Ha_Neon_flats_3s.00000001.FIT
night_2\Ha_Neon_flats_3s.00000002.FIT
night_2\Ha_Neon_flats_3s.00000003.FIT
night_2\Ha_Neon_flats_3s.00000004.FIT
night_2\Ha_Neon_flats_3s.00000005.FIT
night_2\Ha_Neon_flats_3s.00000006.FIT
night_2\Ha_Neon_flats_3s.00000007.FIT
night_2\Ha_Neon_flats_3s.00000008.FIT
night_2\Ha_Neon_flats_3s.00000009.FIT
Number of Files: 5
night_2\hbg_flat_spectrum_sets1and2_60s.00000000.FIT
night_2\hbg_flat_spectrum_sets1and2_60s.00000001.FIT
night_2\hbg_flat_spectrum_sets1and2_60s.00000002.FIT
night_2\hbg_flat_spectrum_sets1and2_60s.00000003.FIT
night_2\hbg_flat_spectrum_sets1and2_60s.00000004.FIT
Flats Median Combines
HaNe_flats_3s, (255, 765)
HbgAr_flats_60s, (255, 765)
```



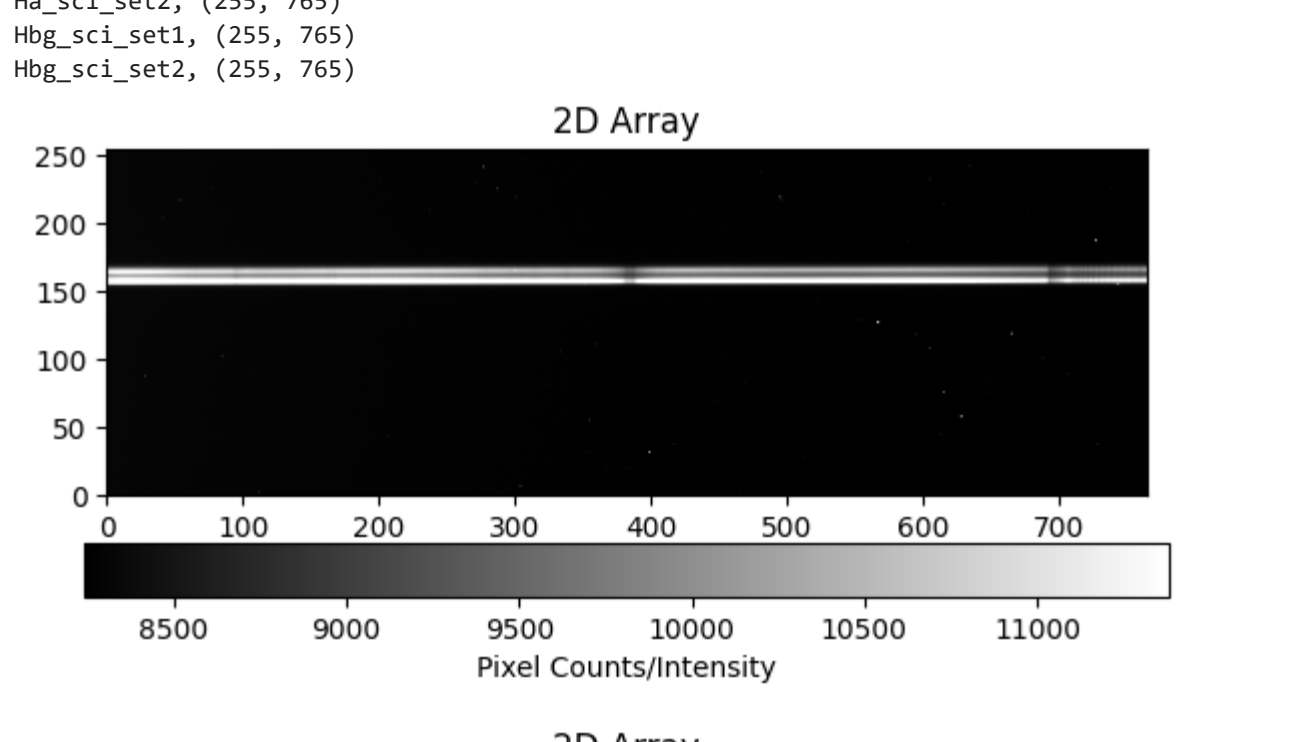


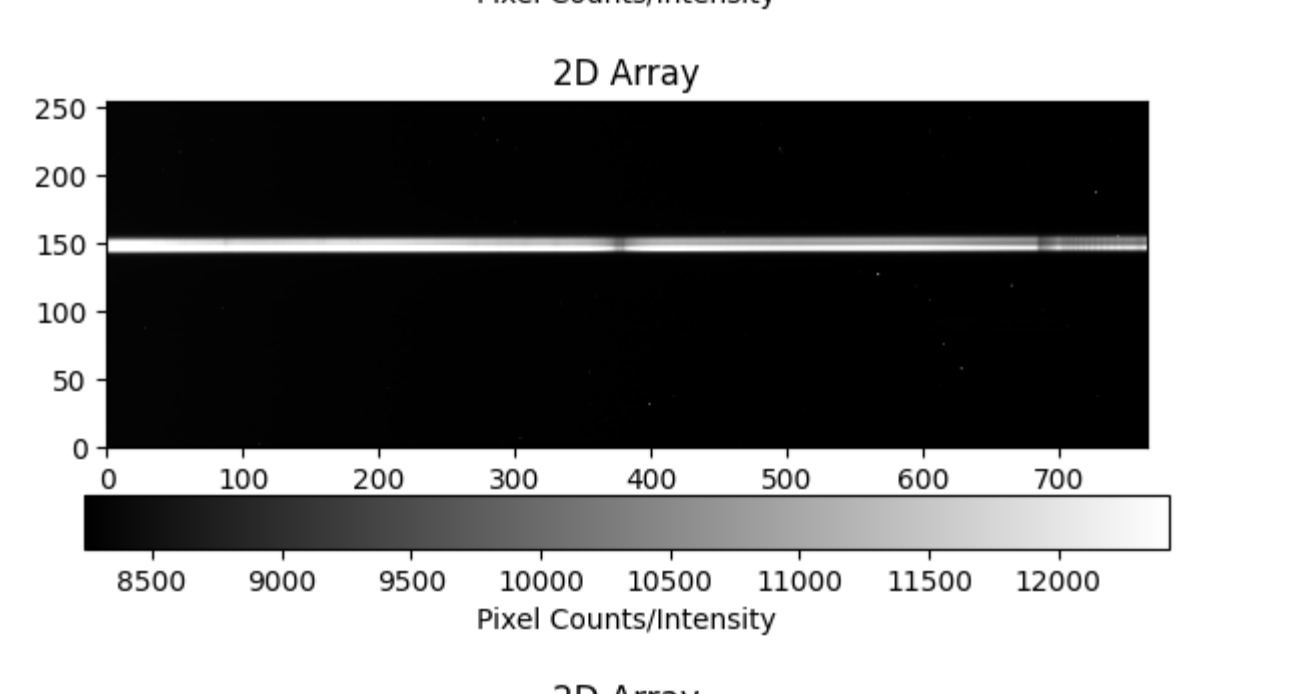
Science Frames

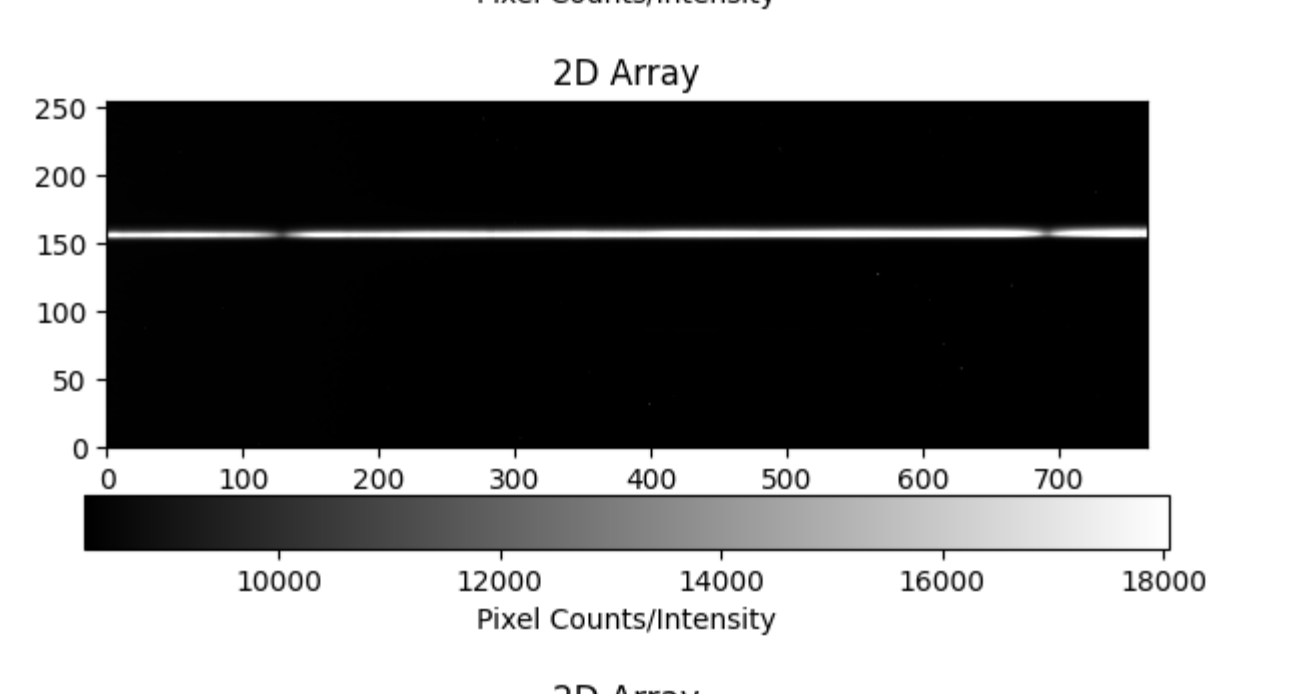
In [6]:

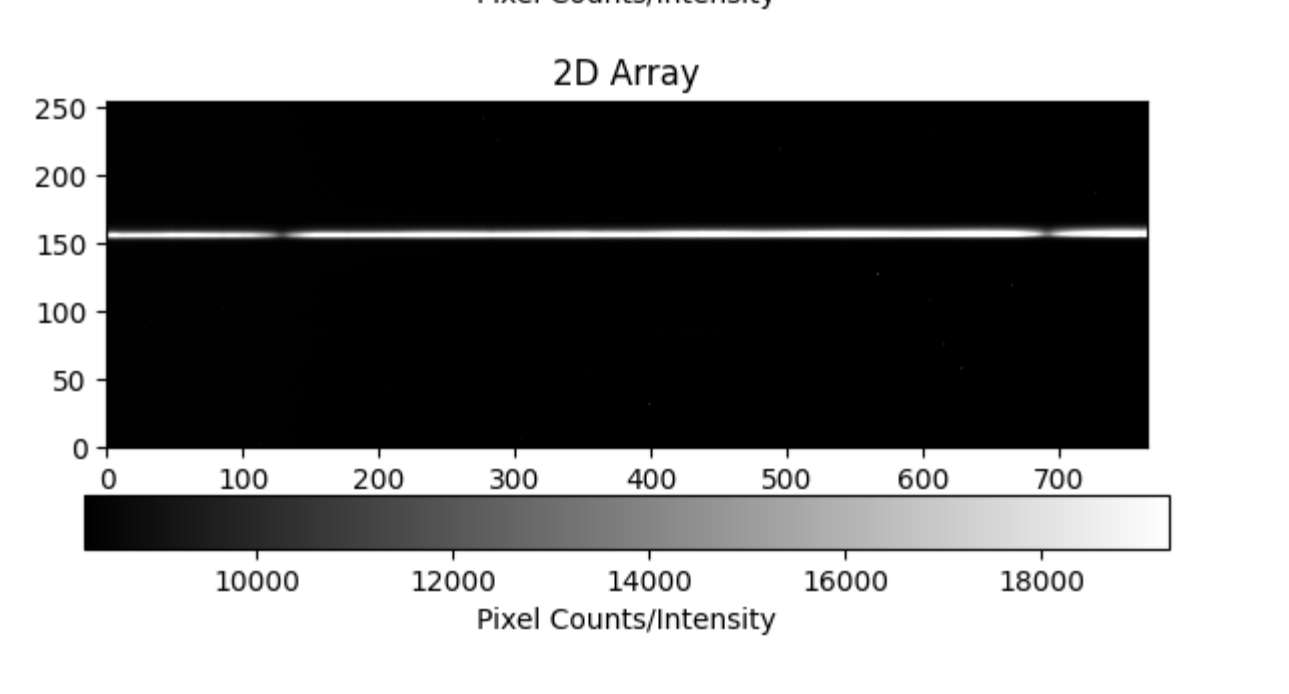
```
show_files = True
show_arrays = True
names = ['ha_lights_set1', 'ha_lights_set2', 'hbg_light_set1', 'hbg_light_set2']
sci_mcombines = {}
sci_mcombines_unc = {}
for name in names:
    fits_files = glob_files('night_2', name, show=show_files)
    sci_mcombines[name], sci_mcombines_unc[name] = calibrate_fits_files(fits_file_list=fits_files, display_array=show_a
    #display_2d_array(sci_mcombines_unc[name])

new_keys = {
    'ha_lights_set1': 'Ha_sci_set1',
    'ha_lights_set2': 'Ha_sci_set2',
    'hbg_light_set1': 'Hbg_sci_set1',
    'hbg_light_set2': 'Hbg_sci_set2',
}
for dictionary in [sci_mcombines, sci_mcombines_unc]:
    updated_dict = {new_keys.get(old_key, old_key): value for old_key, value in dictionary.items()}
    dictionary.clear()
    dictionary.update(updated_dict)

print('Science Median Combines')
for key, value in sci_mcombines.items():
    print(f"{key}, {np.shape(value)}")
```

```
Number of Files: 20
night_2\ha_lights_set1_25s.00000000.FIT
night_2\ha_lights_set1_25s.00000001.FIT
night_2\ha_lights_set1_25s.00000002.FIT
night_2\ha_lights_set1_25s.00000003.FIT
night_2\ha_lights_set1_25s.00000004.FIT
night_2\ha_lights_set1_25s.00000005.FIT
night_2\ha_lights_set1_25s.00000006.FIT
night_2\ha_lights_set1_25s.00000007.FIT
night_2\ha_lights_set1_25s.00000008.FIT
night_2\ha_lights_set1_25s.00000009.FIT
night_2\ha_lights_set1_25s.00000010.FIT
night_2\ha_lights_set1_25s.00000011.FIT
night_2\ha_lights_set1_25s.00000012.FIT
night_2\ha_lights_set1_25s.00000013.FIT
night_2\ha_lights_set1_25s.00000014.FIT
night_2\ha_lights_set1_25s.00000015.FIT
night_2\ha_lights_set1_25s.00000016.FIT
night_2\ha_lights_set1_25s.00000017.FIT
night_2\ha_lights_set1_25s.00000018.FIT
night_2\ha_lights_set1_25s.00000019.FIT
Number of Files: 20
night_2\ha_lights_set2_25s.00000000.FIT
night_2\ha_lights_set2_25s.00000001.FIT
night_2\ha_lights_set2_25s.00000002.FIT
night_2\ha_lights_set2_25s.00000003.FIT
night_2\ha_lights_set2_25s.00000004.FIT
night_2\ha_lights_set2_25s.00000005.FIT
night_2\ha_lights_set2_25s.00000006.FIT
night_2\ha_lights_set2_25s.00000007.FIT
night_2\ha_lights_set2_25s.00000008.FIT
night_2\ha_lights_set2_25s.00000009.FIT
night_2\ha_lights_set2_25s.00000010.FIT
night_2\ha_lights_set2_25s.00000011.FIT
night_2\ha_lights_set2_25s.00000012.FIT
night_2\ha_lights_set2_25s.00000013.FIT
night_2\ha_lights_set2_25s.00000014.FIT
night_2\ha_lights_set2_25s.00000015.FIT
night_2\ha_lights_set2_25s.00000016.FIT
night_2\ha_lights_set2_25s.00000017.FIT
night_2\ha_lights_set2_25s.00000018.FIT
night_2\ha_lights_set2_25s.00000019.FIT
Number of Files: 20
night_2\hbg_light_set1_25s.00000000.FIT
night_2\hbg_light_set1_25s.00000001.FIT
night_2\hbg_light_set1_25s.00000002.FIT
night_2\hbg_light_set1_25s.00000003.FIT
night_2\hbg_light_set1_25s.00000004.FIT
night_2\hbg_light_set1_25s.00000005.FIT
night_2\hbg_light_set1_25s.00000006.FIT
night_2\hbg_light_set1_25s.00000007.FIT
night_2\hbg_light_set1_25s.00000008.FIT
night_2\hbg_light_set1_25s.00000009.FIT
night_2\hbg_light_set1_25s.00000010.FIT
night_2\hbg_light_set1_25s.00000011.FIT
night_2\hbg_light_set1_25s.00000012.FIT
night_2\hbg_light_set1_25s.00000013.FIT
night_2\hbg_light_set1_25s.00000014.FIT
night_2\hbg_light_set1_25s.00000015.FIT
night_2\hbg_light_set1_25s.00000016.FIT
night_2\hbg_light_set1_25s.00000017.FIT
night_2\hbg_light_set1_25s.00000018.FIT
night_2\hbg_light_set1_25s.00000019.FIT
Number of Files: 20
night_2\hbg_light_set2_25s.00000000.FIT
night_2\hbg_light_set2_25s.00000001.FIT
night_2\hbg_light_set2_25s.00000002.FIT
night_2\hbg_light_set2_25s.00000003.FIT
night_2\hbg_light_set2_25s.00000004.FIT
night_2\hbg_light_set2_25s.00000005.FIT
night_2\hbg_light_set2_25s.00000006.FIT
night_2\hbg_light_set2_25s.00000007.FIT
night_2\hbg_light_set2_25s.00000008.FIT
night_2\hbg_light_set2_25s.00000009.FIT
night_2\hbg_light_set2_25s.00000010.FIT
night_2\hbg_light_set2_25s.00000011.FIT
night_2\hbg_light_set2_25s.00000012.FIT
night_2\hbg_light_set2_25s.00000013.FIT
night_2\hbg_light_set2_25s.00000014.FIT
night_2\hbg_light_set2_25s.00000015.FIT
night_2\hbg_light_set2_25s.00000016.FIT
night_2\hbg_light_set2_25s.00000017.FIT
night_2\hbg_light_set2_25s.00000018.FIT
night_2\hbg_light_set2_25s.00000019.FIT
Science Median Combines
Ha_sci_set1, (255, 765)
Ha_sci_set2, (255, 765)
Hbg_sci_set1, (255, 765)
Hbg_sci_set2, (255, 765)
```





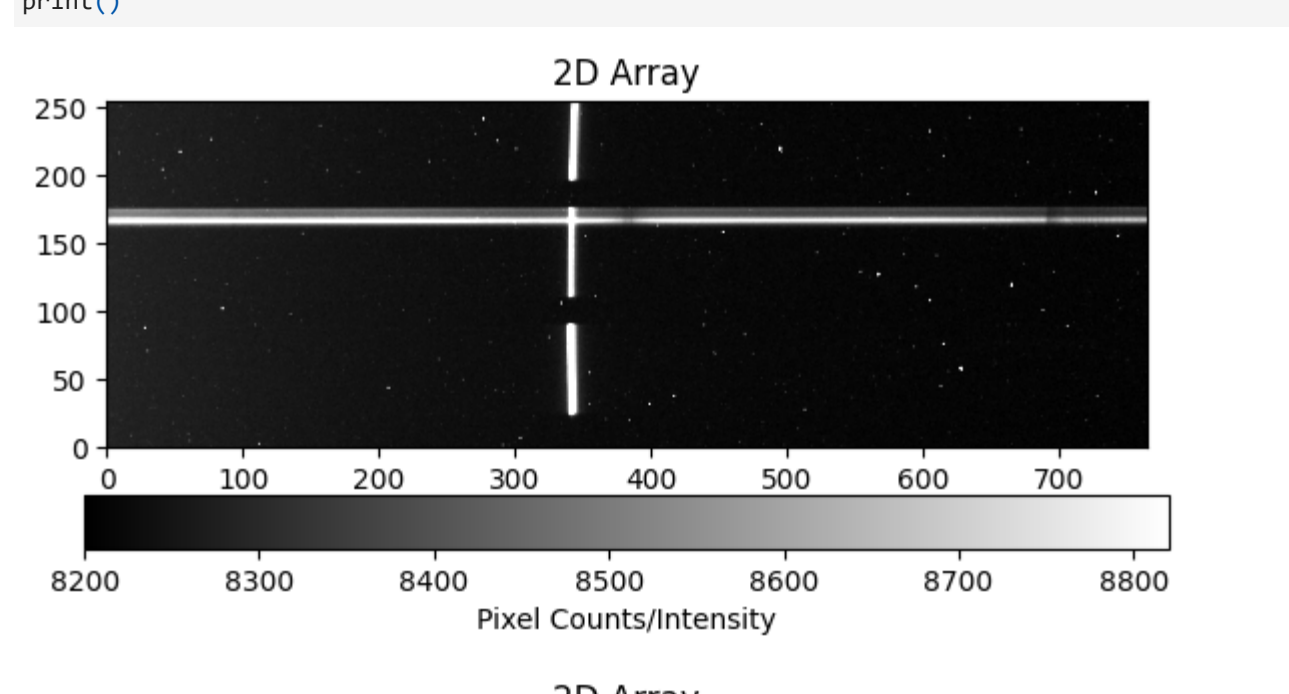


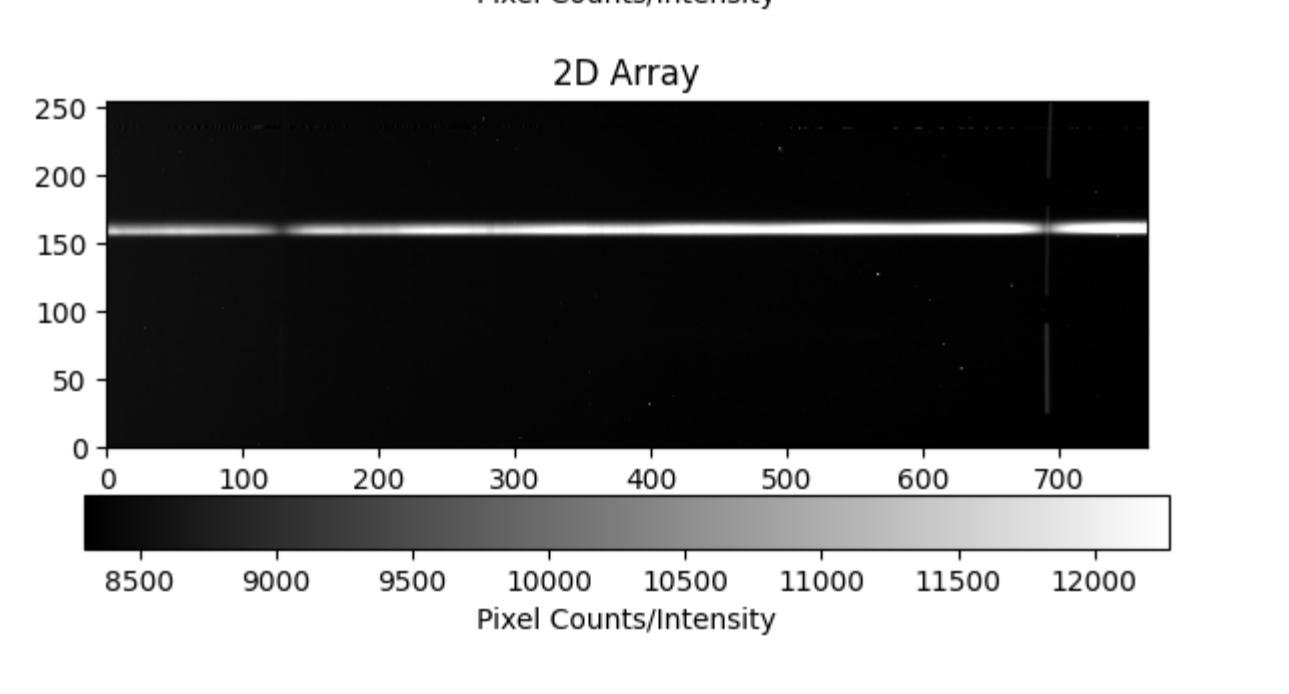


Sanity Check

In [7]:

```python
calibrate_fits_files(fits_file_list=glob_files('night_2','ha_laser_sanity'), display_array=True)
calibrate_fits_files(fits_file_list=glob_files('night_2','hbg_Hlamp_spectrum_sanity'), display_array=True)
print()
```





Dicts:
darks_mcombines
spectra_calib
flats_mcombines
sci_mcombines

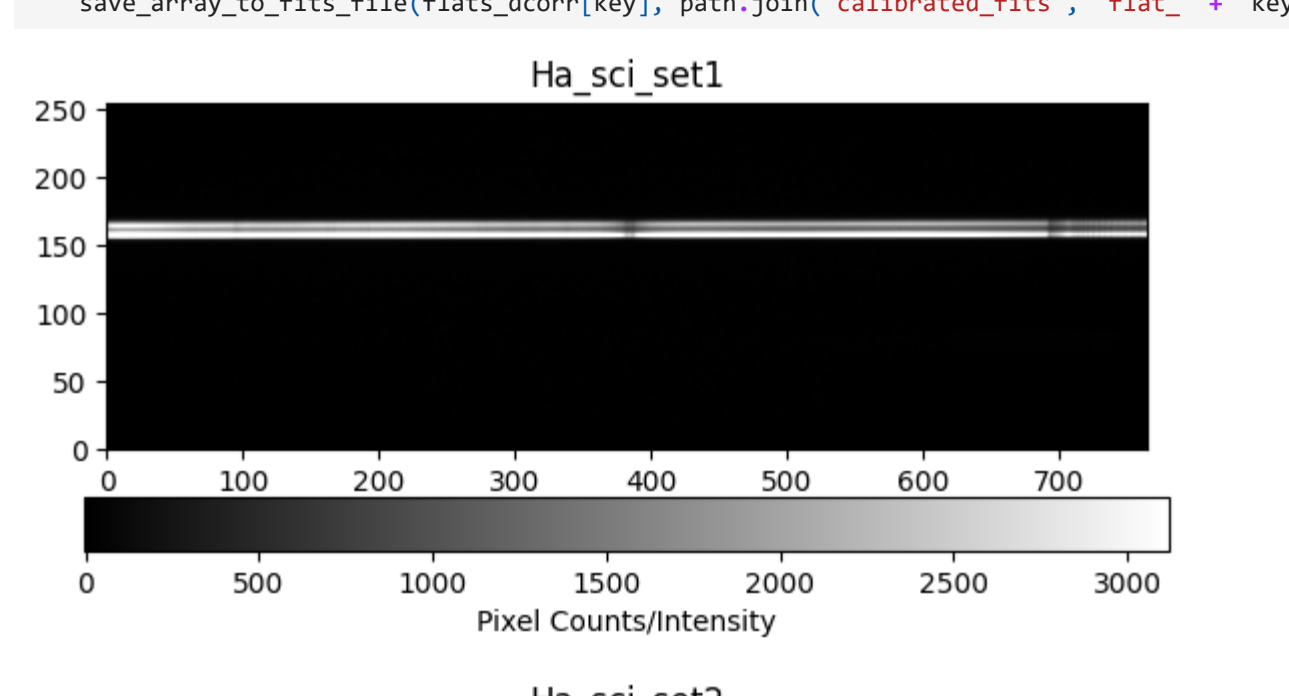
Dark Correction

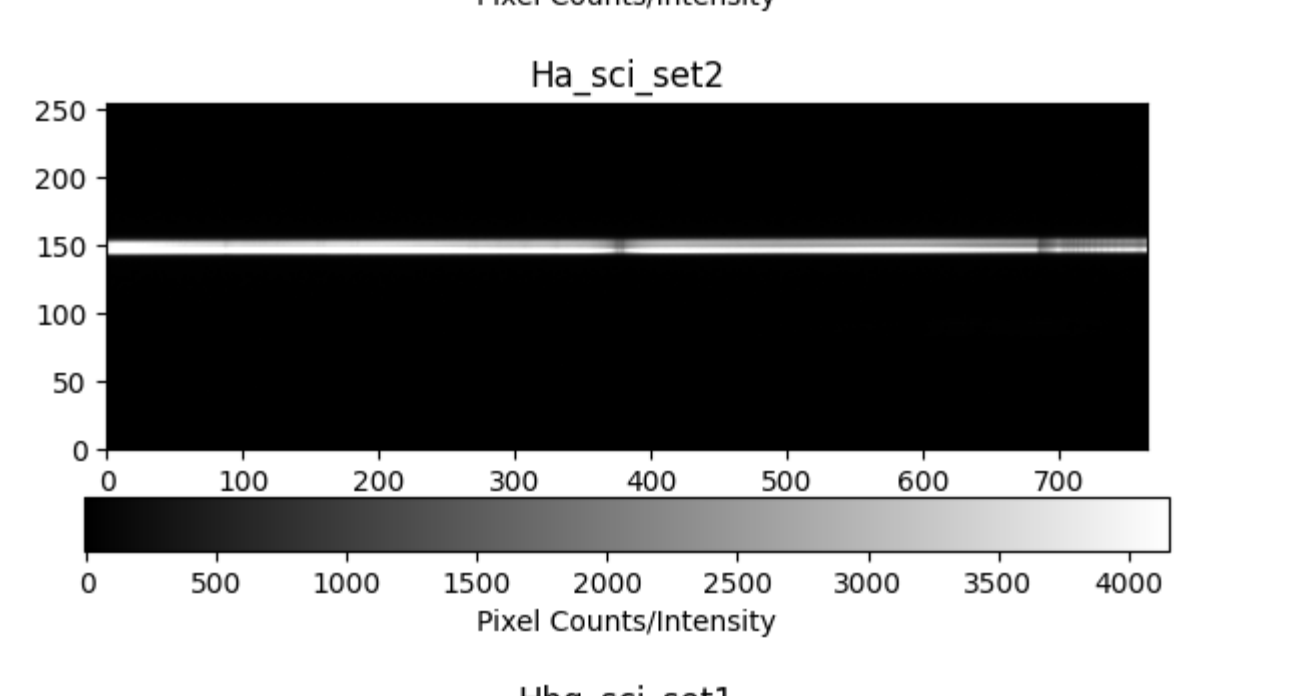
These will all be stored in dictionaries named:

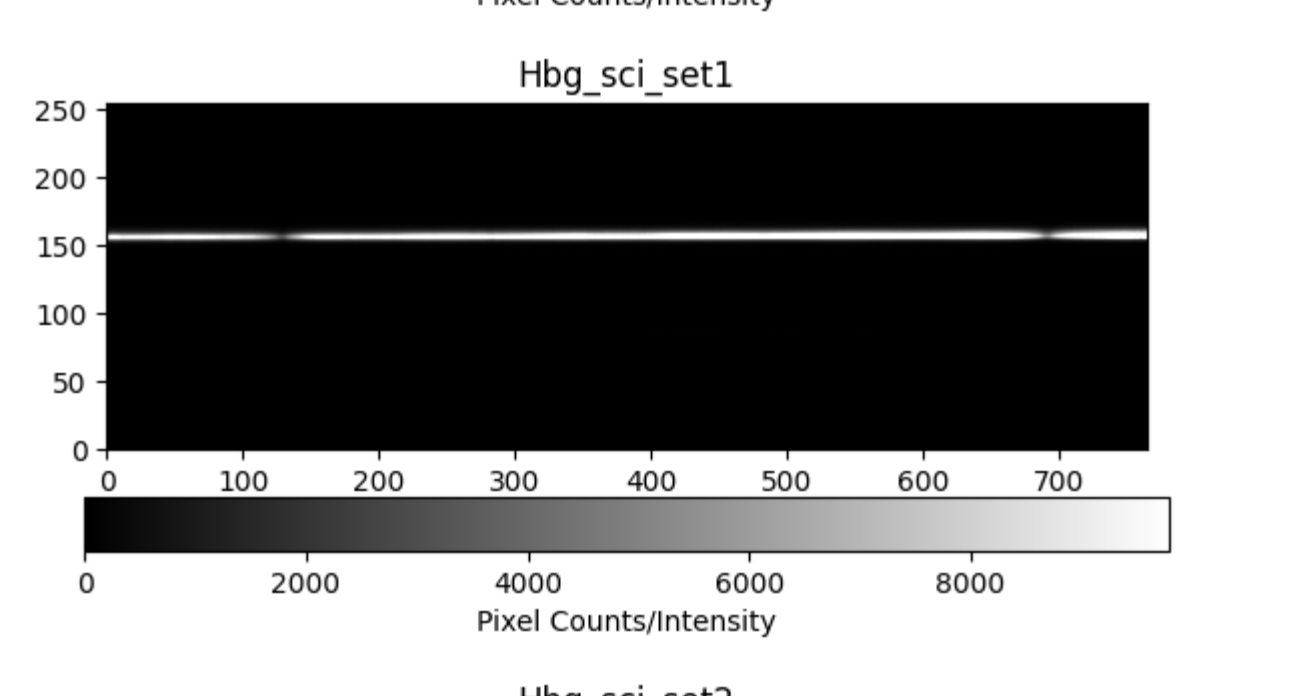
- `name_dcorr`
- `name_dcorr_unc`

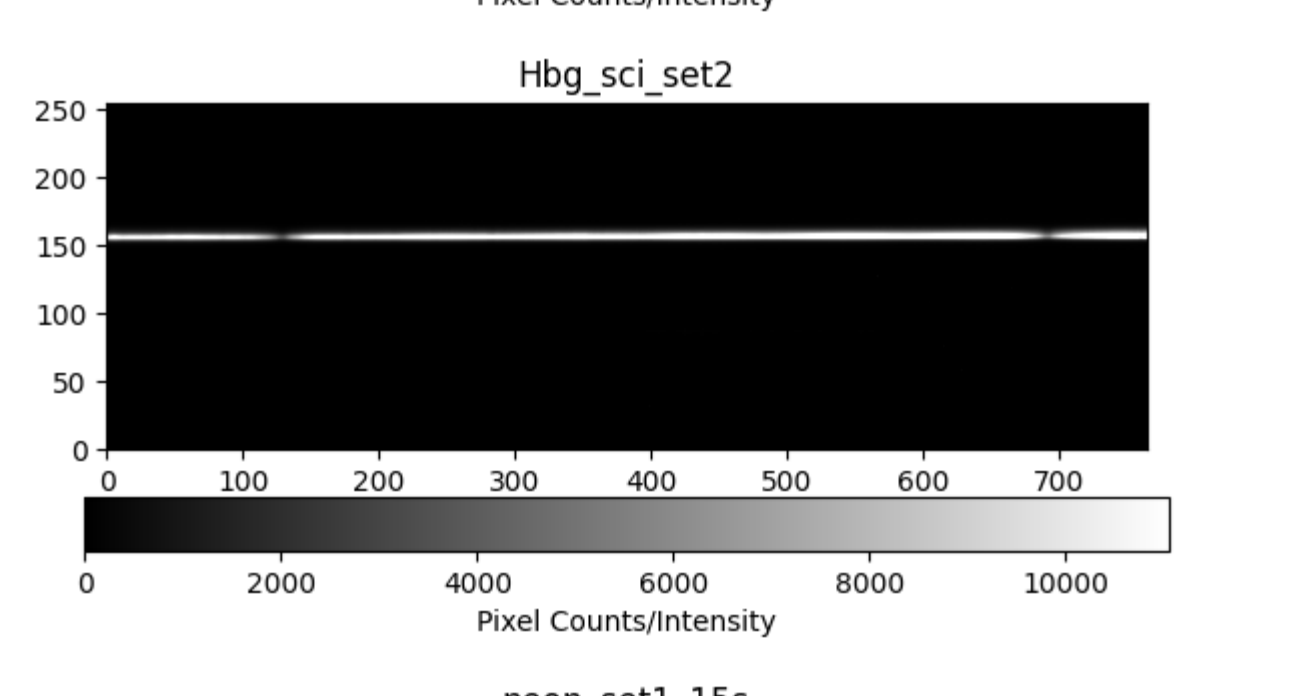
In [8]:

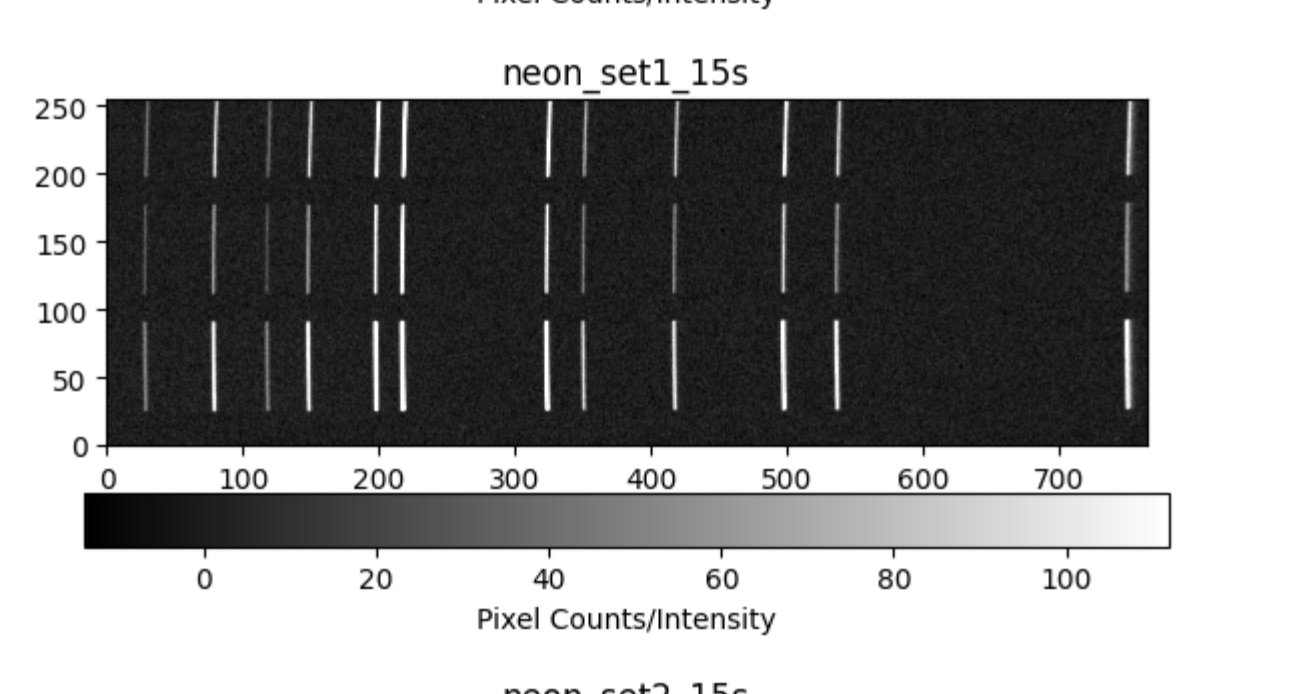
```python
spectra_dcorr = spectra_calib.copy()
flats_dcorr = flats_mcombines.copy()
sci_dcorr = sci_mcombines.copy()
spectra_dcorr_unc = spectra_calib.copy()
flats_dcorr_unc = flats_mcombines.copy()
sci_dcorr_unc = sci_mcombines.copy()

os.makedirs('calibrated_fits', exist_ok=True)

#Science:
for (key, value), (key_u, value_u) in zip(sci_mcombines.items(), sci_mcombines_unc.items()):
    sci_dcorr[key] = darks_correction(value, darks_mcombines['dark_25s'])
    sci_dcorr_unc[key_u] = darks_correction_uncertianty(value_u, darks_mcombines_unc['dark_25s'])
    display_2d_array(sci_dcorr[key], title=key)
    #display_2d_array(sci_dcorr[key] + sci_dcorr_unc[key], title=key)
    save_array_to_fits_file(sci_dcorr[key], path.join('calibrated_fits', 'sci_' + key + 'dark_correction.FIT'))

#Spectra:
darks_order = ['dark_15s','dark_15s','dark_10s']
for (key, value), dark_key in zip(spectra_calib.items(), darks_order):
    spectra_dcorr[key] = darks_correction(value, darks_mcombines[dark_key])
    spectra_dcorr_unc[key] = darks_correction_uncertianty(spectra_calib_unc[key], darks_mcombines_unc[dark_key])
    display_2d_array(spectra_dcorr[key], title=key)
    #display_2d_array(spectra_dcorr[key] + spectra_dcorr_unc[key], title=key)
    save_array_to_fits_file(spectra_dcorr[key], path.join('calibrated_fits', 'spec_' +  key + 'dark_correction.FIT'))

#Flats
darks_order = ['dark_3s','dark_60s']
for (key, value), dark_key in zip(flats_mcombines.items(), darks_order):
    flats_dcorr[key] = darks_correction(value, darks_mcombines[dark_key])
    flats_dcorr_unc[key] = darks_correction_uncertianty(flats_mcombines_unc[key], darks_mcombines_unc[dark_key])
    display_2d_array(flats_dcorr[key], title=key)
    #display_2d_array(flats_dcorr[key] + flats_dcorr_unc[key], title=key)
    save_array_to_fits_file(flats_dcorr[key], path.join('calibrated_fits', 'flat_' +  key + 'dark_correction.FIT'))
```

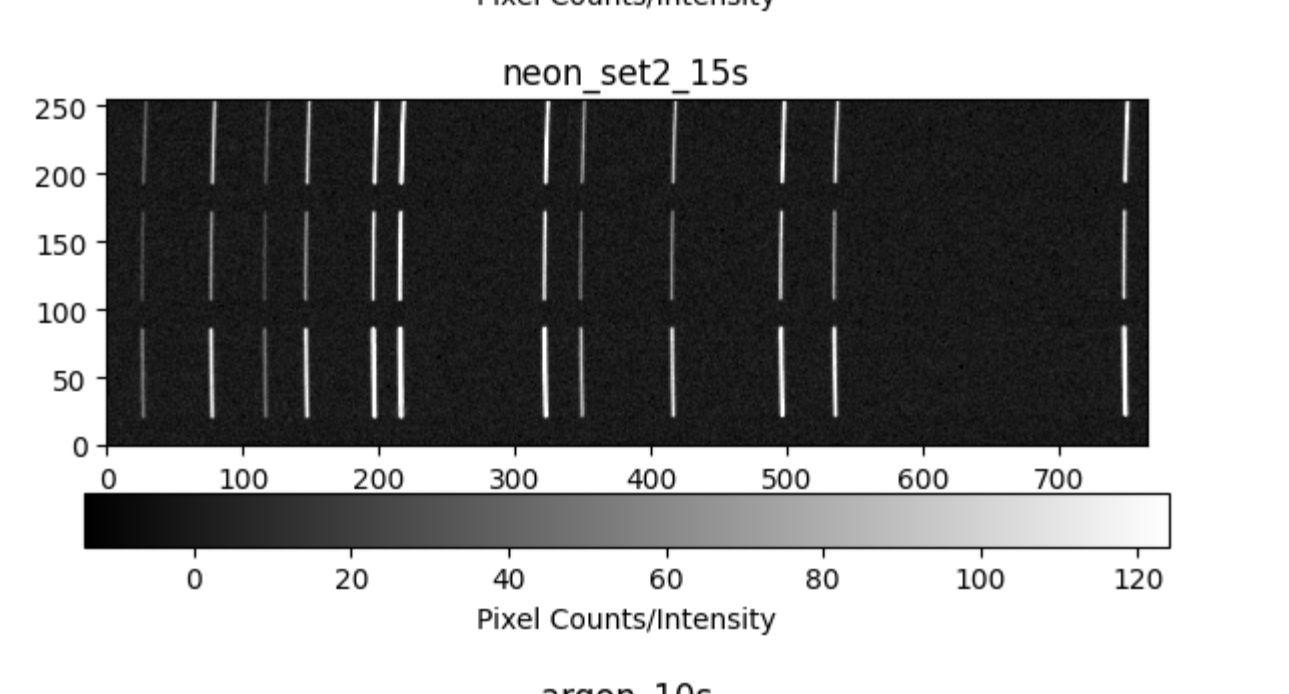


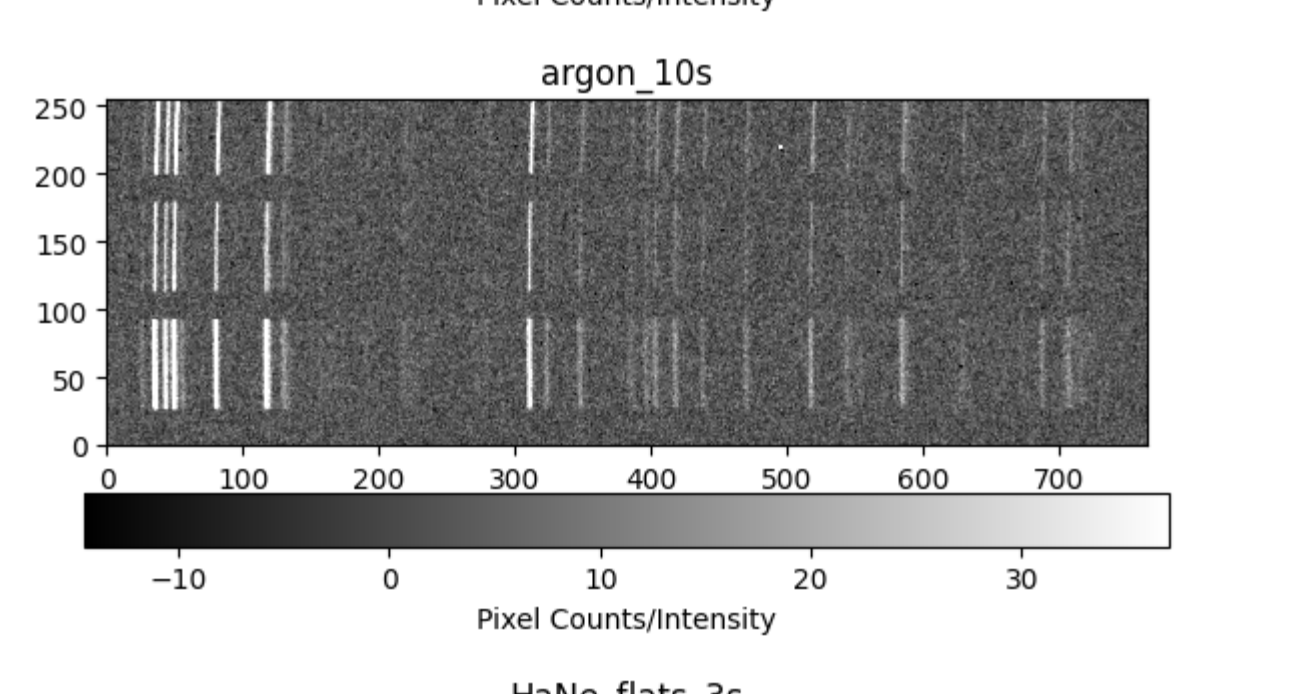


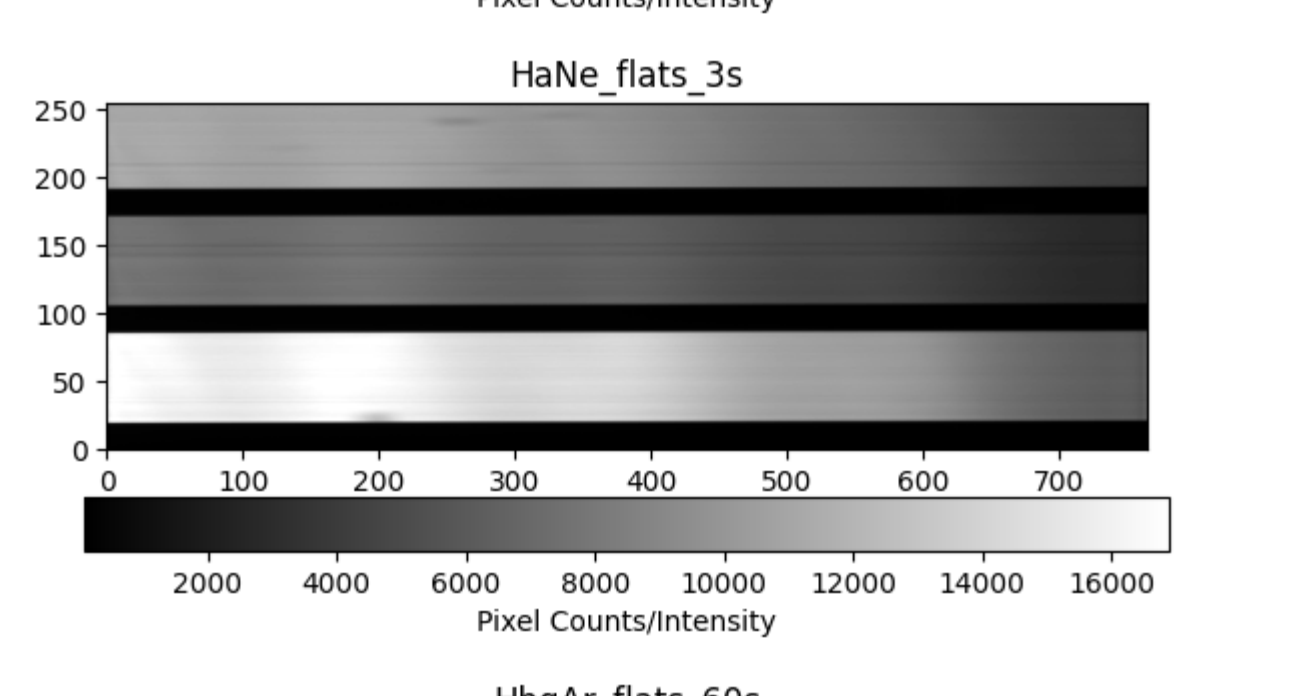


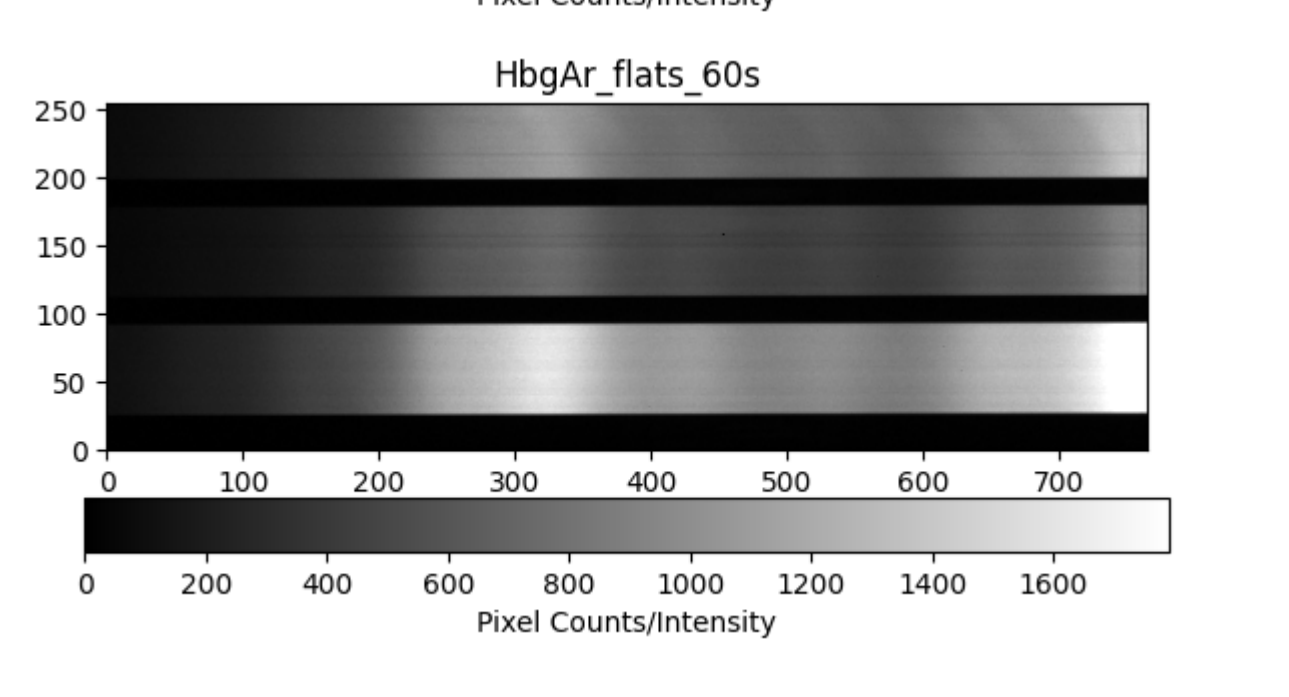












Defining Cuts for 25 um slit

We need to cut these arrays to only look at the slit that our star is in ($25\mu m$). We will worry about obtaining the target signal from the science frames in the Gaussian Cutting section.
Looking at the flats in ds9, here are the cut ranges for our frames:

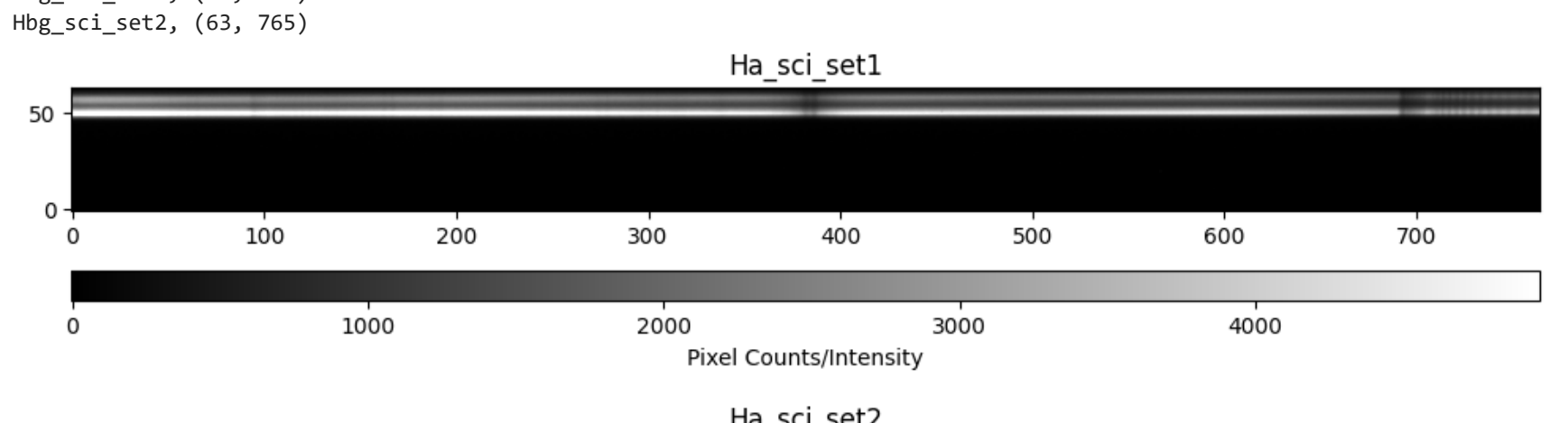
Hα and Neon: $107 \leq y_{pix} \leq 171$
Hβ, Hγ (Set 1): $114 \leq y_{pix} \leq 177$

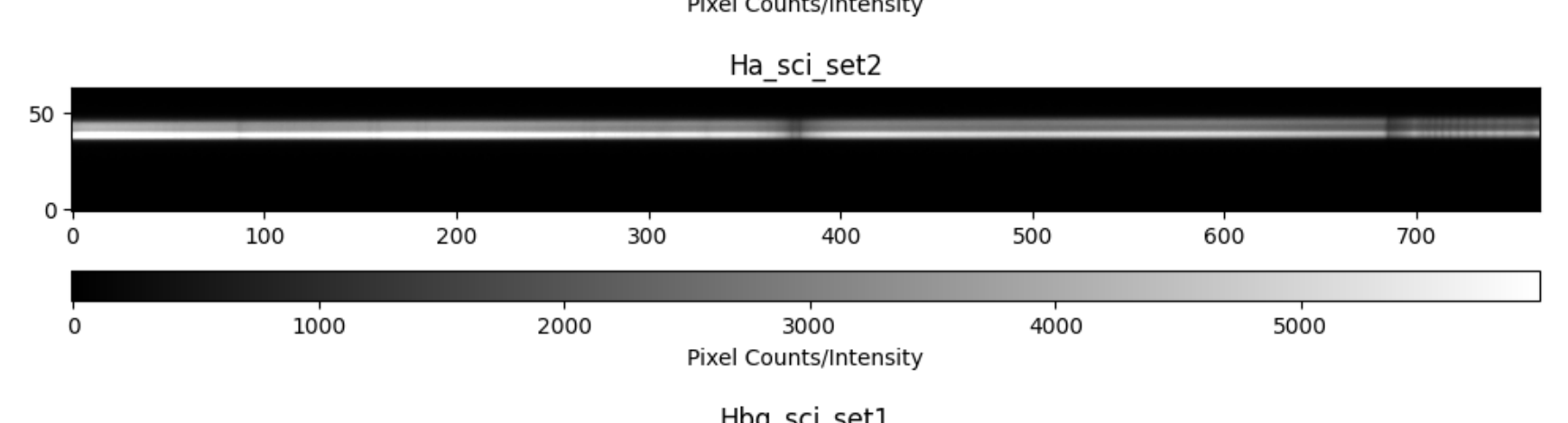
Lets cut these science arrays first

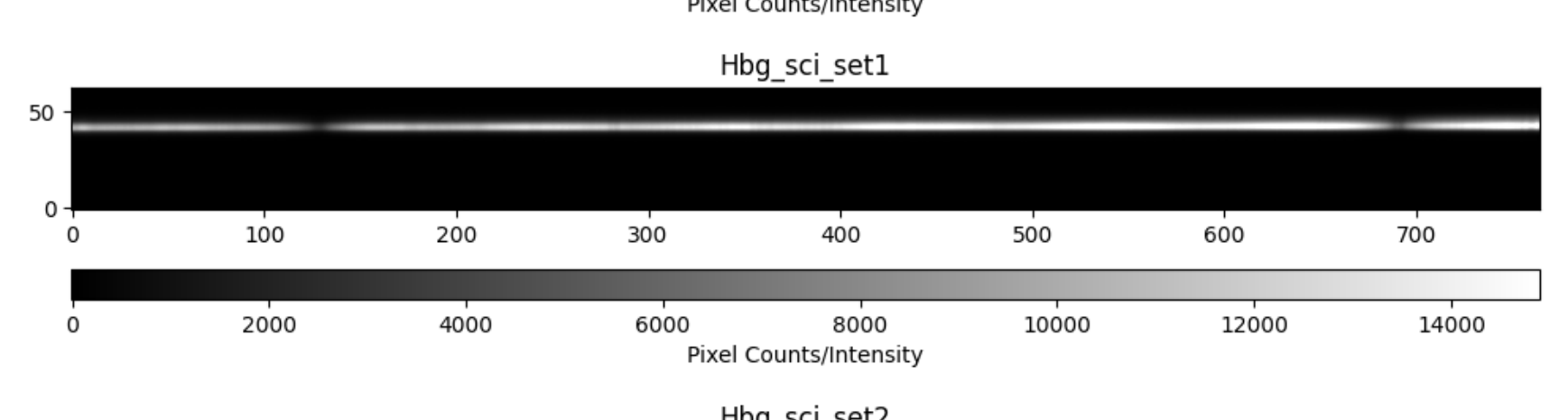
In [9]:

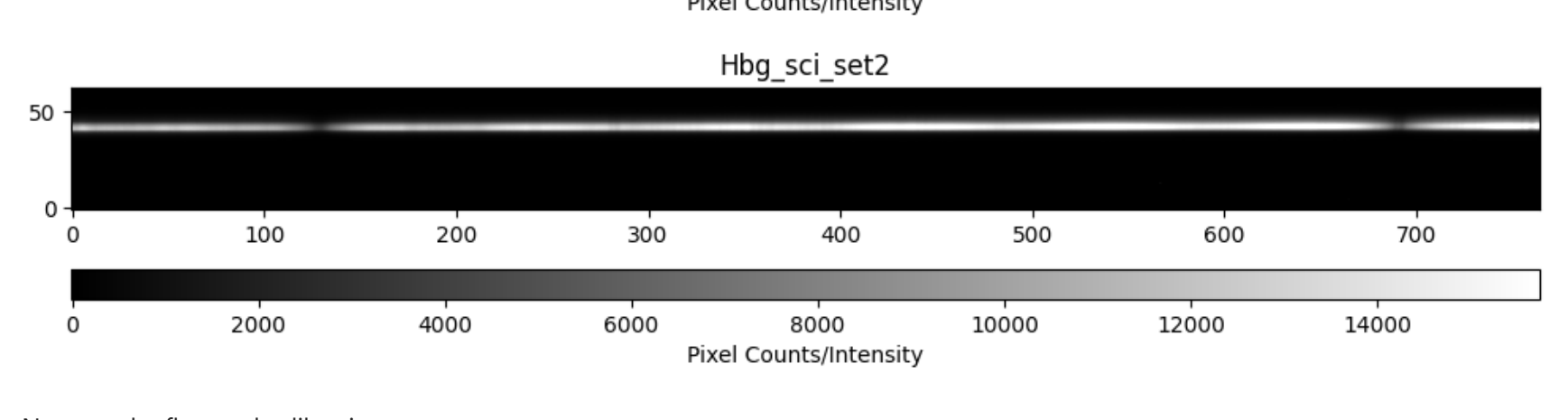
```python
#Create a dict of cuts
cut1 = [107,171]
cut2 = [114,177]
cuts_dict = {'Ha_1' : cut1,
             'Ha_2' : cut1,
             'Hgb_1' : cut2,
             'Hgb_2' : cut2,
             }
#Create dicts to store cut arrays
sci_cut = sci_dcorr.copy()
sci_cut_unc = sci_dcorr_unc.copy()
os.makedirs('cut_fits', exist_ok=True) #New directory

#Perform cuts
for (key, value), cut_key in zip(sci_cut.items(), cuts_dict):
    sci_cut[key] = cut_pixel_data_array(value, *cuts_dict[cut_key])
    sci_cut_unc[key] = cut_pixel_data_array(sci_cut_unc[key], *cuts_dict[cut_key])
    display_cut_spectrum(sci_cut[key], title=key)
    save_array_to_fits_file(sci_cut[key], path.join('cut_fits', 'sci_' +  key + '_cut.FIT'))

for key, value in sci_cut.items():
    print(f"{key}, {np.shape(value)}")
```

```
Ha_sci_set1, (64, 765)
Ha_sci_set2, (64, 765)
Hbg_sci_set1, (63, 765)
Hbg_sci_set2, (63, 765)
```





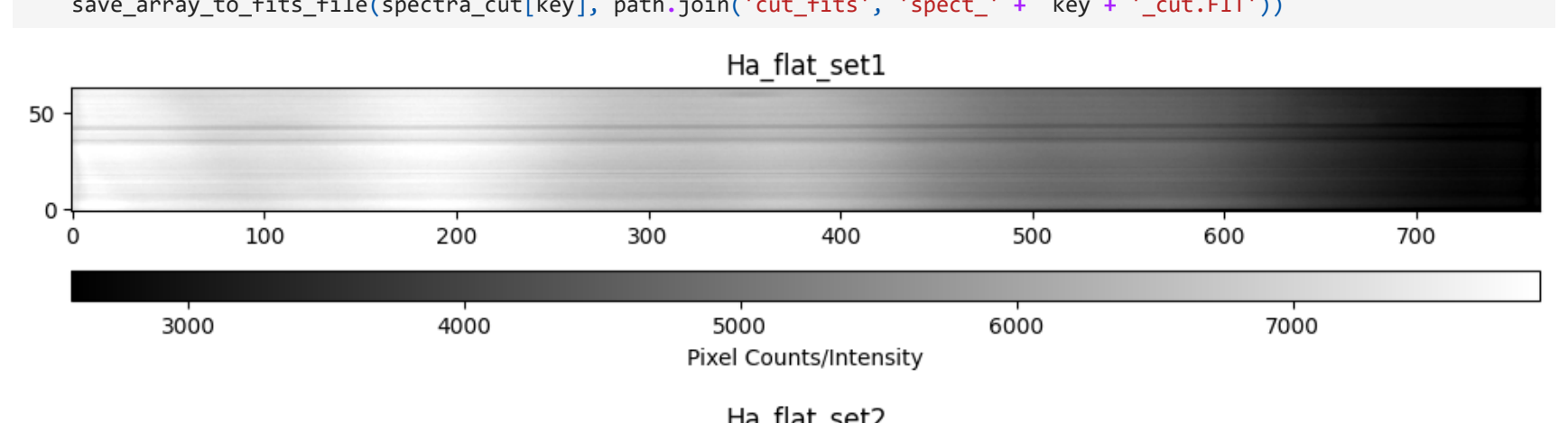


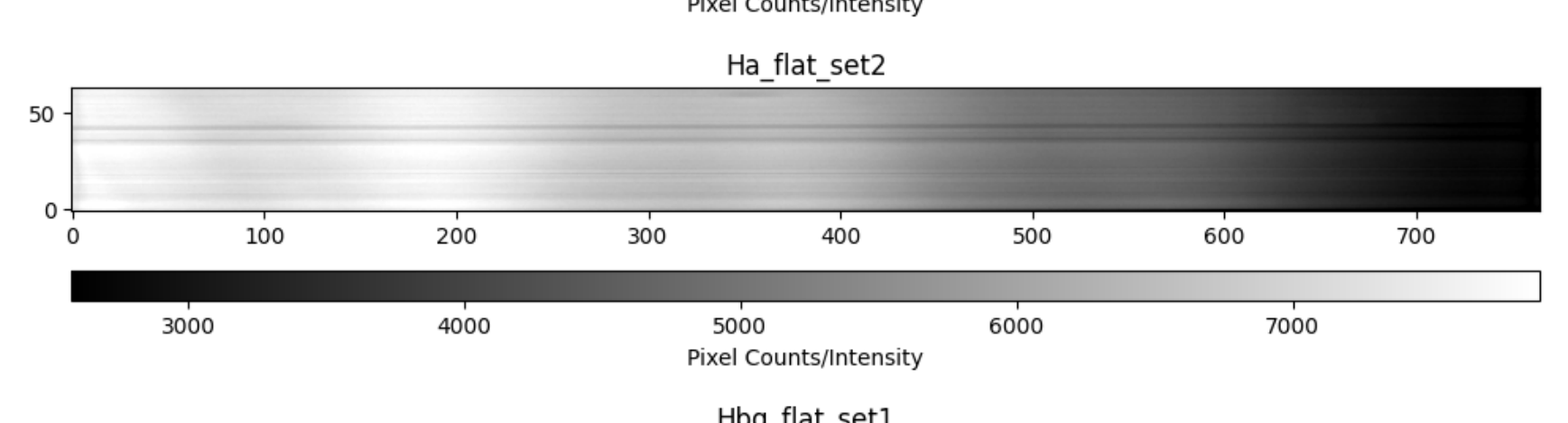


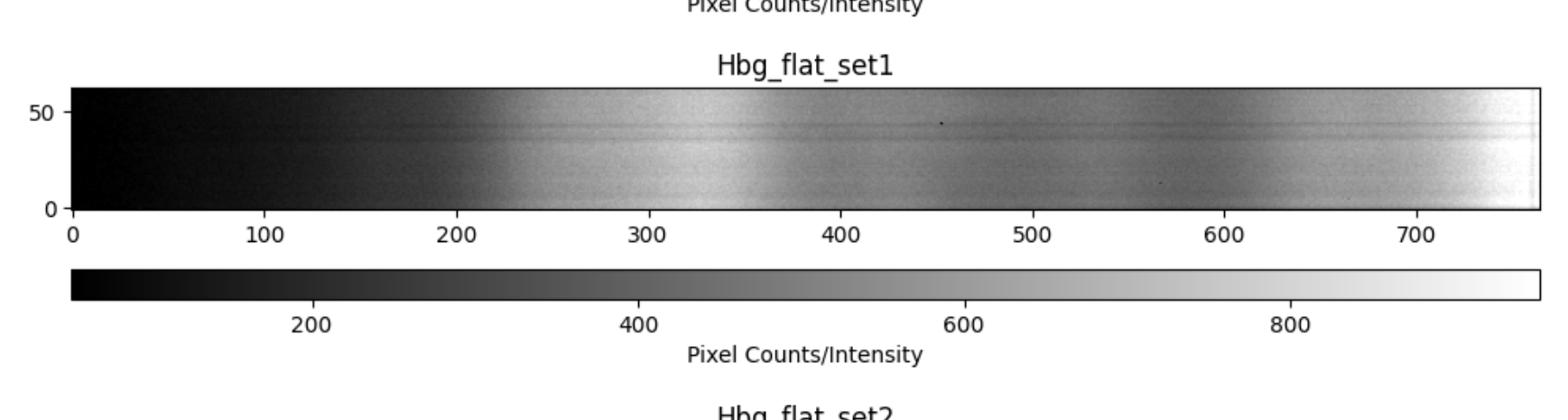
Now cut the flats and calibration spectra

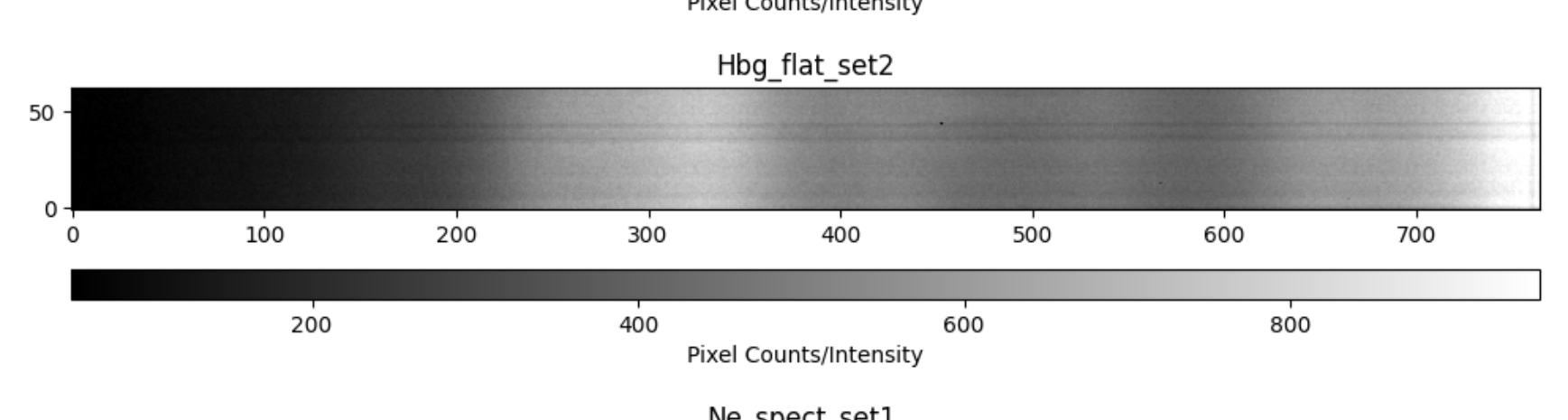
In [10]:

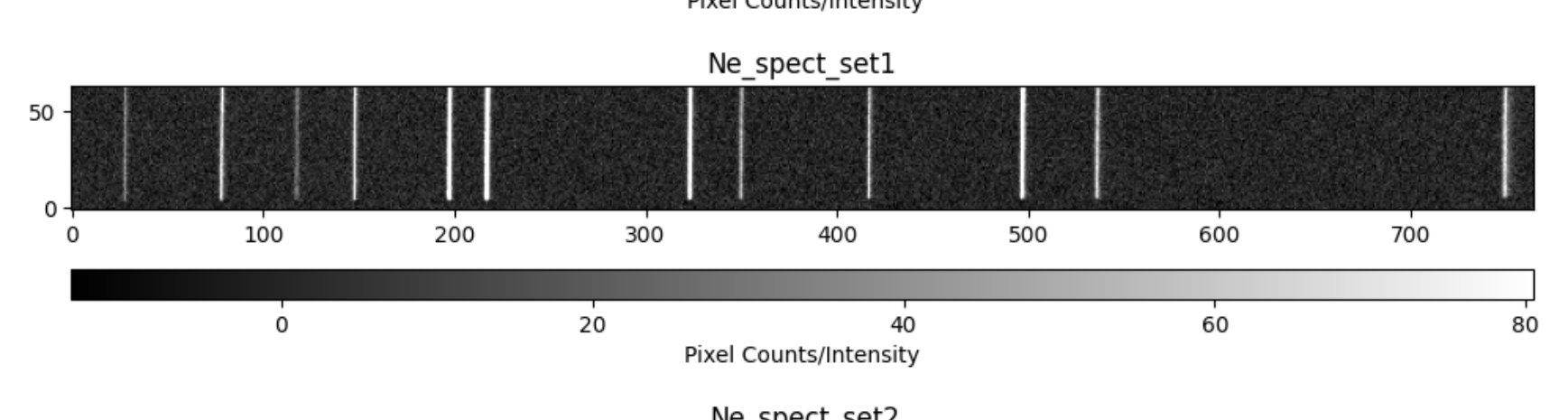
```python
flats_cut = flats_dcorr.copy()
flats_cut_unc = flats_dcorr_unc.copy()
spectra_cut = spectra_dcorr.copy()
spectra_cut_unc = spectra_dcorr_unc.copy()

flats_cut = {'Ha_flat_set1' : cut_pixel_data_array(flats_dcorr['HaNe_flats_3s'], *cuts_dict['Ha_1']),
             'Ha_flat_set2' : cut_pixel_data_array(flats_dcorr['HaNe_flats_3s'], *cuts_dict['Ha_2']),
             'Hbg_flat_set1' : cut_pixel_data_array(flats_dcorr['HbgAr_flats_60s'], *cuts_dict['Hgb_1']),
             'Hbg_flat_set2' : cut_pixel_data_array(flats_dcorr['HbgAr_flats_60s'], *cuts_dict['Hgb_2']),
            }
flats_cut_unc = {'Ha_flat_set1' : cut_pixel_data_array(flats_dcorr_unc['HaNe_flats_3s'], *cuts_dict['Ha_1']),
                 'Ha_flat_set2' : cut_pixel_data_array(flats_dcorr_unc['HaNe_flats_3s'], *cuts_dict['Ha_2']),
                 'Hbg_flat_set1' : cut_pixel_data_array(flats_dcorr_unc['HbgAr_flats_60s'], *cuts_dict['Hgb_1']),
                 'Hbg_flat_set2' : cut_pixel_data_array(flats_dcorr_unc['HbgAr_flats_60s'], *cuts_dict['Hgb_2']),
                }
spectra_cut = {'Ne_spect_set1' : cut_pixel_data_array(spectra_dcorr['neon_set1_15s'], *cuts_dict['Ha_1']),
               'Ne_spect_set2' : cut_pixel_data_array(spectra_dcorr['neon_set2_15s'], *cuts_dict['Ha_2']),
               'Ar_spect_set1' : cut_pixel_data_array(spectra_dcorr['argon_10s'], *cuts_dict['Hgb_1']),
               'Ar_spect_set2' : cut_pixel_data_array(spectra_dcorr['argon_10s'], *cuts_dict['Hgb_2']),
              }
spectra_cut_unc = {'Ne_spect_set1' : cut_pixel_data_array(spectra_dcorr_unc['neon_set1_15s'], *cuts_dict['Ha_1']),
                   'Ne_spect_set2' : cut_pixel_data_array(spectra_dcorr_unc['neon_set2_15s'], *cuts_dict['Ha_2']),
                   'Ar_spect_set1' : cut_pixel_data_array(spectra_dcorr_unc['argon_10s'], *cuts_dict['Hgb_1']),
                   'Ar_spect_set2' : cut_pixel_data_array(spectra_dcorr_unc['argon_10s'], *cuts_dict['Hgb_2']),
                  }

for key,value in flats_cut.items():
    display_cut_spectrum(flats_cut[key], title=key)
    save_array_to_fits_file(flats_cut[key], path.join('cut_fits', 'flat_' +  key + '_cut.FIT'))

for key,value in spectra_cut.items():
    display_cut_spectrum(spectra_cut[key], title=key)
    save_array_to_fits_file(spectra_cut[key], path.join('cut_fits', 'spect_' +  key + '_cut.FIT'))
```

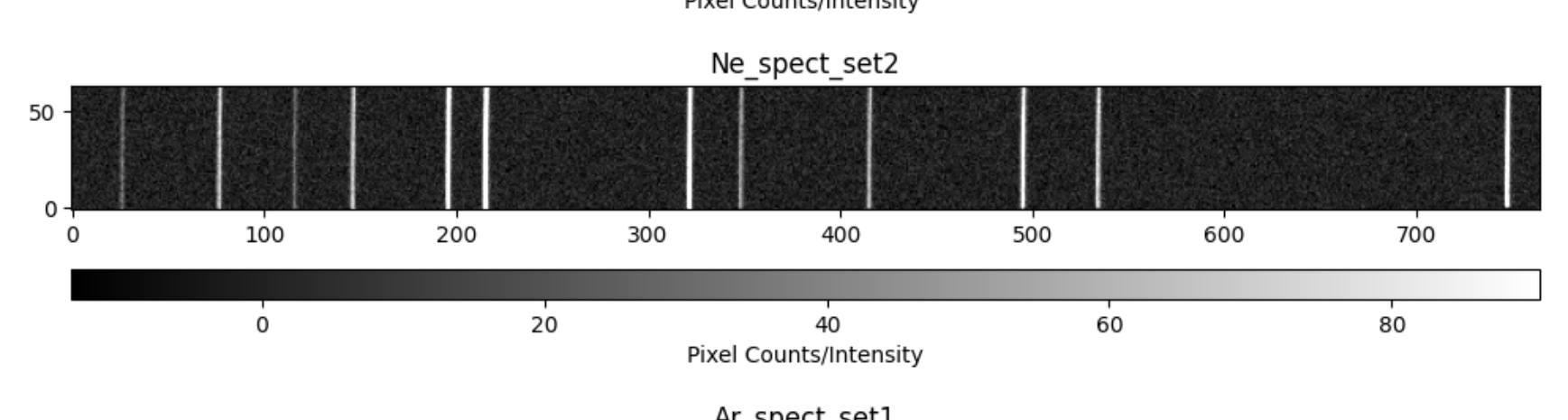


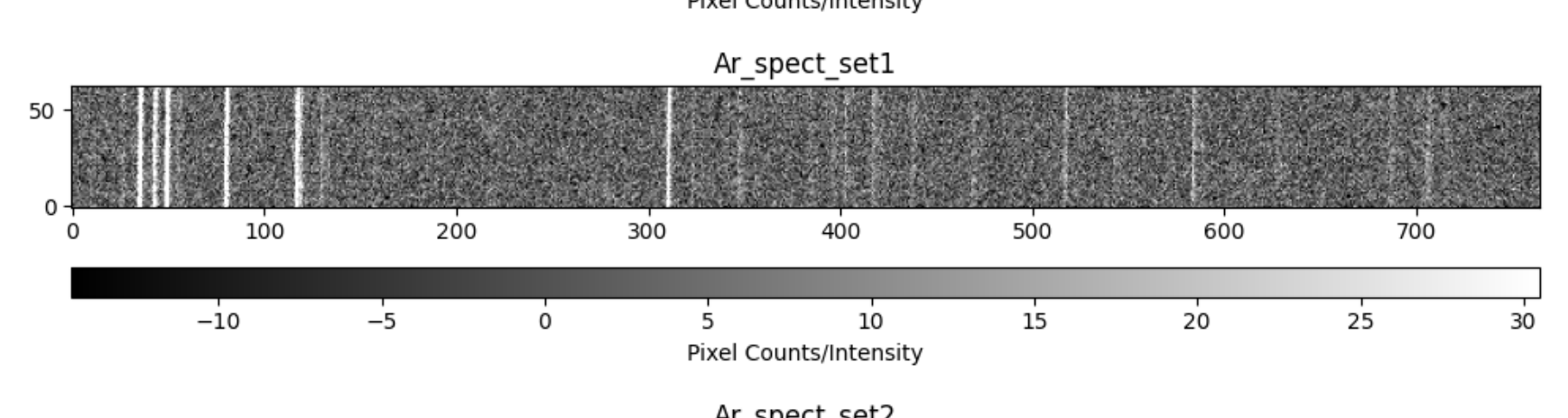


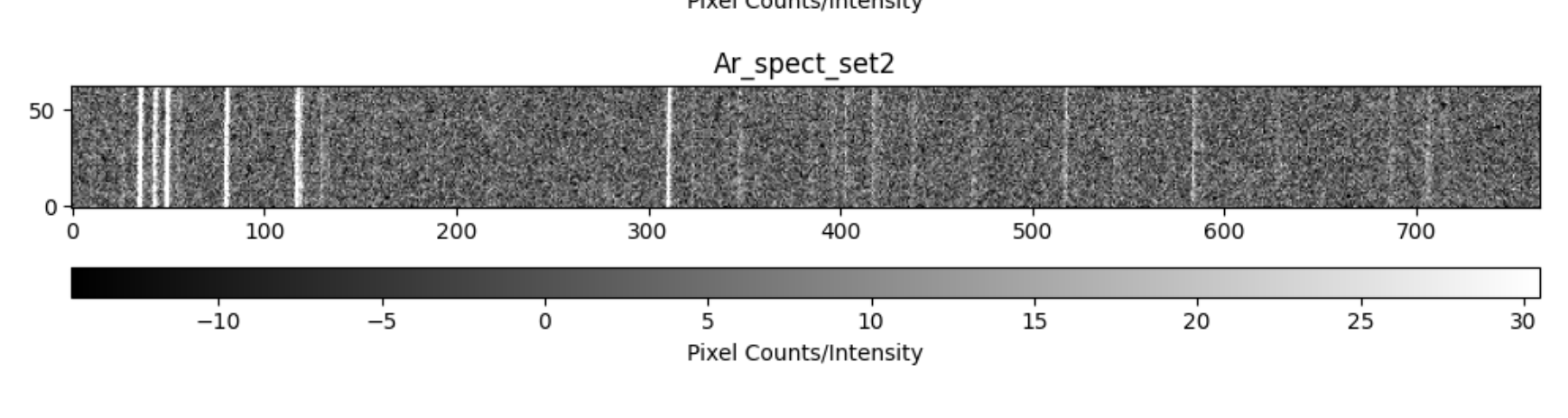












Flat Correction

We will try two different methods

In [11]:

```python
Polyfit = True
FlatDivision = False
```

Method 1: Polynomial Fit

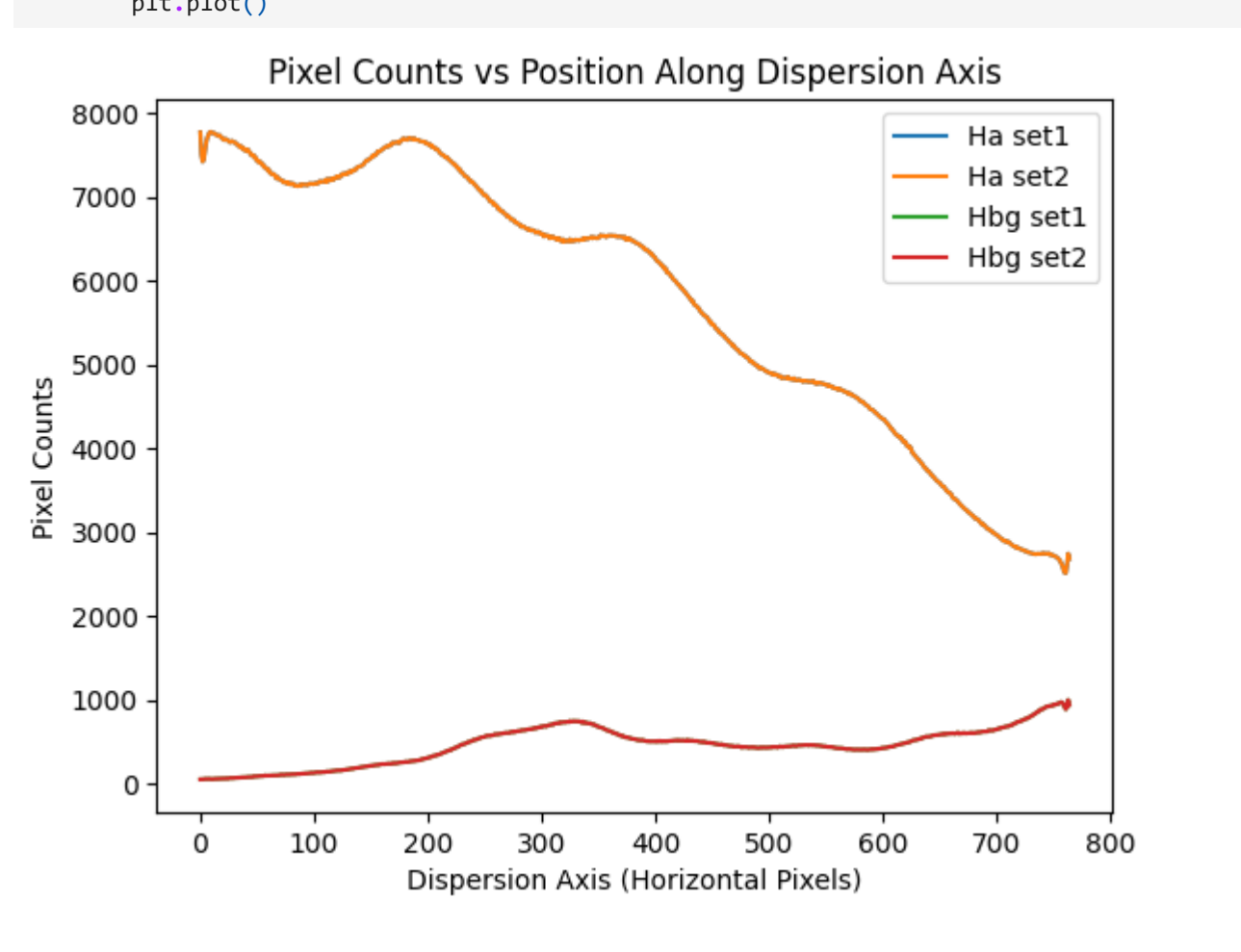
Now it is time for the flat correction. Here are the steps:

1. Take the 2D array of the flat field and find the median value of each column. This will create a 1D array with intensity values corresponding to horizontal pixel positions.
2. Plot the 1D array vs pixel position.
3. Fit a polynomial through the intensity curve.
4. Divide all data by this curve fit.

1 and 2) Creating 1D Flat Field Arrays and Plotting Along Dispersion Axis

In [12]:

```python
if Polyfit:
    flat_field_col_medians = {}
    flat_field_col_medians_unc = {}

    for key,value in flats_cut.items():
        flat_field_col_medians[key + '_col_median'] = np.median(value, axis=0)
        flat_field_col_medians_unc[key + '_col_median'] = median_uncertainty(value, axis=0)

    indices = np.arange(len(flat_field_col_medians['Ha_flat_set1_col_median']))

    for key,value in flat_field_col_medians.items():
        label = key.replace('_flat_',' ').replace('_col_median','')
        plt.plot(indices, value, label=label, ls='solid',)# marker='.', ms=0)
        #plt.errorbar(indices, value, yerr=flat_field_col_medians_unc[key], fmt='.', ms=1, lw=1,label=label + ' Uncerta
        plt.title('Pixel Counts vs Position Along Dispersion Axis')
        plt.xlabel('Dispersion Axis (Horizontal Pixels)')
        plt.ylabel('Pixel Counts')
        plt.legend()
        plt.plot()
```

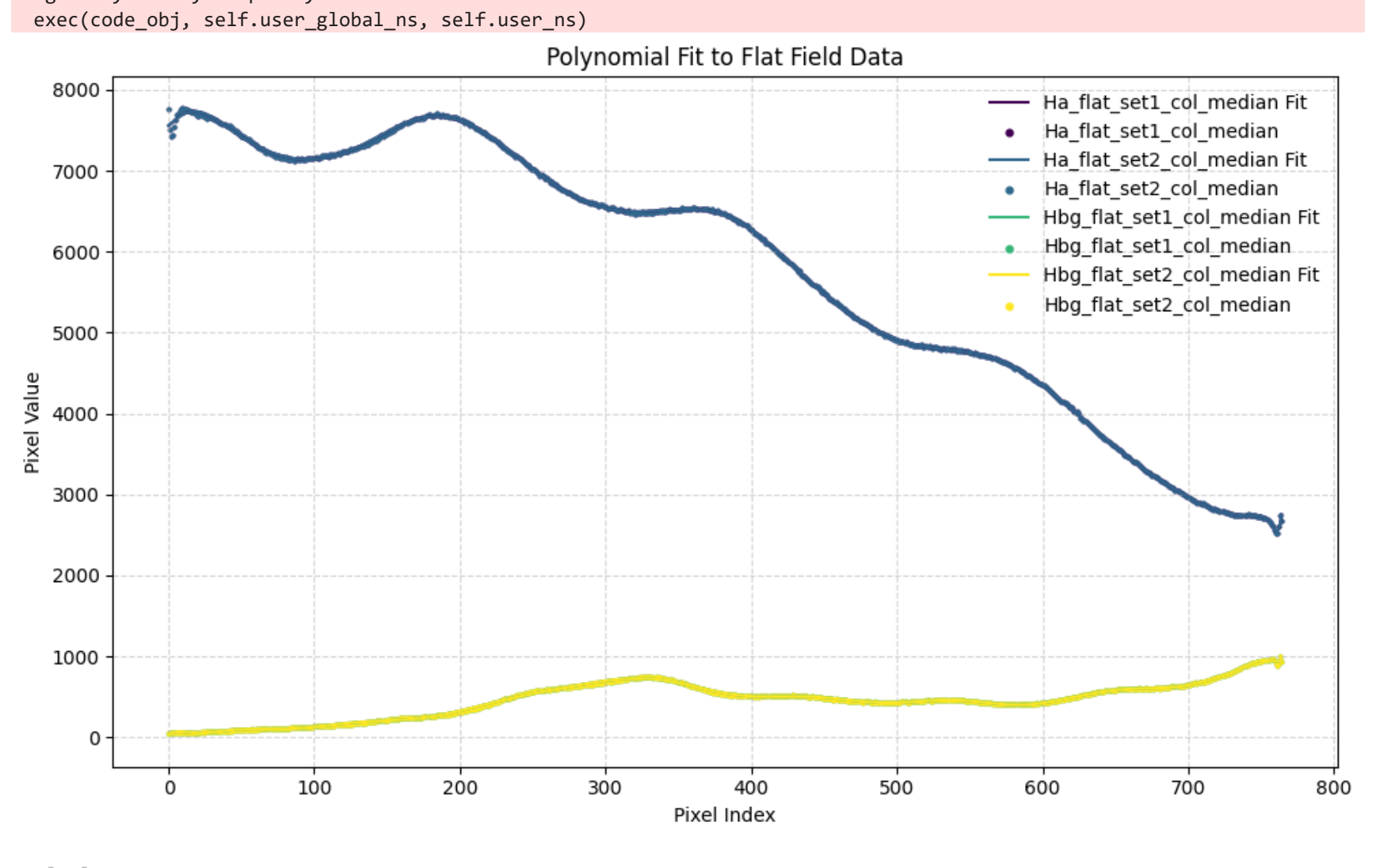


3 and 4) Polynomial Fit and Flat Normalization

In [13]:

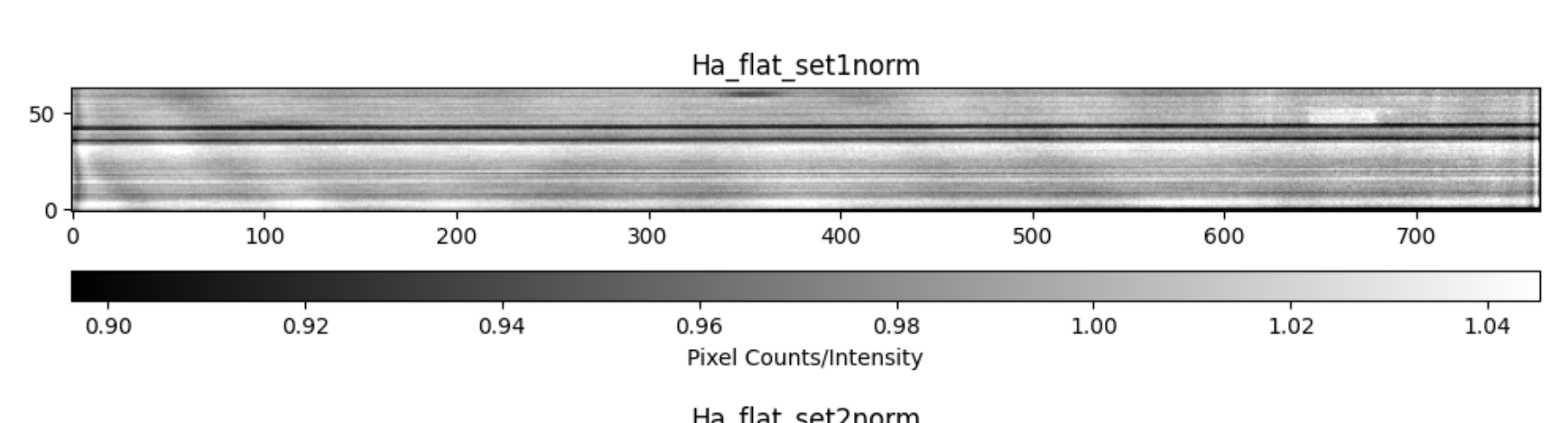
```python
if Polyfit:
    specific_plot = False
    specific_plot_label = 'Ha_flat_set1'
    os.makedirs('normalized_fits', exist_ok=True)
    normalized_flat_fields = {}
    normalized_flat_fields_unc = {}
    degree = 50
    fig, ax = plt.subplots(figsize=(10,6))
    colors = plt.cm.viridis(np.linspace(0, 1, len(flat_field_col_medians))) #plasma viridis

    for i, ((med_key, med_values), (cut_flat_key, cut_flat_values)) in enumerate(zip(flat_field_col_medians.items(), fl
        if specific_plot:
            if cut_flat_key != specific_plot_label:  # Change the condition to choose which set to display
                continue
        print(med_key,cut_flat_key)
        coefficients = np.polyfit(indices, med_values, deg=degree)
        polynomial_fit = np.polyval(coefficients, indices)
        polynomial_fit_2d = np.tile(polynomial_fit, (cut_flat_values.shape[0], 1))
        ax.plot(indices, polynomial_fit, color=colors[i], label=f'{med_key} Fit')
        ax.scatter(indices, med_values, marker='o', s=3, color=colors[i], label=f'{med_key}')
        ax.set_ylabel('Pixel Value')
        ax.set_xlabel('Pixel Index')
        ax.set_title('Polynomial Fit to Flat Field Data')
        ax.grid(True, which='both', linestyle='--', alpha=0.5)
        plt.legend()

        normalized_flat_fields[cut_flat_key] = flats_cut[cut_flat_key] / polynomial_fit_2d
        normalized_flat_fields_unc[cut_flat_key] = flats_cut_unc[cut_flat_key] / polynomial_fit_2d    # good approximati
        save_array_to_fits_file(normalized_flat_fields[cut_flat_key], path.join('normalized_fits', 'norm_' +  cut_flat_

    ax.legend(loc='upper right', frameon=False, fontsize=10, markerscale=2)
    # Show the plot
    plt.tight_layout()  # Adjusts layout to fit everything
    plt.show()
```

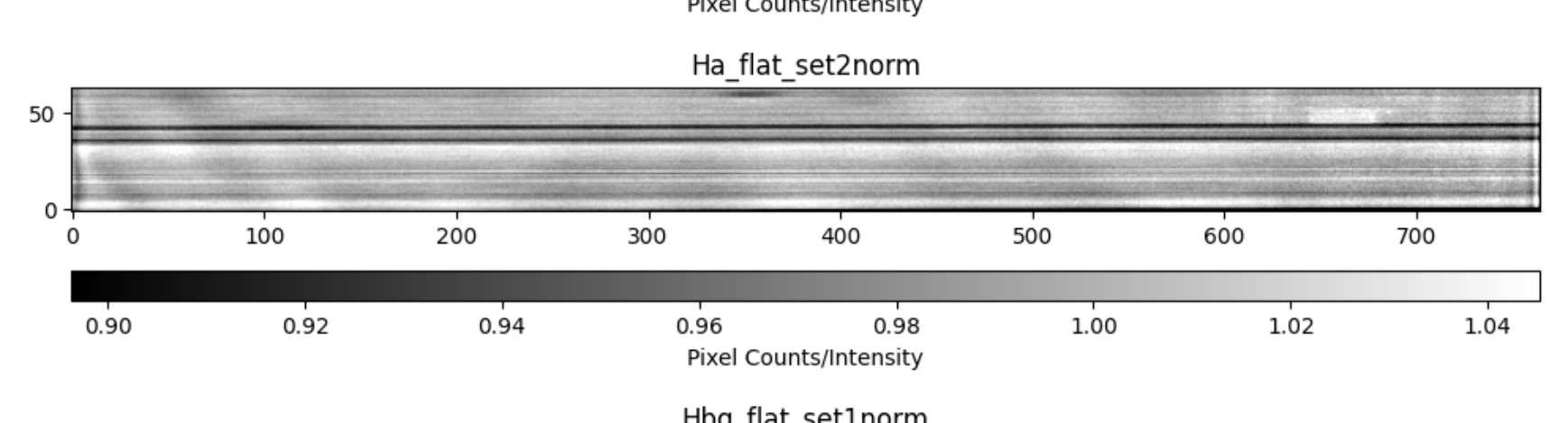
```
Ha_flat_set1_col_median Ha_flat_set1
Ha_flat_set2_col_median Ha_flat_set2
Hbg_flat_set1_col_median Hbg_flat_set1
Hbg_flat_set2_col_median Hbg_flat_set2
C:\Users\ronan\AppData\Local\Programs\Python\Python311\Lib\site-packages\IPython\core\interactiveshell.py:3526: RankWar
ning: Polyfit may be poorly conditioned
  exec(code_obj, self.user_global_ns, self.user_ns)
C:\Users\ronan\AppData\Local\Programs\Python\Python311\Lib\site-packages\IPython\core\interactiveshell.py:3526: RankWar
ning: Polyfit may be poorly conditioned
  exec(code_obj, self.user_global_ns, self.user_ns)
C:\Users\ronan\AppData\Local\Programs\Python\Python311\Lib\site-packages\IPython\core\interactiveshell.py:3526: RankWar
ning: Polyfit may be poorly conditioned
  exec(code_obj, self.user_global_ns, self.user_ns)
C:\Users\ronan\AppData\Local\Programs\Python\Python311\Lib\site-packages\IPython\core\interactiveshell.py:3526: RankWar
ning: Polyfit may be poorly conditioned
  exec(code_obj, self.user_global_ns, self.user_ns)
```

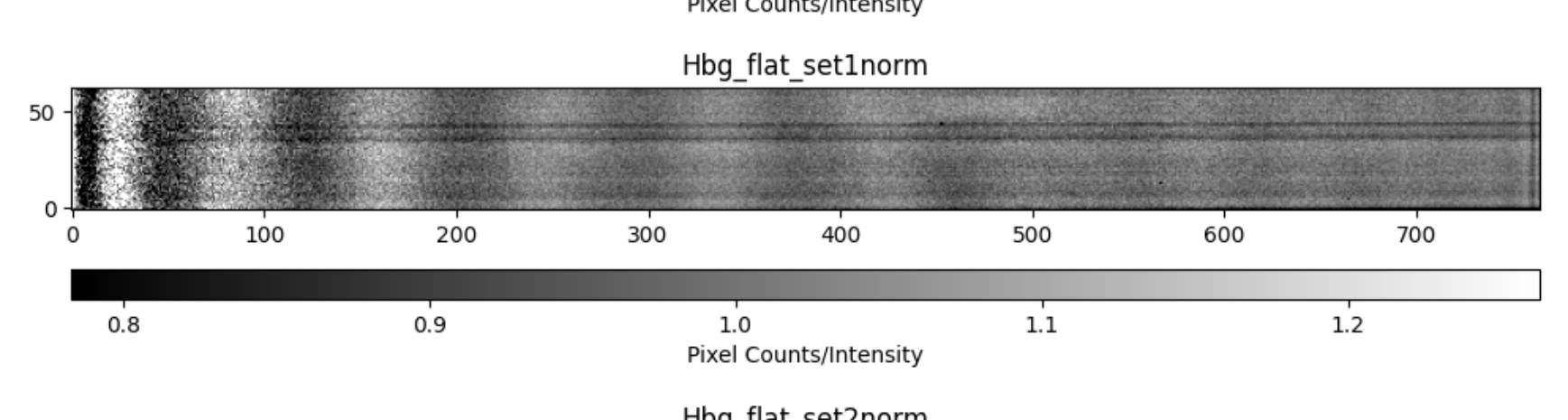


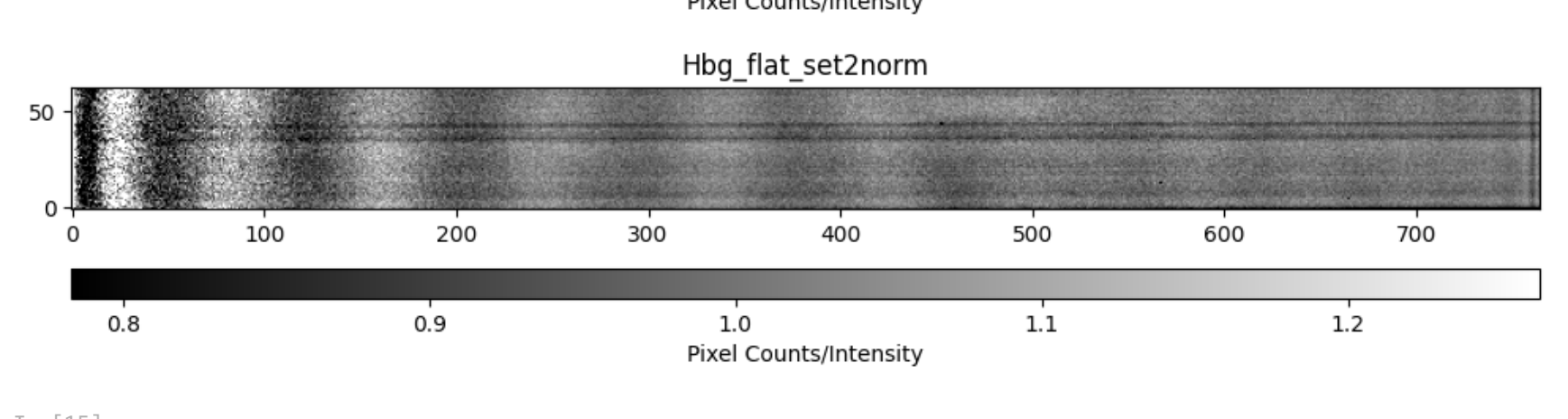
In [14]:

```python
if Polyfit:
    for key, values in normalized_flat_fields.items():
        display_cut_spectrum(values, title=key+'norm')
```

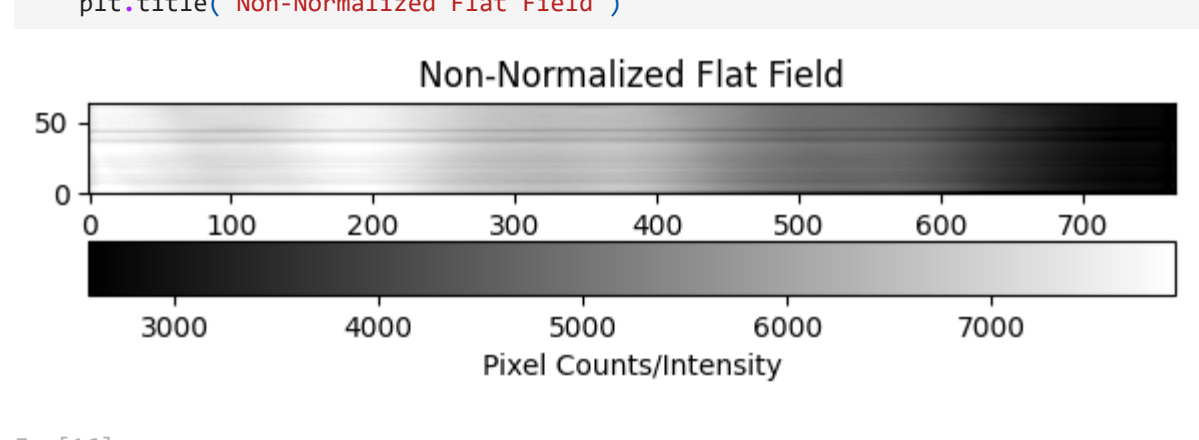




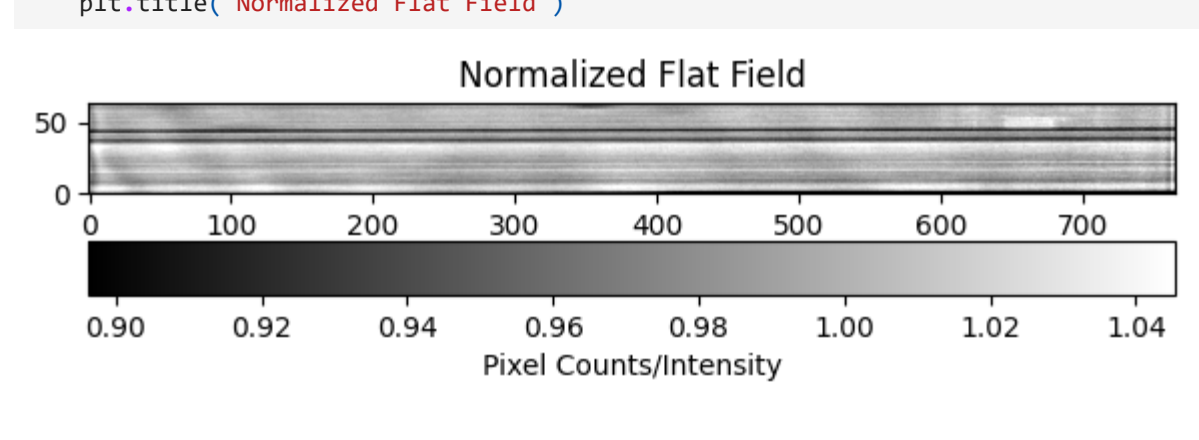




In [15]:
```python
if Polyfit:
    display_2d_array(flats_cut['Ha_flat_set1'])
    plt.title('Non-Normalized Flat Field')
```

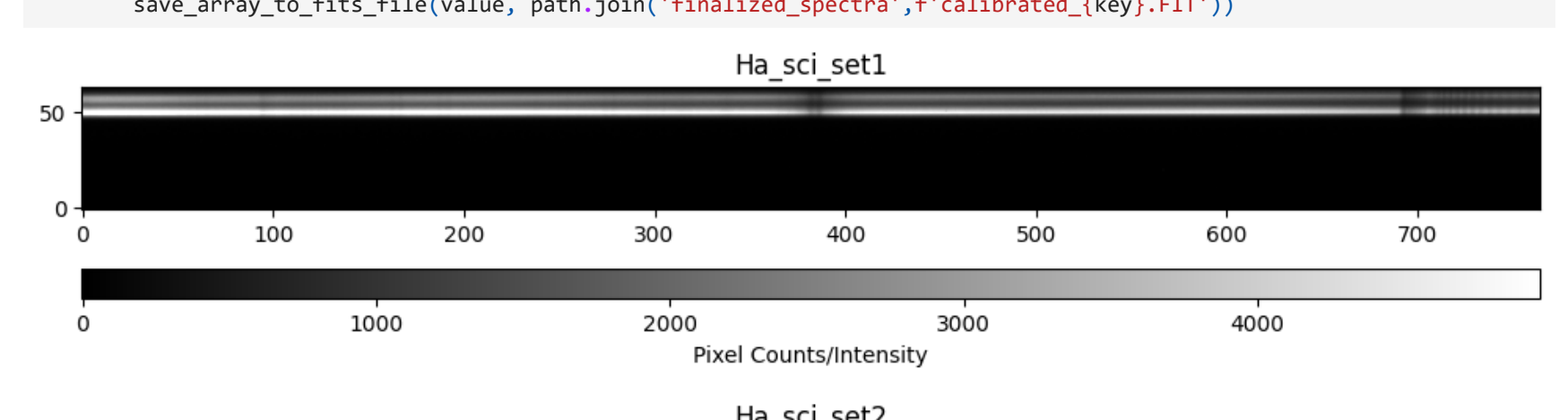


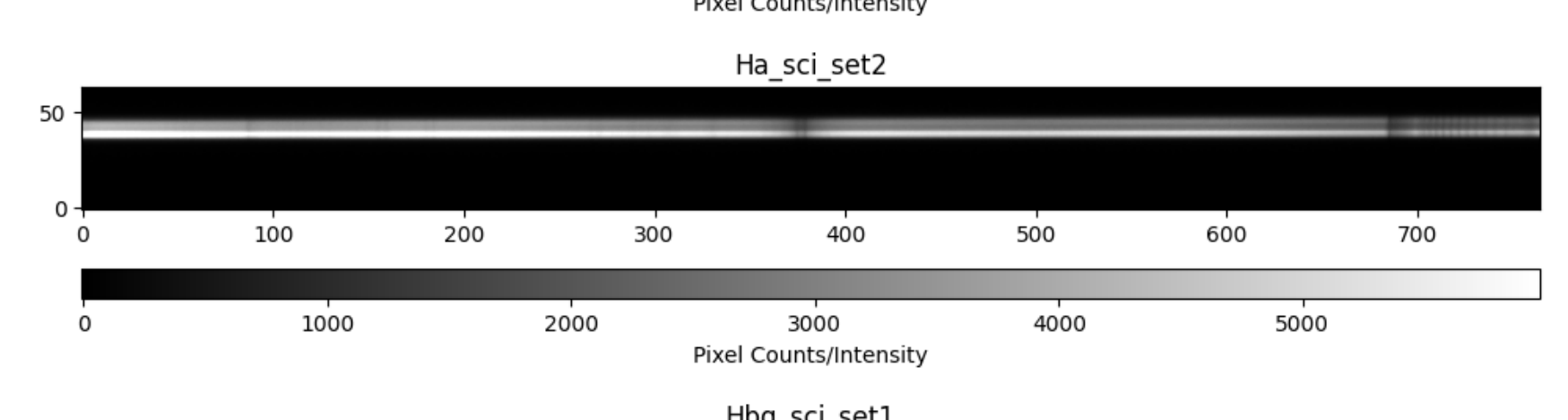
In [16]:
```python
if Polyfit:
    display_2d_array(normalized_flat_fields['Ha_flat_set1'])
    plt.title('Normalized Flat Field')
```

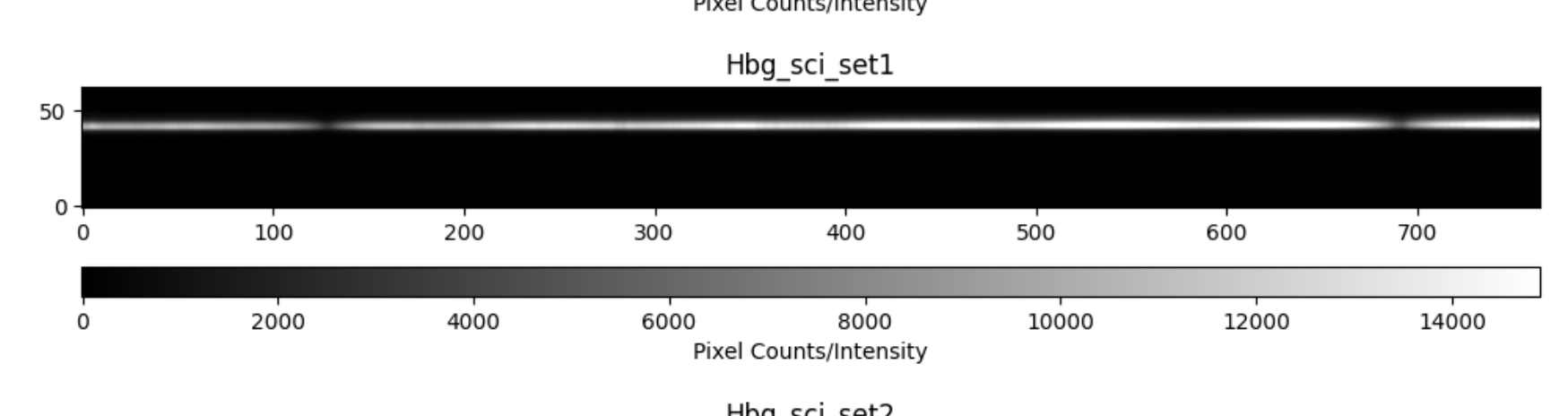


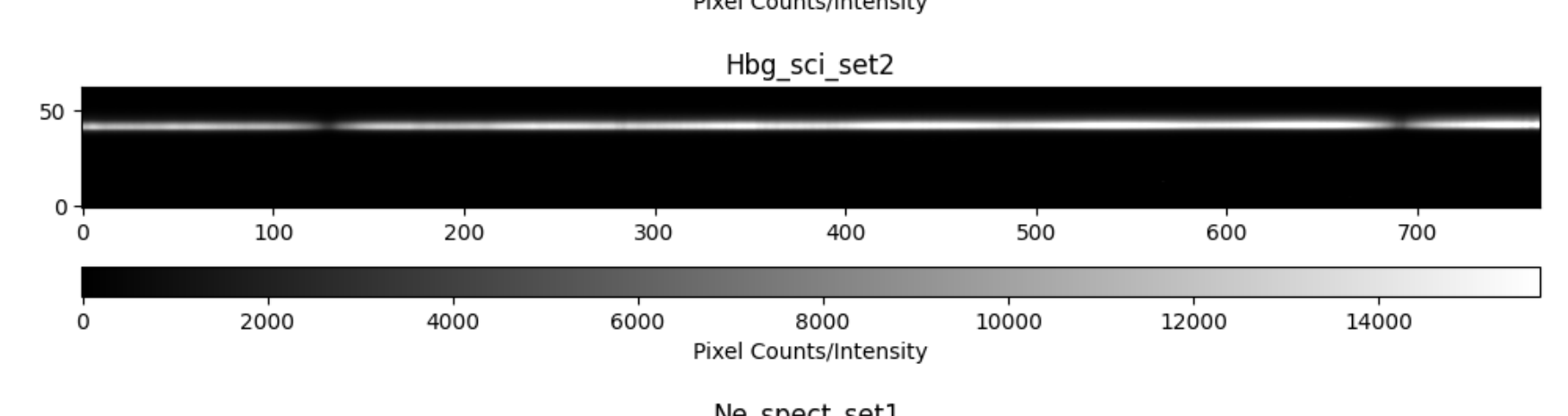
Now we have to finish calibrating our spectra (both sci and calibration) by dividing by the flat field

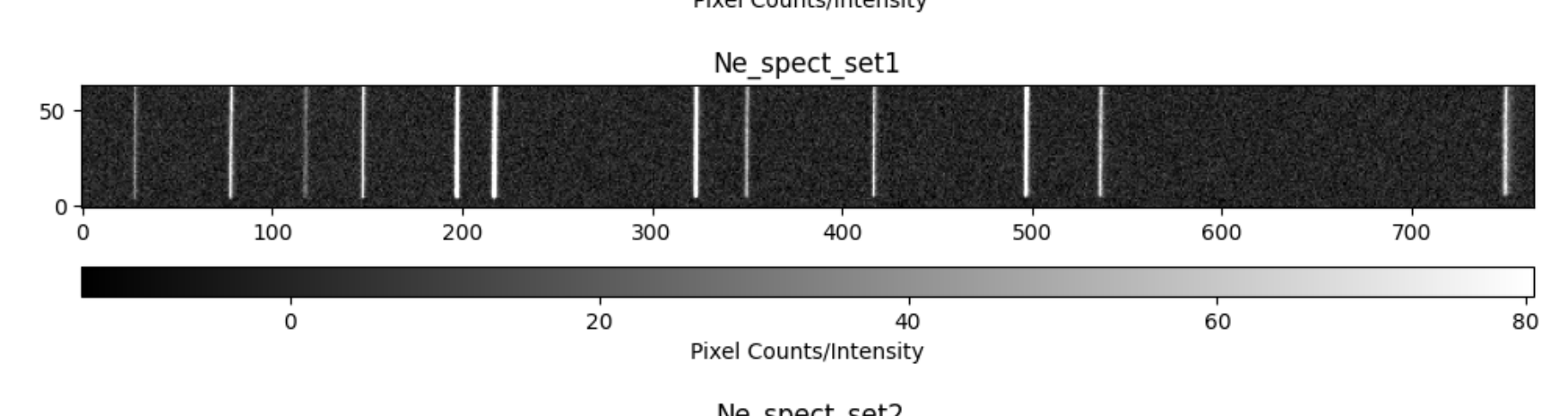
In [17]:
```python
def uncertainty_division(numerator, denominator, numerator_unc, denominator_unc):
    #epsilon = 1e-10  # Small value to prevent division by zero
    #safe_denominator = np.where(denominator == 0, epsilon, denominator)
    var1 = np.square(numerator_unc / denominator)
    var2 = np.square((numerator * denominator_unc) / np.square(denominator))
    return np.sqrt(var1 + var2)

if Polyfit:
    calibrated_science = sci_cut.copy()
    calibrated_science_unc = sci_cut_unc.copy()
    calibrated_spectra = spectra_cut.copy()
    calibrated_spectra_unc = spectra_cut_unc.copy()

    os.makedirs('finalized_spectra', exist_ok=True)
    for (key, value), (flat_key, flat_value) in zip(calibrated_science.items(), normalized_flat_fields.items()):
        calibrated_science[key] = value / flat_value
        calibrated_science_unc[key] = uncertainty_division(sci_cut[key], normalized_flat_fields[flat_key], sci_cut_unc[
        display_cut_spectrum(value, title=key)
        save_array_to_fits_file(value, path.join('finalized_spectra',f'calibrated_{key}.FIT'))
    for (key, value), (flat_key, flat_value), (skey,svalue) in zip(calibrated_spectra.items(), normalized_flat_fields.i
        calibrated_spectra[key] = value / flat_value
        calibrated_spectra_unc[key] = uncertainty_division(spectra_cut[skey], normalized_flat_fields[flat_key], spectra
        display_cut_spectrum(value, title=key)
        save_array_to_fits_file(value, path.join('finalized_spectra',f'calibrated_{key}.FIT'))
```

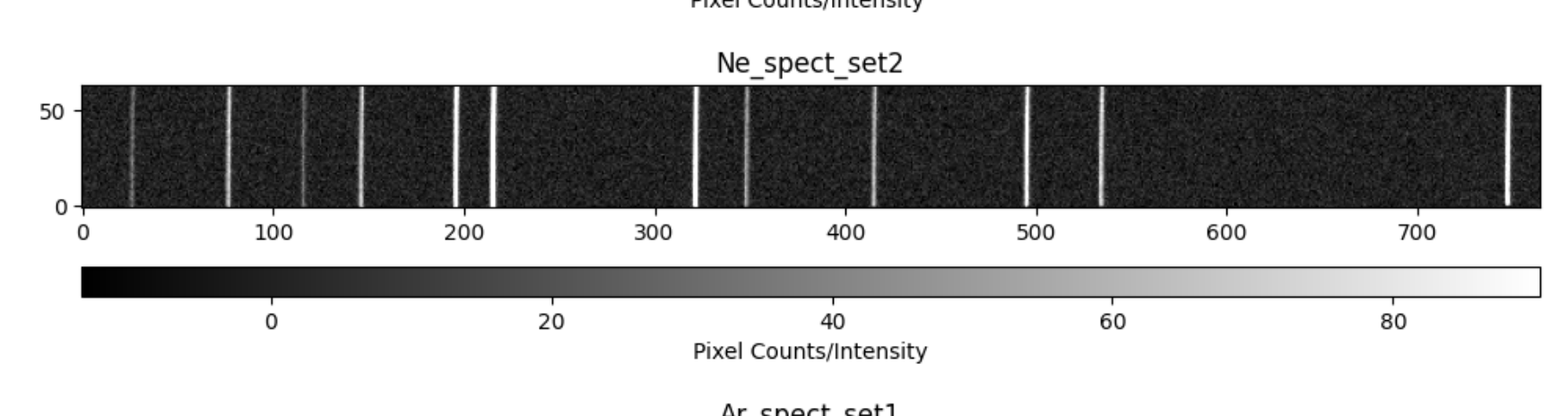


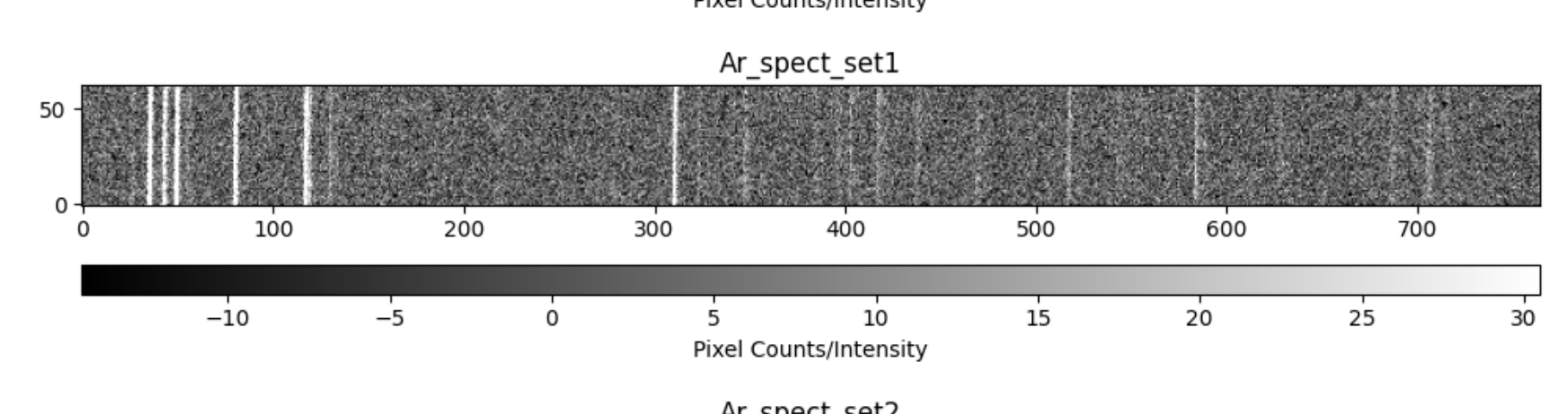


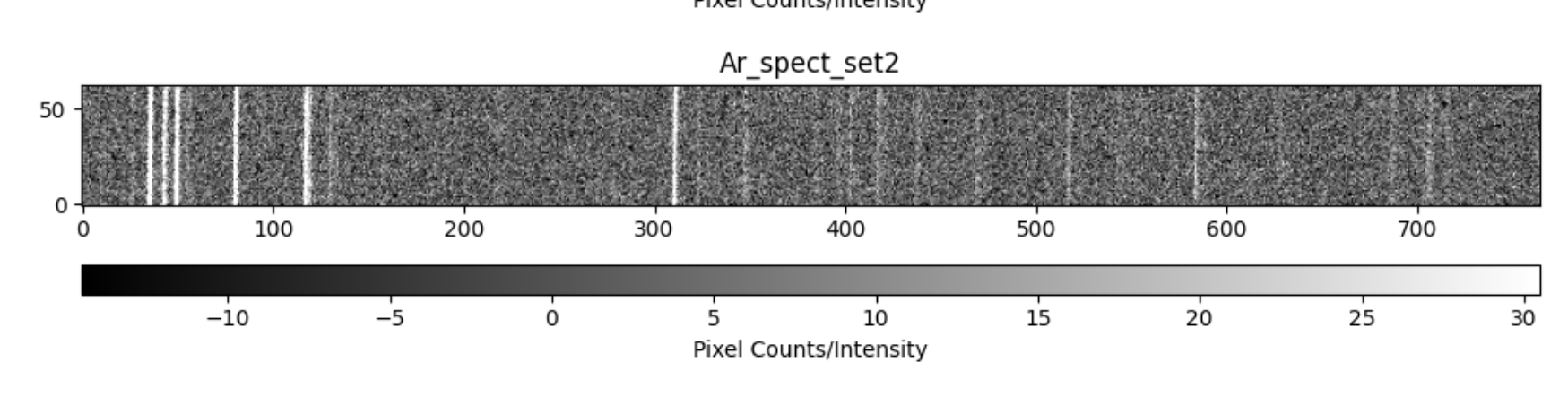












Method 2: Flat Field Division

Normalize The Flat Fields

Divide the flat fields by the median of the flat field to normalize them,
The intensity of the calibration spectrum does not matter as much to us so we are okay with proceeding cosi

In [18]:
```python
if FlatDivision:
    value = flats_cut['Ha_flat_set1']
    print(np.median(value),np.mean(value))
    plt.hist(value.flatten(), bins=50, range=(0, np.max(value)), histtype='step', color='blue')
    plt.plot()
```

In [19]:
```python
if FlatDivision:
    normalized_flat_fields = flats_cut.copy()
    normalized_flat_fields_unc = flats_cut_unc.copy()
    for (key,value), (ukey,uvalue) in zip(normalized_flat_fields.items(), normalized_flat_fields_unc.items()):
        print(np.median(value),np.mean(value))
        v = value / np.median(value)
        normalized_flat_fields[key] = v
        print(np.median(value),np.mean(value))
        normalized_flat_fields_unc[ukey] = uvalue / np.median(value)
        #print(np.median(normalized_flats_unc[ukey]))
```

Apply Flat Correction

In [20]:
```python
#Display normalized flat fields
if FlatDivision:
    for key, values in normalized_flat_fields.items():
        display_cut_spectrum(values, title=key+'norm')
```

In [21]:
```python
if FlatDivision:
    calibrated_science = sci_cut.copy()
    calibrated_science_unc = sci_cut_unc.copy()
    calibrated_spectra = spectra_cut.copy()
    calibrated_spectra_unc = spectra_cut_unc.copy()

    for (key,value), (ukey,uvalue), (fkey,fvalue), (ufkey, ufvalue) in zip(calibrated_science.items(),
                calibrated_science_unc.items(), normalized_flat_fields.items(), normalized_flat_fields_unc.items()):
        #print(key,ukey,fkey,ufkey)
        calibrated_science[key] = value / fvalue
        calibrated_science_unc[ukey] = flat_correction_uncertainty(value,uvalue,fvalue,ufvalue)

    for (key,value), (ukey,uvalue), (fkey,fvalue), (ufkey, ufvalue) in zip(calibrated_spectra.items(),
                calibrated_spectra_unc.items(), normalized_flat_fields.items(), normalized_flat_fields_unc.items()):
        calibrated_spectra[key] = value / fvalue
        calibrated_spectra_unc[ukey] = flat_correction_uncertainty(value,uvalue,fvalue,ufvalue)
```

In [22]:
```python
#Testing
if FlatDivision:
    value_2d = spectra_dcorr['neon_set1_15s']
    value = value_2d.flatten()
    print(len(value))
    uncert = spectra_dcorr_unc['neon_set1_15s']
    plot=False
    if plot==True:
        #plt.xlim(25000,26000)
        #plt.ylim(8200,8500)
        plt.scatter(x=np.arange(len(value)), y=value, marker='.', s=1,label='1')
        plt.errorbar(x=np.arange(len(value)), y=value, yerr=uncert.flatten(), fmt='.',label='2',capsize=2)
        plt.legend()
    max_index_flat = np.argmax(uncert)
    max_coordinates = np.unravel_index(max_index_flat, uncert.shape)
    max_uncert = uncert[max_coordinates]
    corr_value = value_2d[max_coordinates]
    print(f"Coords: {max_coordinates}, Value: {corr_value:.3f} +- {max_uncert:.3f}, MagRatio = {np.log10((corr_value/ma
```

In [23]:
```python
if FlatDivision:
    value = calibrated_science['Hbg_sci_set1']
    print(np.mean(value), np.std(value))
    plt.hist(value.flatten(), bins=50, range=(0, np.max(value)), histtype='step', color='blue')
    plt.plot()
```

Show Flat Corrected Data

In [24]:
```python
if FlatDivision:
    for key,value in calibrated_science.items():
        display_cut_spectrum(value,title=key)
        #display_cut_spectrum(calibrated_science_unc[key], title=key+'unc')
        #display_cut_spectrum(value - calibrated_science_unc[key], title=key+'+unc')
        save_array_to_fits_file(value, path.join('finalized_spectra',f'calibrated_{key}_V2.FIT'))
    for key,value in calibrated_spectra.items():
        display_cut_spectrum(value,title=key)
        #display_cut_spectrum(calibrated_spectra_unc[key], title=key+'unc')
        #display_cut_spectrum(value - calibrated_spectra_unc[key], title=key+'+unc')
        save_array_to_fits_file(value, path.join('finalized_spectra',f'calibrated_{key}_V2.FIT'))
```

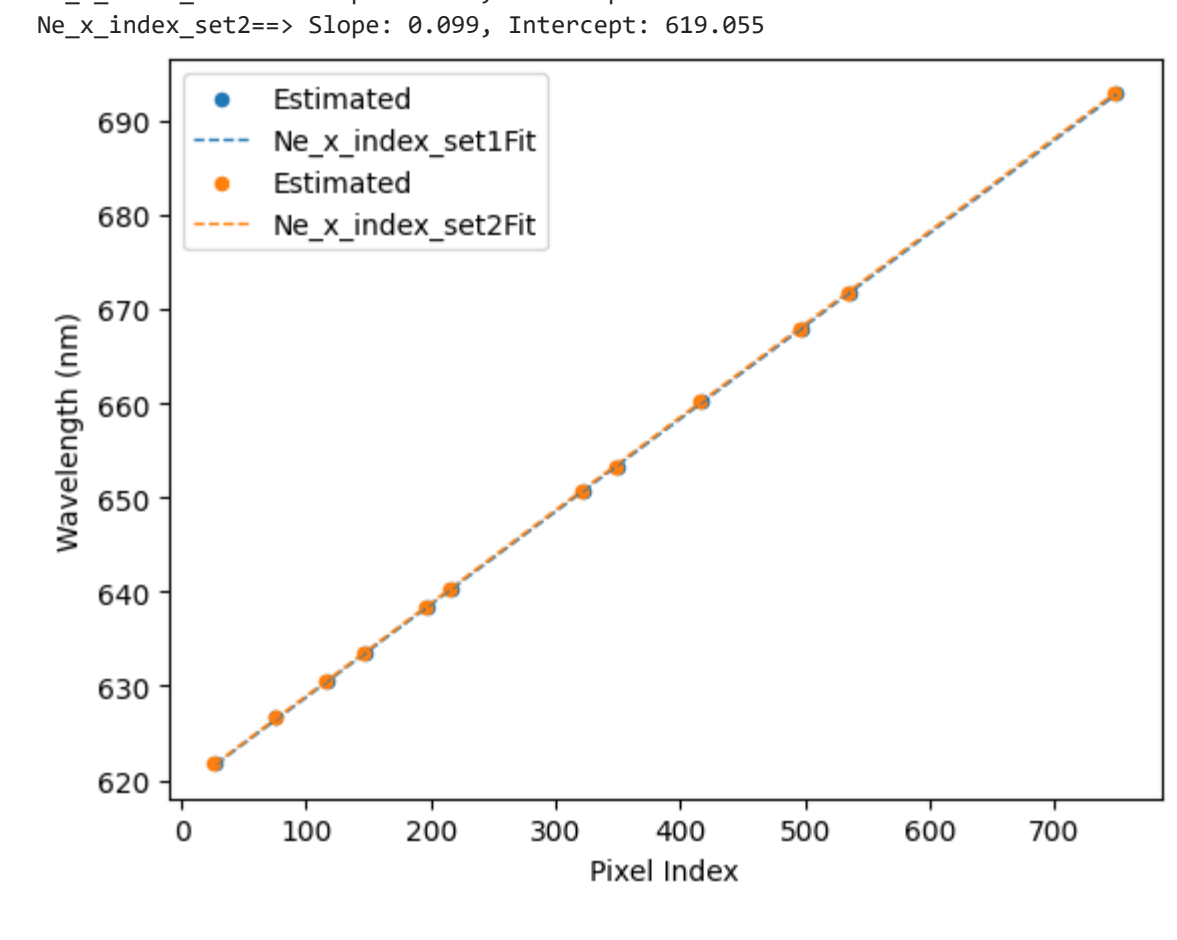
Wavelength Calibration

First, we matched spectral lines from our spectrum to emission lines of the actual neon spectrum, and we made a list of pixel positions that lined up with emission wavelengths from the NIST website. We will show how to fit a linear function to a plot of pixel positions vs wavelength to derive a wavelength calibration formula.

Neon

In [25]:
```python
def pixel_to_wavelength_index(x,slope,b):
    return slope*x + b

pix_vs_wavelength = pd.read_csv('Ne_pixels_vs_wavelength.csv')
print(pix_vs_wavelength)
print(pix_vs_wavelength.columns)
pix_vs_wavelength['Ne_x_index_set1'] = pix_vs_wavelength['x_pixel_set1'] - 1.5 #****** starts at 0.5 not 0
pix_vs_wavelength['Ne_x_index_set2'] = pix_vs_wavelength['x_pixel_set2'] - 1.5
uncertainty = np.ones(len(pix_vs_wavelength['x_pixel_set1']))

indices = ['Ne_x_index_set1', 'Ne_x_index_set2']
wavelengths = pix_vs_wavelength['wavelength']
calibration_stats = {}
for index in indices:
    x_index = pix_vs_wavelength[index]
    slope, intercept = np.polyfit(x_index, wavelengths, 1)
    calibration_stats[index] = [slope, intercept]
    plt.scatter(x_index, wavelengths, marker='o', s=20, label='Estimated')
    plt.plot(x_index, pixel_to_wavelength_index(x_index, slope, intercept), ls='--', lw=1.0, label=index+'Fit')
    #plt.errorbar(x_index, pixel_to_wavelength_index(x_index, slope, intercept), uncertainty, fmt='.', label=index+' Un
    plt.ylabel('Wavelength (nm)')
    plt.xlabel('Pixel Index')
    #plt.yticks(np.arange(590,671,10))
    #plt.xticks(np.arange(40,600,50))
    plt.legend()
    print(f'{index}==> Slope: {slope:.3f}, Intercept: {intercept:.3f}')
```

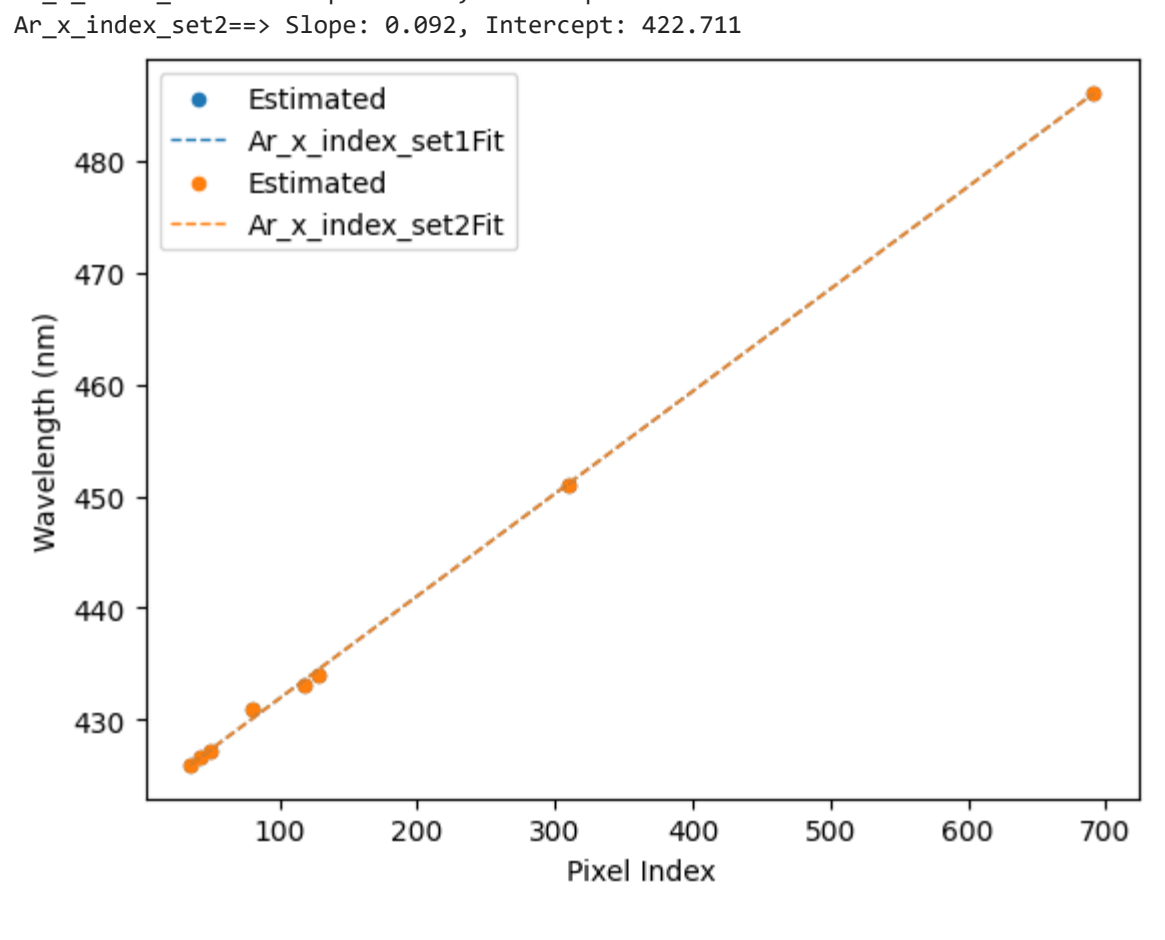
```
    x_pixel_set1  x_pixel_set2  wavelength  delta
0         29.164        27.550   621.72810  1.614
1         76.730        77.380   626.64950 -0.650
2        118.989       117.110   630.47893  1.879
3        149.220       147.635   633.44280  1.585
4        199.176       197.200   638.29900  1.976
5        218.647       216.600   640.22500  2.047
6        324.402       322.900   650.65277  1.502
7        351.690       349.416   653.28820  2.274
8        418.493       416.450   660.29007  2.043
9        498.650       496.600   667.82766  2.050
10       537.600       535.481   671.70430  2.119
11       750.965       748.945   692.94672  2.020
Index(['x_pixel_set1', 'x_pixel_set2', 'wavelength', 'delta'], dtype='object')
Ne_x_index_set1==> Slope: 0.099, Intercept: 618.937
Ne_x_index_set2==> Slope: 0.099, Intercept: 619.055
```



Argon

In [26]:
```python
pix_vs_wavelength = pd.read_csv('Ar_pixels_vs_wavelength.csv').dropna()
print(pix_vs_wavelength.columns)
pix_vs_wavelength['Ar_x_index_set1'] = pix_vs_wavelength['x_pixel'] - 1.5
pix_vs_wavelength['Ar_x_index_set2'] = pix_vs_wavelength['Ar_x_index_set1'].copy()
uncertainty = np.ones(len(pix_vs_wavelength['Ar_x_index_set1']))

print(pix_vs_wavelength)
indices = ['Ar_x_index_set1', 'Ar_x_index_set2']
wavelengths = pix_vs_wavelength['wavelength']
for index in indices:
    x_index = pix_vs_wavelength[index]
    slope, intercept = np.polyfit(x_index, wavelengths, 1)
    calibration_stats[index] = [slope, intercept]
    plt.scatter(x_index, wavelengths, marker='o', s=20, label='Estimated')
    plt.plot(x_index, pixel_to_wavelength_index(x_index, slope, intercept), ls='--', lw=1.0, label=index+'Fit')
    #plt.errorbar(x_index, pixel_to_wavelength_index(x_index, slope, intercept), ls='--', lw=1.0, label=index+'Fit')
    plt.ylabel('Wavelength (nm)')
    plt.xlabel('Pixel Index')
    #plt.yticks(np.arange(590,671,10))
    #plt.xticks(np.arange(40,600,50))
    plt.legend()
    print(f'{index}==> Slope: {slope:.3f}, Intercept: {intercept:.3f}')
```

```
Index(['x_pixel', 'wavelength', 'all_wavelengths'], dtype='object')
    x_pixel  wavelength  all_wavelengths  Ar_x_index_set1  Ar_x_index_set2
0   37.0000   425.93620        425.11850          35.5000          35.5000
1   44.3398   426.65265        425.93620          42.8398          42.8398
2   50.7600   427.21690        426.65265          49.2600          49.2600
3   82.2900   430.92385        427.21690          80.7900          80.7900
4  119.0000   433.11992        427.75279         117.5000         117.5000
5  129.6000   434.00000        430.92385         128.1000         128.1000
6  311.5000   451.07330        433.11992         310.0000         310.0000
7  692.3280   486.10000        434.00000         690.8280         690.8280
Ar_x_index_set1==> Slope: 0.092, Intercept: 422.711
Ar_x_index_set2==> Slope: 0.092, Intercept: 422.711
```



Science (Target Spectra) Gaussian Cutting

The idea here is, column by column:

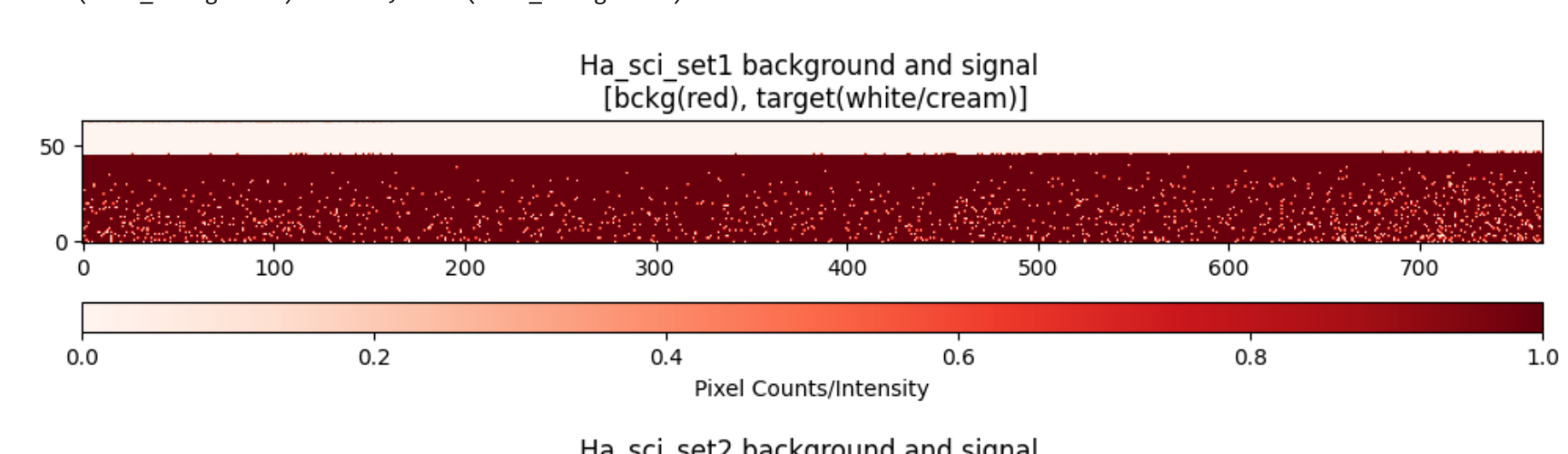
1. Find the mean and standard deviation
2. Define a cut that selects data points that are above/within a certain threshold range. These data points correspond to the signal from our target, which are the high signal pixels.
3. Move all data points that fall outside of this range into a new background array.
4. Find the mean and standard deviation of this array and remove any outliers.
5. The final signal will be calcualted as follows:
   $S' = S - N\mu_{background}$,
   where N is the number of pixels in the signal array.

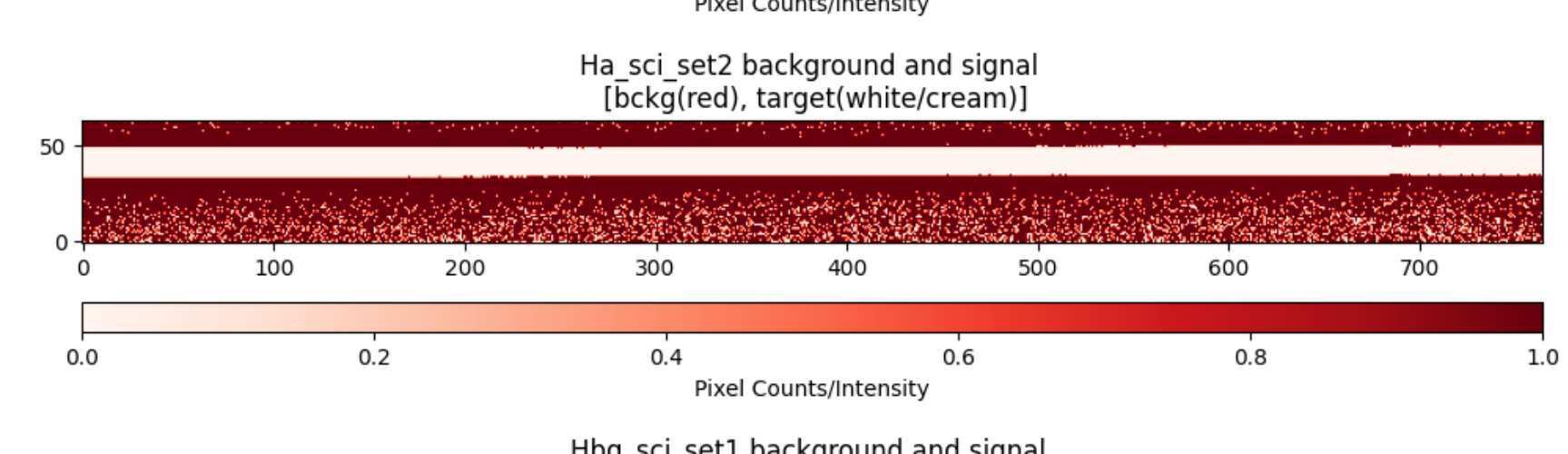
Signal and Background Thresholds

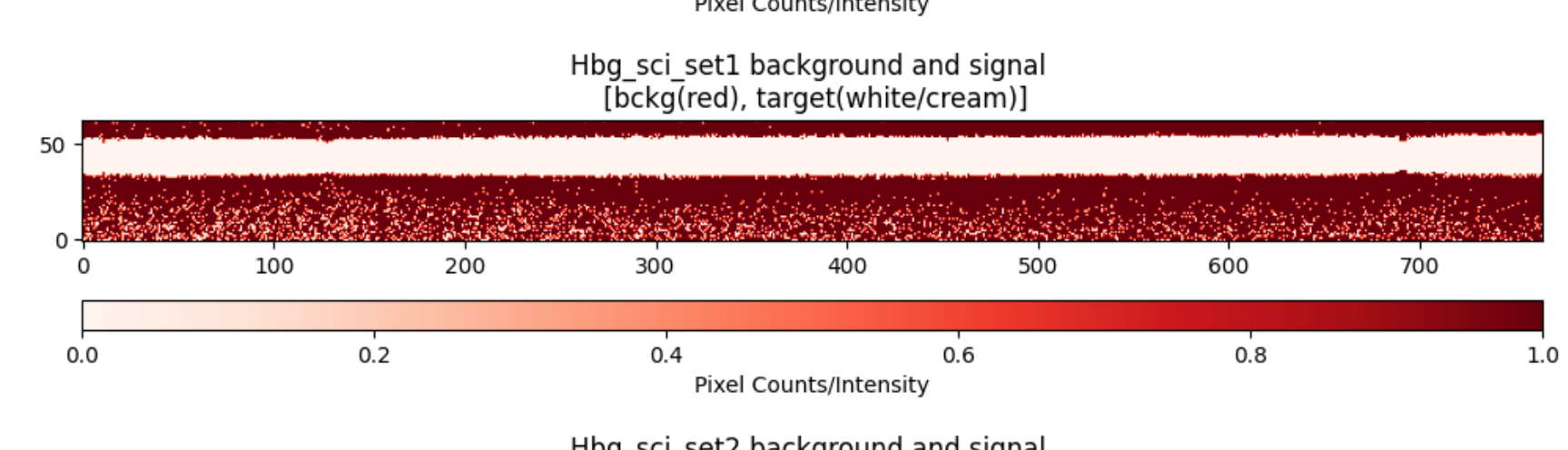
In [27]:
```python
def histograms_bkg(data,range1,range2,ylim=()):
    plt.figure(figsize=(10,3))
    plt.hist(data.flatten(),bins=50, range=range1, alpha=.8, color='red',label='3stds')
    plt.hist(data.flatten(),bins=50, range=range2, alpha=.5, color='blue',label='max')
    plt.title('background counts - does not show all')
    plt.grid(True)
    plt.legend()
    if ylim != ():
        plt.ylim(ylim)
```

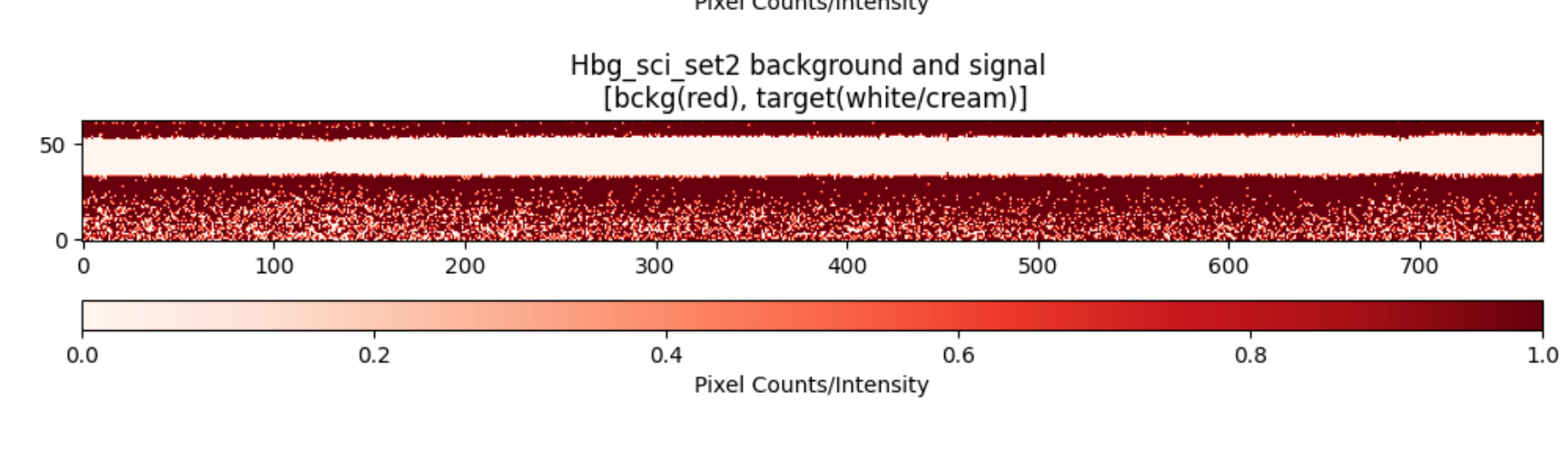
In [28]:
```python
target_signal = {key: [] for key in calibrated_science.keys()}
target_signal_unc = {key: [] for key in calibrated_science.keys()}
background_signal = {key: [] for key in calibrated_science.keys()}
background_signal_unc = {key: [] for key in calibrated_science.keys()}

for (key,value), (ukey,uvalue), (tkey,tvalue) in zip(calibrated_science.items(), calibrated_science_unc.items(), target
    #Look at statistics of data
    medians = np.median(value, axis=0)
    means = np.mean(value, axis=0)
    stds = np.std(value, axis=0)
    print(f"mean(means1)={np.mean(medians):.3f}, mean(means1)={np.mean(means):.3f}, mean(stds1)={np.mean(stds):.3f}")

    #Need to look at data above initial background threshold (median)
    initial_cut = medians + 100
    initial_mask = value > initial_cut
    data_cut = np.where(initial_mask, value, np.nan)
    data_unc_cut = np.where(initial_mask, uvalue, np.nan)
    medians = np.nanmedian(data_cut, axis=0)
    means = np.nanmean(data_cut, axis=0)
    stds = np.nanstd(data_cut, axis=0)
    print(f"mean(medians2)={np.mean(medians):.3f}, mean(means2)={np.mean(means):.3f}, mean(stds2)={np.mean(stds):.3f}")

    #Define Second Cut on Target
    target_min = means - stds #1
    target_max = means + 3*stds #3
    target_mask = (data_cut >= target_min) & (data_cut <= target_max)
    filtered_data = np.where(target_mask, data_cut, np.nan)
    filtered_data_unc = np.where(target_mask, data_unc_cut, np.nan)
    #Save to dict
    target_signal[key] = filtered_data
    target_signal_unc[key] = filtered_data_unc
    #print(filtered_data, '\n\n\n\n', filtered_data_unc)

    #Background Data
    bkg_mask_nan = np.isnan(filtered_data)
    background = np.where(bkg_mask_nan, value, np.nan)
    background_unc = np.where(bkg_mask_nan, uvalue, np.nan)
    background = np.where(background == 0, np.nan, background)
    # display_cut_spectrum(np.where(bkg_mask_nan, 1, 0),
    #    title=key+' background and signal \n [bckg(red), target(white/cream)]', cmap='Reds')
    # display_cut_spectrum(np.where(bkg_mask_nan, value, 0),
    #    title=key+' background and signal actual values \n [bckg(red), target(white/cream)]', cmap='Reds')

    #Background Statistics
    medians = np.nanmedian(background, axis=0)
    means = np.nanmean(background, axis=0)
    stds = np.nanstd(background, axis=0)
    maximum = np.max(np.where(np.isnan(background), 0, background))
    mu=np.mean(means)
    sig=np.std(stds)
    print(f"mean(mediansbkg)={np.mean(medians):.3f}, mean(meansbkg)={mu:.3f}, mean(stdsbkg)={sig:.3f}, max={maximum}",e
    #histograms_bkg(background, range1=[mu-sig,mu+3*sig], range2=[-10,maximum], ylim=(0,1000))

    #Now define a cut on the background to remove excess noise
    bkg_min = 0
    bkg_max = means + 1*stds #1
    bkg_mask = (background > bkg_min) & (background <= bkg_max)
    background_signal[key] = np.where(bkg_mask, background, np.nan)
    background_signal_unc[key] = np.where(bkg_mask, background_unc, np.nan)
    means = np.nanmean(background, axis=0)
    stds = np.nanstd(background, axis=0)
    print(f"mean(mean_background)={np.mean(means):.3f}, mean(stds_background)={np.mean(stds):.3f}",end='\n\n')
    display_cut_spectrum(np.where(bkg_mask, 1, 0),
        title=key+' background and signal \n [bckg(red), target(white/cream)]', cmap='Reds')
```

```
mean(means1)=15.113, mean(means1)=514.527, mean(stds1)=1079.201
mean(medians2)=1651.923, mean(means2)=1871.436, mean(stds2)=1341.354
mean(mediansbkg)=12.152, mean(meansbkg)=30.974, mean(stdsbkg)=13.384, max=613.0443383625844

mean(mean_background)=30.974, mean(stds_background)=67.944

mean(means1)=13.263, mean(means1)=630.461, mean(stds1)=1380.310
mean(medians2)=2883.489, mean(means2)=2522.680, mean(stds2)=1730.319
mean(mediansbkg)=10.291, mean(meansbkg)=38.494, mean(stdsbkg)=26.925, max=1070.9692582616638

mean(mean_background)=38.494, mean(stds_background)=97.514

mean(means1)=16.428, mean(means1)=1061.323, mean(stds1)=2980.025
mean(medians2)=2100.928, mean(means2)=4332.400, mean(stds2)=4748.176
mean(mediansbkg)=11.727, mean(meansbkg)=17.776, mean(stdsbkg)=120.426, max=21733.566132715965

mean(mean_background)=17.776, mean(stds_background)=26.045

mean(means1)=14.885, mean(means1)=1136.141, mean(stds1)=3209.735
mean(medians2)=2034.907, mean(means2)=4524.162, mean(stds2)=5111.041
mean(mediansbkg)=10.070, mean(meansbkg)=15.268, mean(stdsbkg)=50.188, max=226.27237780124344

mean(mean_background)=15.268, mean(stds_background)=21.609
```









Adjusted Signal

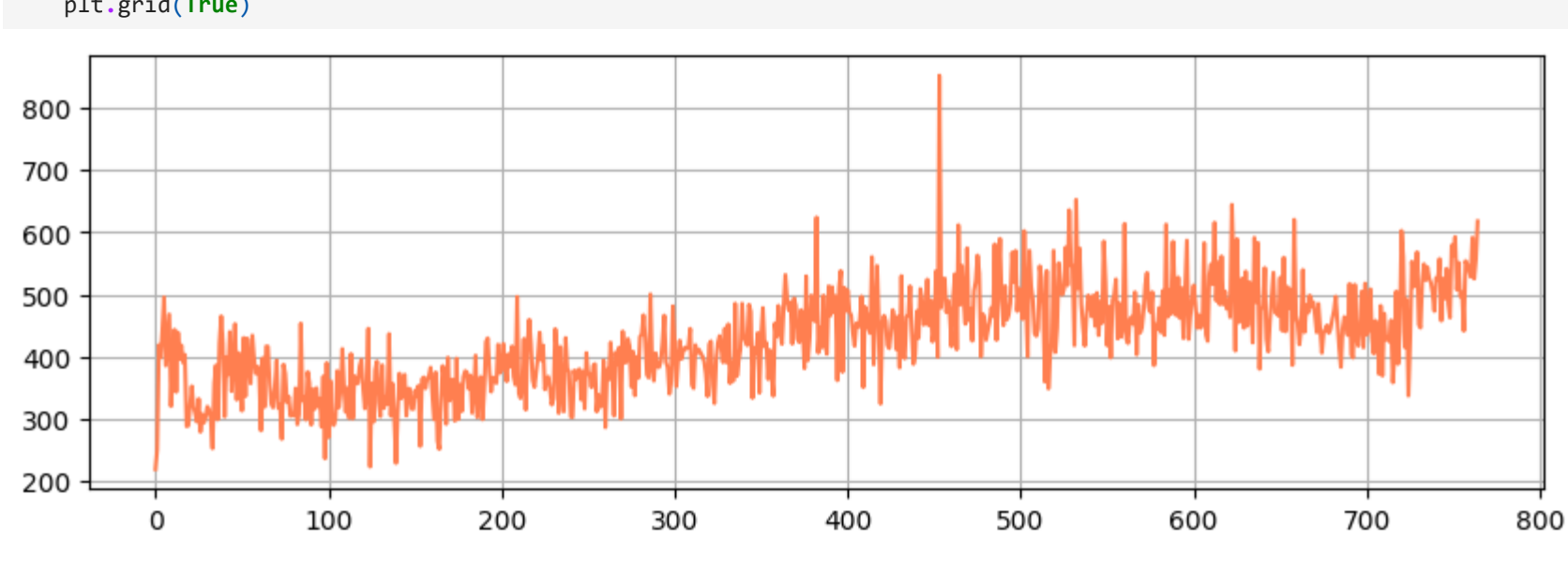
In [29]:
```python
def weighted_mean(data,uncertainties,axis):
    uncertainties = np.nan_to_num(uncertainties, nan=np.inf)
    # Weights
    weights = 1 / uncertainties**2
    # Weighted Mean
    weighted_mean = np.nansum(data * weights, axis=axis) / np.nansum(weights, axis=axis)
    # Uncertainty in Mean
    uncertainty_mean = np.sqrt(1 / np.nansum(weights, axis=axis))
    return weighted_mean, uncertainty_mean
```

In [30]:
```python
finalized_science = {key: [] for key in calibrated_science.keys()}
finalized_science_unc = {key: [] for key in calibrated_science.keys()}

for (key,value), (ukey,uvalue), (bkey, bvalue), (ubkey, ubvalue) in zip(target_signal.items(),
                target_signal_unc.items(), background_signal.items(), background_signal_unc.items()):
    #print(key,ukey,bkey,ubkey)'

    # Calculate S'
    target_sum = np.nansum(value, axis=0)
    target_pixels = np.sum(~np.isnan(value), axis=0) # Number of target pixels per column
    #print(target_pixels)
    background_mean = weighted_mean(bvalue, ubvalue, axis=0)[0] # mean of each column
    #print(background_mean)
    finalized_science[key] = target_sum - (target_pixels*background_mean)

    # Calculate uncertainties
    target_sum_unc = quadrature_unc(uvalue, axis=0)
    #print(np.nansum(target_sum_unc))
    background_mean_unc = weighted_mean(bvalue, ubvalue, axis=0)[1]
    #print(np.nansum(background_mean_unc))
    finalized_science_unc[key] = darks_correction_uncertianty(target_sum_unc, target_pixels*background_mean_unc)
```

Initial Check

In [31]:
```python
#Raw target (without background subtraction)
plot = False
data = target_signal['Ha_sci_set1']
data_unc = target_signal_unc['Ha_sci_set1']
y = normalize_to_unity(data, array_uncert=data_unc, return_uncert=False)
yerror = normalize_to_unity(data, array_uncert=data_unc, return_uncert=True)
x = np.arange(len(y))
if plot:
    plt.figure(figsize=(10,6))
    plt.plot(x,y, color='coral')
    plt.yticks(np.arange(0,1.1,.1))
    #plt.errorbar(x,y,yerr=yerror, fmt='.',color='coral', alpha=.6)
    #plt.xlim(370,400)
    #plt.ylim(.2,.5)
    plt.grid(True)
```

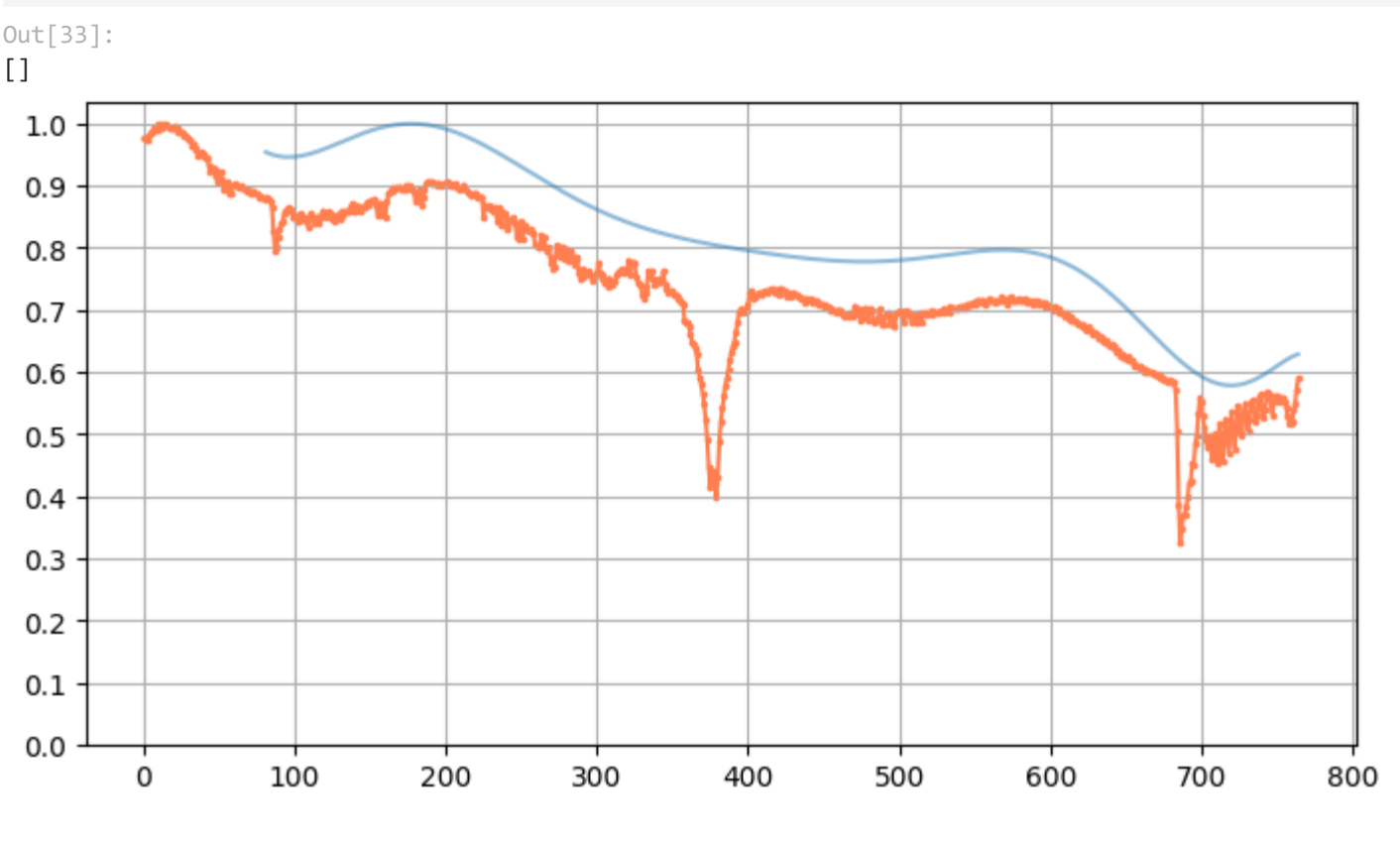
In [32]:
```python
plot = True
ploterr = False
data = background_signal['Hbg_sci_set2']
data_unc = background_signal_unc['Hbg_sci_set2']
y = np.nansum(data, axis=0)
yerror = np.sqrt(np.nansum(np.square(data_unc), axis=0))
x = np.arange(len(y))
if plot:
    plt.figure(figsize=(10,3))
    plt.plot(x,y, color='coral')
    if ploterr: plt.errorbar(x,y,yerr=yerror, fmt='.',color='coral', alpha=.6)
    #plt.xlim(370,400)
    #plt.xlim(685,710)
    #plt.ylim(.2,.5)
    #plt.ylim(.45,.9)
    plt.grid(True)
```

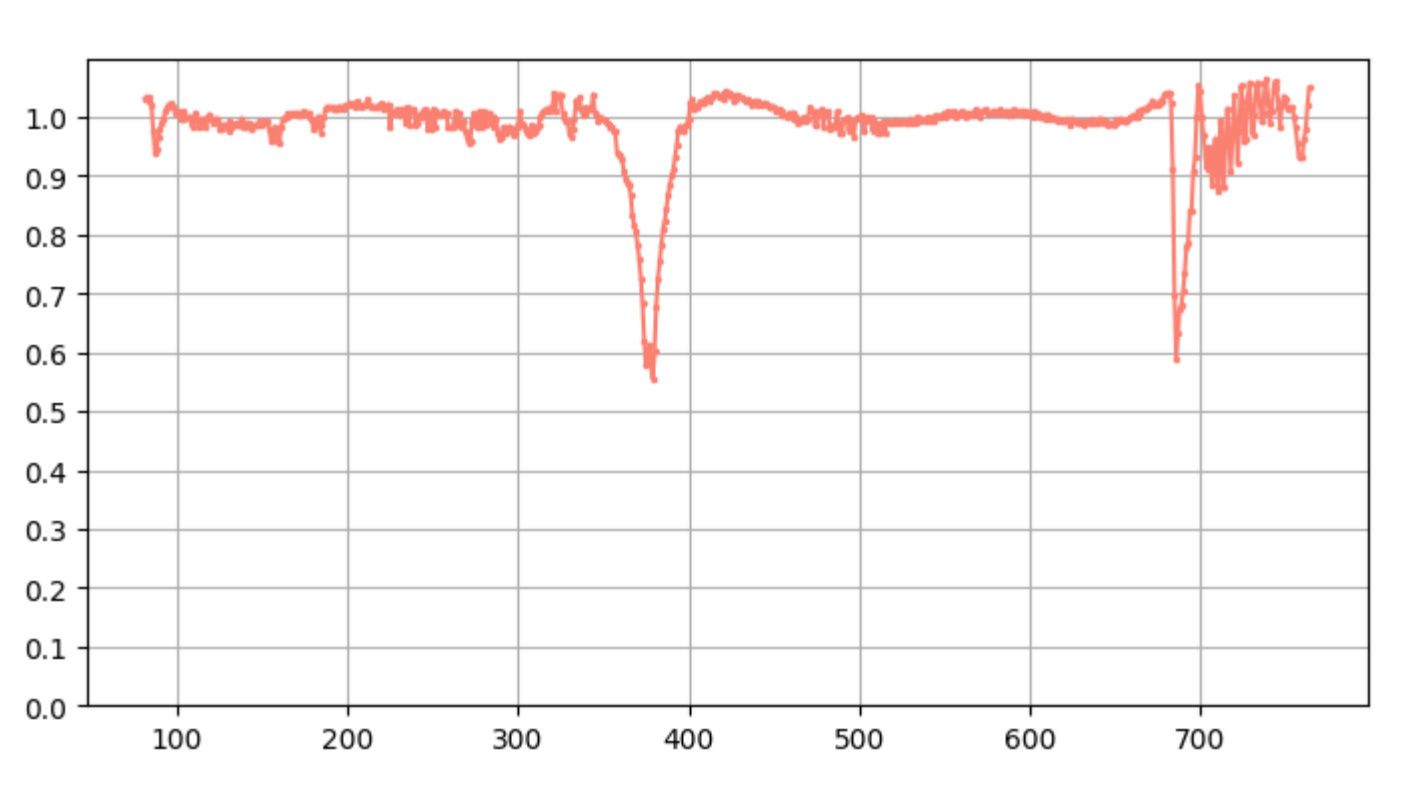


If you see a dip in the background that might mean we dont have all the signal. If there is a spike maybe it is emission

In [33]:
```python
data = finalized_science['Ha_sci_set2']
data_unc = finalized_science_unc['Ha_sci_set2']

y = normalize_to_unity_1d(data, data_unc)[0]
yerror = normalize_to_unity_1d(data, data_unc)[1]
x = np.arange(len(y))
mask = ((x <= 350) | (x >= 438)) & ((x <= 91) | (x >= 111)) & ((x <= 684) | (x >= 710))
#mask = ((x >= 200)) & ((x <= 690) | (x >= 710)) & (x > 200)
good_data = x > 80
degree = 10
coefficents = np.polyfit(x=x[mask], y=y[mask], deg=degree)
polynomial_fit = np.polyval(coefficents, x[good_data])

fig, (ax1,ax2) = plt.subplots(2,1,figsize=(8,8))
ax1.plot(x,y, color='coral')
scatter = ax1.scatter(x,y, color='coral', marker='.', s=10)
ax1.plot(x[good_data], polynomial_fit/np.max(polynomial_fit), alpha=0.5)
ax1.set_yticks(np.arange(0,1.1,.1))
ax1.grid(True)
mplcursors.cursor(scatter, hover=False).connect(
    "add", lambda sel: sel.annotation.set_text(f"({x[sel.index]}, {y[sel.index]})")
)

ax2.plot(x[good_data],y[good_data]/polynomial_fit, marker='.', ms=3,color='salmon')
ax2.set_yticks(np.arange(0,1.1,.1))
ax2.set_ylim(0,1.1)
ax2.grid(True)

plt.subplots_adjust(left=0.1, right=0.9, top=0.99, bottom=0.1)
plt.plot()
```

Out[33]:
```
[]
```





Plot Calibrated Spectrum and Science Spectra

Key initialization

In [34]:
```python
for key in calibration_stats.items(): print(key)
```

```
('Ne_x_index_set1', [0.09855494474948556, 618.9369313375616])
('Ne_x_index_set2', [0.0987205118456858, 619.054802588689])
('Ar_x_index_set1', [0.09165163100599681, 422.7107994766804])
('Ar_x_index_set2', [0.09165163100599681, 422.7107994766804])
```

In [35]:
```python
##### Normalize Spectra to be capped at 1 (for visualizing separately)
normalized_spectra = {'Ne_set1' : normalize_to_unity(calibrated_spectra['Ne_spect_set1'], ctype='median'),
                      'Ne_set2' : normalize_to_unity(calibrated_spectra['Ne_spect_set2'], ctype='median'),
                      'Ha_set1' : normalize_to_unity_1d(finalized_science['Ha_sci_set1'], finalized_science_unc['Ha_sci
                      'Ha_set2' : normalize_to_unity_1d(finalized_science['Ha_sci_set2'], finalized_science_unc['Ha_sci
                      'Ar_set1' : normalize_to_unity(calibrated_spectra['Ar_spect_set1'], ctype='median'),
                      'Ar_set2' : normalize_to_unity(calibrated_spectra['Ar_spect_set2'], ctype='median'),
                      'Hbg_set1' : normalize_to_unity_1d(finalized_science['Hbg_sci_set1'], finalized_science_unc['Hbg_
                      'Hbg_set2' : normalize_to_unity_1d(finalized_science['Hbg_sci_set2'], finalized_science_unc['Hbg_
                     }
normalized_spectra_unc = {'Ne_set1' : normalize_to_unity(calibrated_spectra['Ne_spect_set1'], ctype='median', return_un
                                      array_uncert=calibrated_spectra_unc['Ne_spect_set1']),
                          'Ne_set2' : normalize_to_unity(calibrated_spectra['Ne_spect_set2'], ctype='median', return_un
                                      array_uncert=calibrated_spectra_unc['Ne_spect_set2']),
                          'Ha_set1' : normalize_to_unity_1d(finalized_science['Ha_sci_set1'], finalized_science_unc['Ha
                          'Ha_set2' : normalize_to_unity_1d(finalized_science['Ha_sci_set2'], finalized_science_unc['Ha
                          'Ar_set1' : normalize_to_unity(calibrated_spectra['Ar_spect_set1'], ctype='median', return_un
                                      array_uncert=calibrated_spectra_unc['Ar_spect_set1']),
                          'Ar_set2' : normalize_to_unity(calibrated_spectra['Ar_spect_set2'], ctype='median', return_un
                                      array_uncert=calibrated_spectra_unc['Ar_spect_set2']),
                          'Hbg_set1' : normalize_to_unity_1d(finalized_science['Hbg_sci_set1'], finalized_science_unc['
                          'Hbg_set2' : normalize_to_unity_1d(finalized_science['Hbg_sci_set2'], finalized_science_unc['
                         }
calibration_indices = {'Ne_set1' : 'Ne_x_index_set1',
                      'Ne_set2' : 'Ne_x_index_set2',
                      'Ha_set1' : 'Ne_x_index_set1',
                      'Ha_set2' : 'Ne_x_index_set2',
                      'Ar_set1' : 'Ar_x_index_set1',
                      'Ar_set2' : 'Ar_x_index_set2',
                      'Hbg_set1' : 'Ar_x_index_set1',
                      'Hbg_set2' : 'Ar_x_index_set2',
                     }
label_list = ['Neon Set 1', 'Neon Set 2', 'H$\\alpha$ Set 1', 'H$\\alpha$ Set 2', 'Ar Set 1', 'Ar Set 2', 'H$\\beta , \
labels = {}
for label, (key,value) in zip(label_list, calibration_indices.items()): labels[key] = label
```

Single Plot

All you have to do is change the "spectrum_label" and you can also decide if you want to plot the error bars.

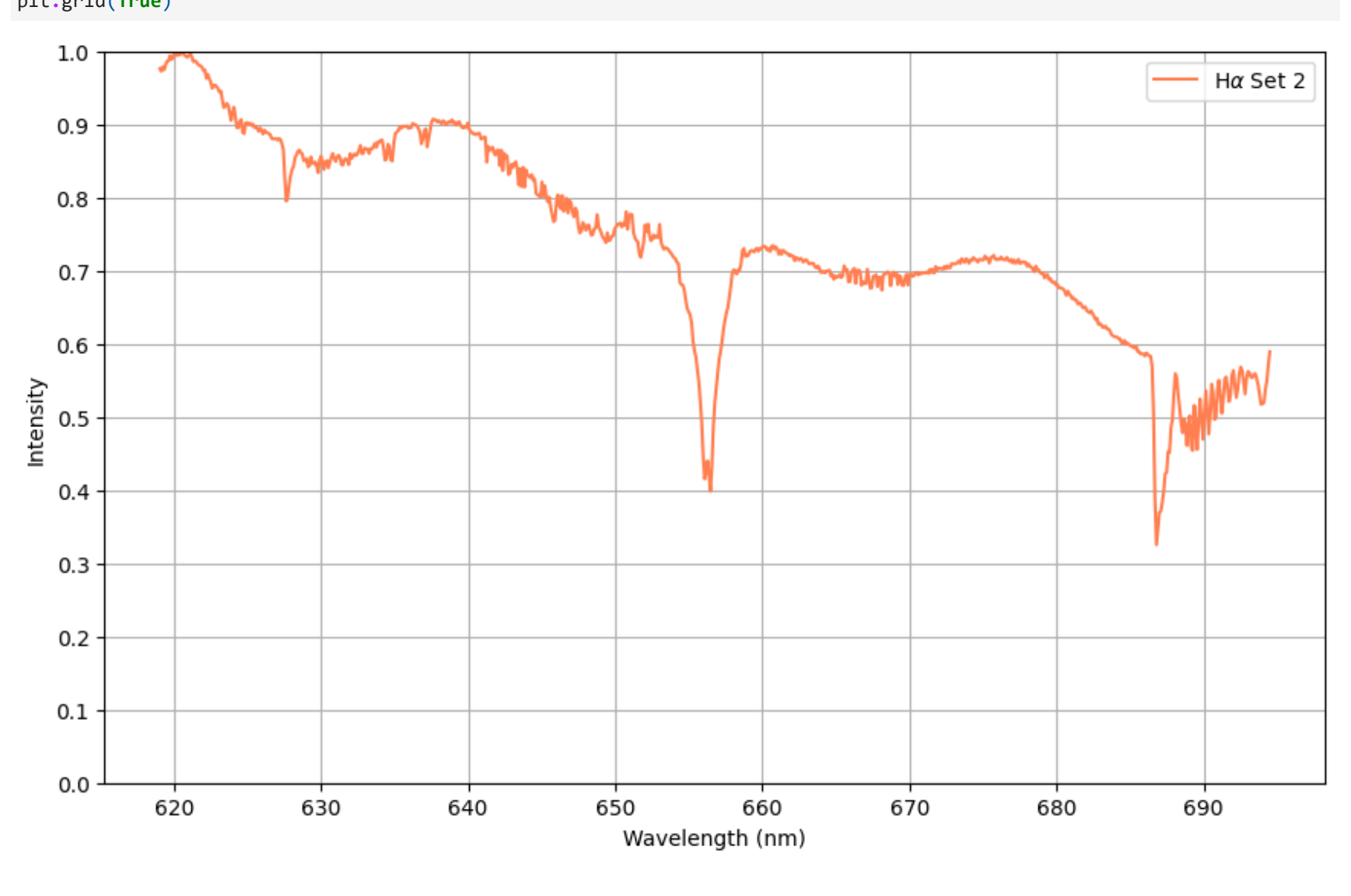
In [36]:

```python
for spectrum_label,index_set in calibration_indices.items():
    print(f"Label: {spectrum_label}")
```

```
Label: Ne_set1
Label: Ne_set2
Label: Ha_set1
Label: Ha_set2
Label: Ar_set1
Label: Ar_set2
Label: Hbg_set1
Label: Hbg_set2
```

In [37]:

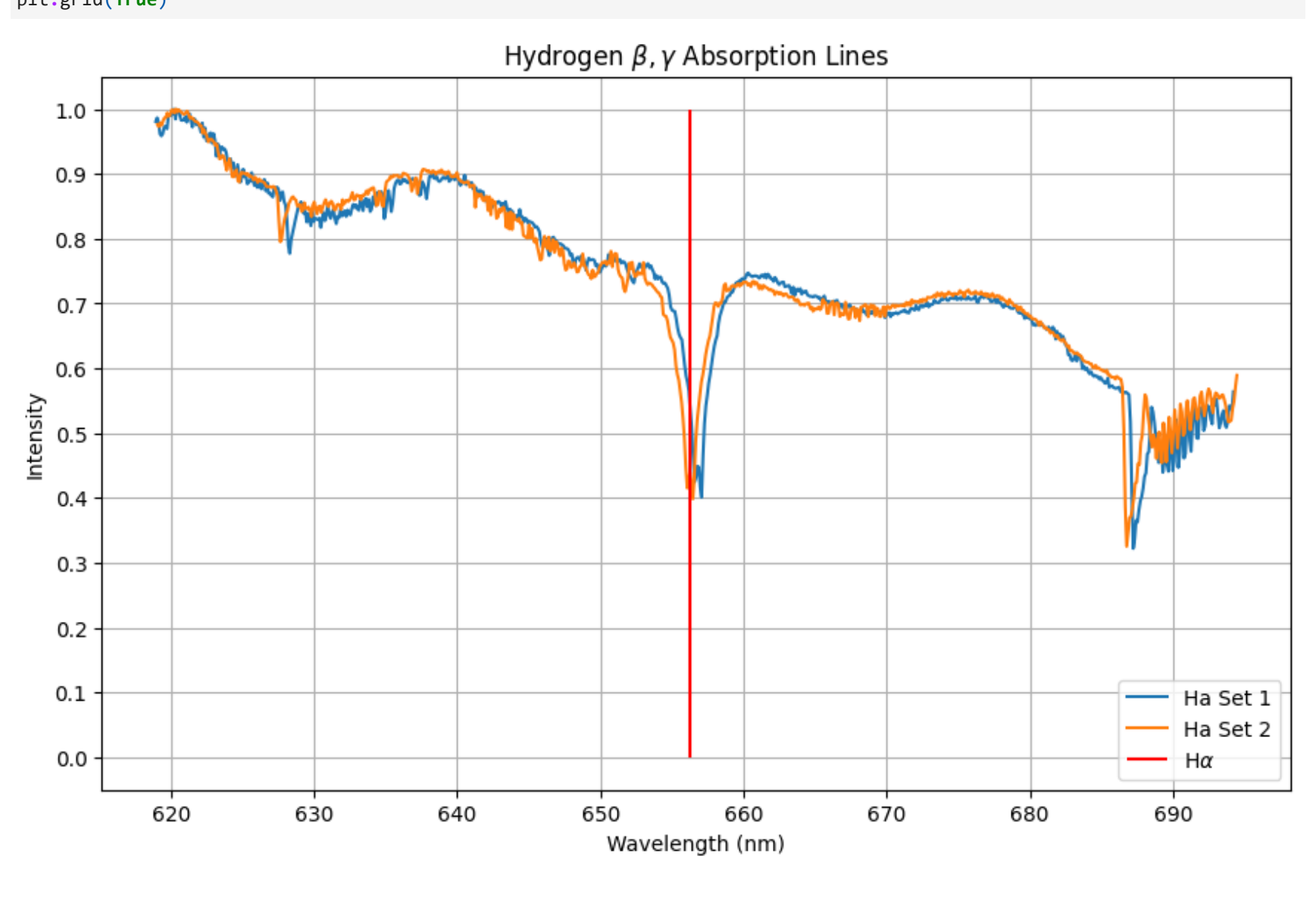
```python
spectrum_label = 'Ha_set2'
plt_error=False

index_set = calibration_indices[spectrum_label]
spectrum_index = np.arange(len(normalized_spectra[spectrum_label])) #pixel index
spectrum_wavelength_index = pixel_to_wavelength_index(spectrum_index, *calibration_stats[index_set]) #pixel index --> w
'''
#Find Peaks
peaks, _ = find_peaks(normalized_spectra[spectrum_label], height=(0.0,.4))  # Customize height threshold if needed
#print(peaks)
for peak in peaks:
    plt.annotate(f'{spectrum_wavelength_index[peak]:.1f} nm',
                 (spectrum_wavelength_index[peak], normalized_spectra[spectrum_label][peak]),
                 xytext=(0, 4), textcoords='offset points', ha='center', fontsize=7, color='red')
#'''
color = 'coral'
fig, ax = plt.subplots(figsize=(10,6))
ax.plot(spectrum_wavelength_index, normalized_spectra[spectrum_label], ms=5, c=color, label=labels[spectrum_label])
if plt_error:
    ax.errorbar(spectrum_wavelength_index, normalized_spectra[spectrum_label], yerr=normalized_spectra_unc[spectrum_lab
                fmt='.', c=color, label=' Uncertainty', alpha=.5)
ax.set_ylabel('Intensity')
ax.set_xlabel('Wavelength (nm)')
ax.set_ylim(0.0,1.0)
ax.set_yticks(ticks=np.arange(0.0,1.1,0.1))
#ax.set_xlim(np.min(spectrum_wavelength_index), np.max(spectrum_wavelength_index))
#ax.xaxis.set_major_locator(MaxNLocator(integer=True, prune='both', nbins=150))
#ax.set_xticks(ticks=np.arange(np.min(spectrum_wavelength_index), np.max(spectrum_wavelength_index),5))#(ax.get_xticks(
#ax.set_xticklabels(ax.get_xticklabels(), rotation=90)
#plt.xlim(600,680)

#Zoom in to a regiOn
#alpha:
#ax.set_xlim(655, 657.5)
#ax.set_ylim(0.3,0.7)
#beta:
#plt.xlim(485,487)
#plt.ylim(0.25,0.6)
#gamma:
#plt.xlim(433,436)
#plt.ylim(0.1,0.6)

#ax.set_xlim(490,500)
#ax.set_ylim(.00,.2)

#ax.set_xlim(686,688)
#ax.set_ylim(.4,.9)
#plt.xlim(685,690)
#plt.ylim(0.2,0.7)

#Plot Balmer Lines:
#plt.vlines(x=656.3, ymin=0, ymax=1, color='r', label='H$\\alpha$')
#plt.vlines(x=486.1, ymin=0, ymax=1, color='blue', label='H$\\beta$ (486.1nm)', alpha=.5)
#plt.vlines(x=434, ymin=0, ymax=1, color='purple', label='H$\\gamma$ (434 nm)',  alpha=.5)
#plt.vlines(x=687, ymin=0, ymax=1, color='lightgreen', label='Telluric O2')
plt.subplots_adjust(left=0.1, right=0.9, top=0.90, bottom=0.1)
plt.legend()
plt.grid(True)
```



Stacked Plots

In [38]:

```python
#View Stacked Spectra
plot_error = False
spectrum_labels = ['Ha_set1','Ha_set2'] #['Ha_set1','Ha_set2','Hbg_set1', 'Hbg_set2']
index_sets=[]
for i in range(len(spectrum_labels)):
    index_sets.append(calibration_indices[spectrum_labels[i]])
fig, ax = plt.subplots(figsize=(10,6))
for (s_label, i_set) in zip(spectrum_labels, index_sets):
    spectrum_index = np.arange(len(normalized_spectra[s_label]))
    spectrum_wavelength_index = pixel_to_wavelength_index(spectrum_index, *calibration_stats[i_set])
    line, = ax.plot(spectrum_wavelength_index, normalized_spectra[s_label], label=s_label.replace('_',' ').replace('b',
    line_color = line.get_color()
    if plot_error:
        ax.errorbar(spectrum_wavelength_index, normalized_spectra[s_label], normalized_spectra_unc[s_label],
                fmt='.', label=s_label+' Uncertainty', color=line_color, alpha=.8)
'''
#Find Peaks
for spectrum_label in spectrum_labels:
    peaks, _ = find_peaks(normalized_spectra[spectrum_label], height=(.3,.4))  # Customize height threshold if needed
    print(peaks)
    for peak in peaks:
        plt.annotate(f'{spectrum_wavelength_index[peak]:.1f} nm',
                     (spectrum_wavelength_index[peak], normalized_spectra[spectrum_label][peak]),
                     xytext=(0, 4), textcoords='offset points', ha='center', fontsize=7, color='red')
'''
plt.title('Hydrogen $\\beta , \\gamma$ Absorption Lines')
plt.ylabel('Intensity')
plt.xlabel('Wavelength (nm)')
plt.yticks(np.arange(0.0,1.1,0.1))
#plt.xticks(np.arange(420,500,5),rotation=90)
#plt.xticks(np.arange(420,500,5),rotation=90)

#Zoom in to a regiOn
#alpha:
#ax.set_xlim(655.5, 658)
#ax.set_ylim(0.25,0.55)
#beta:
#plt.xlim(484,488)
#plt.ylim(0.3,0.9)
#gamma:
#plt.xlim(433,436.5)
#plt.ylim(0.05,0.7)
#O2:
#plt.xlim(685,690)
#plt.ylim(0.2,0.7)

#Plot Balmer Lines:
plt.vlines(x=656.3, ymin=0, ymax=1, color='r', label='H$\\alpha$')
#plt.vlines(x=486.1, ymin=0, ymax=1, color='blue', label='H$\\beta$ (486.1nm)', alpha=.5)
#plt.vlines(x=434, ymin=0, ymax=1, color='purple', label='H$\\gamma$ (434 nm)',  alpha=.5)
#plt.vlines(x=687, ymin=0, ymax=1, color='lightgreen', label='Telluric O2')

plt.legend(loc='lower right')
plt.subplots_adjust(left=0.1, right=0.9, top=0.90, bottom=0.1)
plt.grid(True)
```

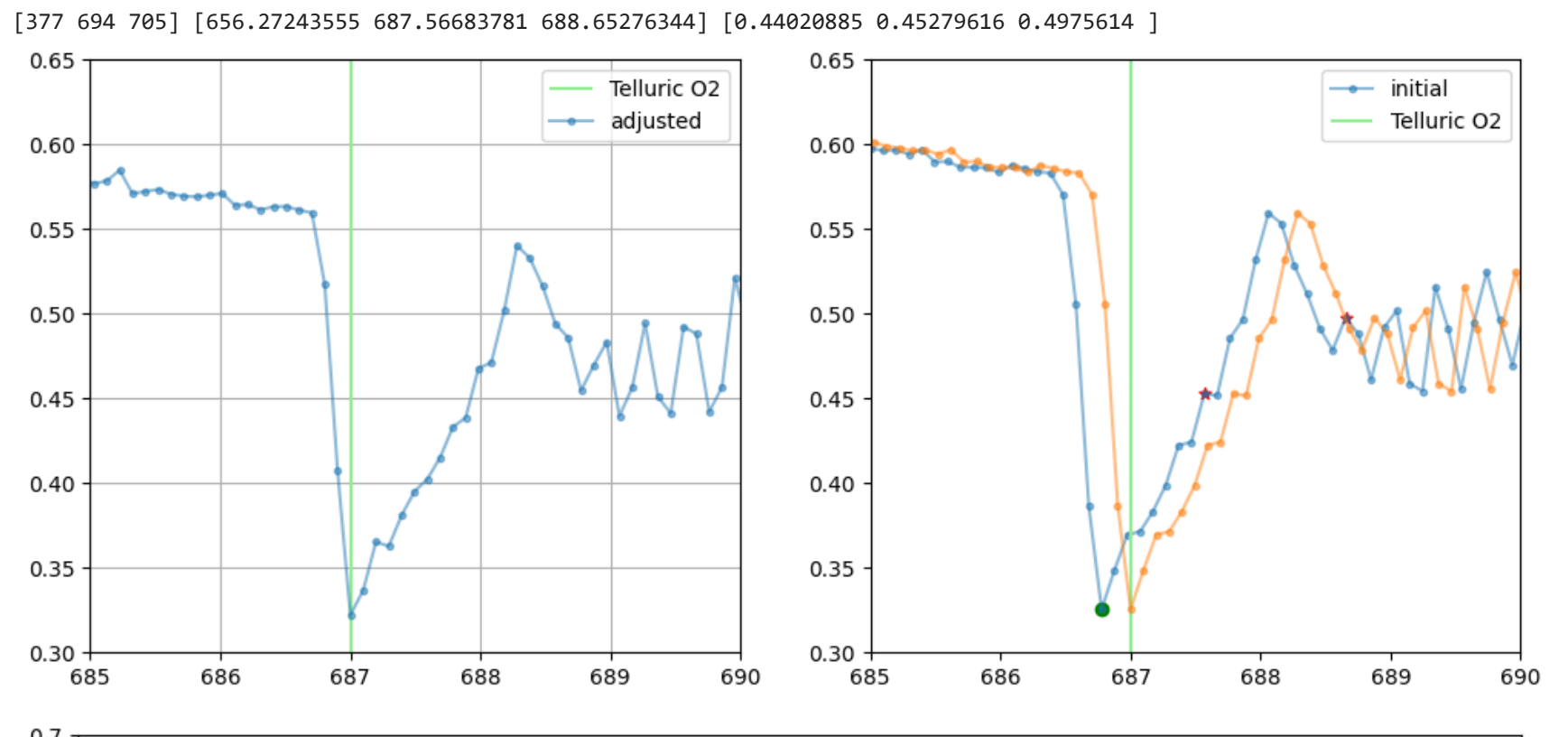


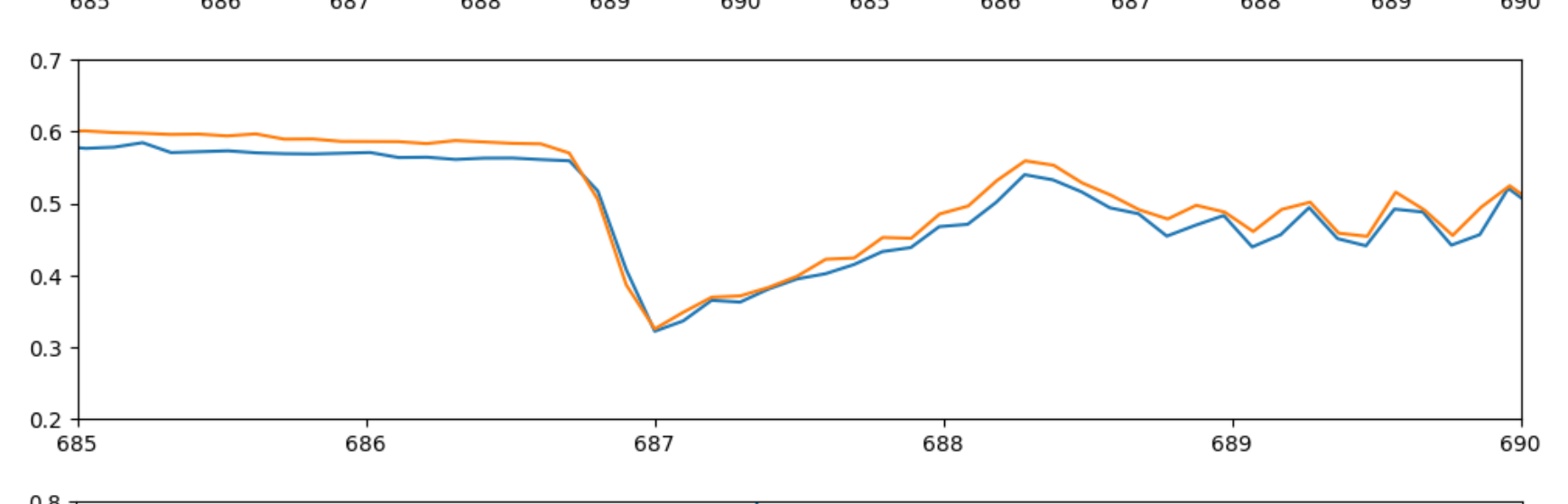
Aligning H alpha data with Telluric Oxygen Line

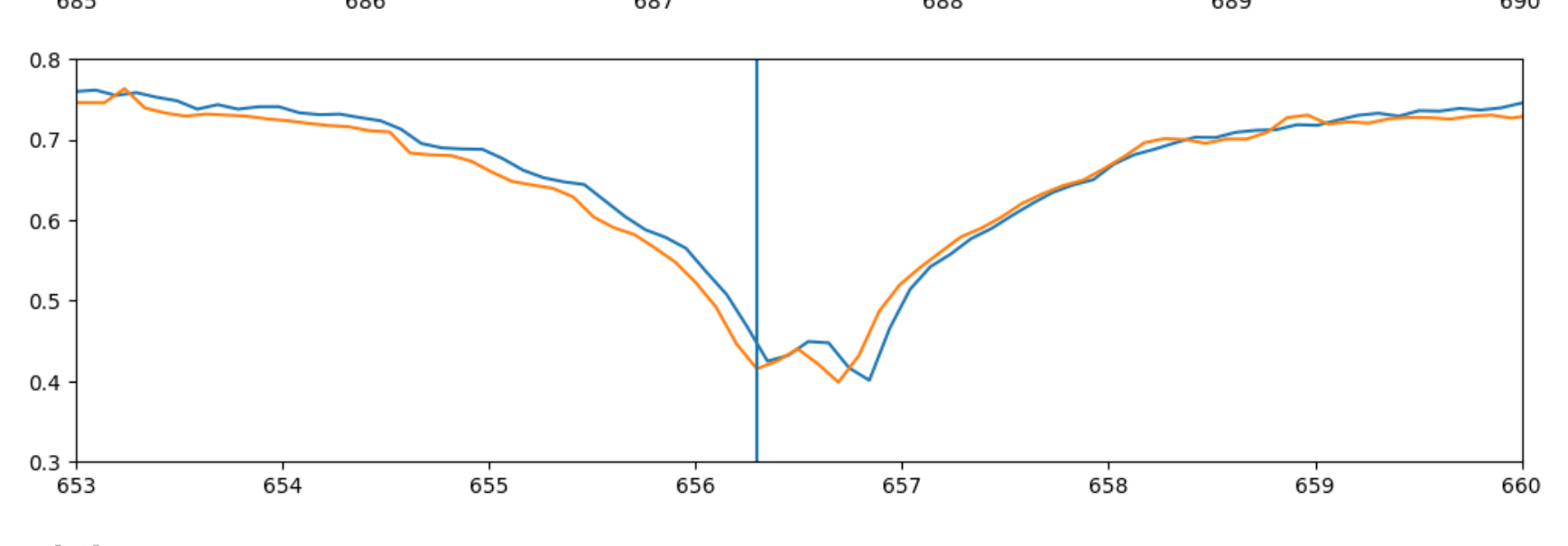
In [39]:

```python
spectrum_label = 'Ha_set1'
index_set = calibration_indices[spectrum_label]
spectrum_index = np.arange(len(normalized_spectra[spectrum_label])) #pixel index
wavelengths = pixel_to_wavelength_index(spectrum_index, *calibration_stats[index_set]) #pixel index --> wavelength inde
spectrum = normalized_spectra[spectrum_label]
peaks, properties = find_peaks(spectrum, height=(.2,.5))  # Set height threshold if needed
#print(peaks, pixel_to_wavelength_index(peaks, *calibration_stats[index_set]), properties["peak_heights"])
fig, ax = plt.subplots(1,2,figsize=(12,5))
#ax[0].scatter(pixel_to_wavelength_index(peaks, *calibration_stats[index_set]), properties["peak_heights"], marker='*',
#ax[0].plot(wavelengths, spectrum, marker='.', alpha=.5, label='initial')
ax[0].set_xlim(685,690)
ax[0].set_ylim(.3,.65)
ax[0].vlines(x=687, ymin=0, ymax=1, color='lightgreen', label='Telluric O2')
ha1_shift = 687 - pixel_to_wavelength_index(693, *calibration_stats[index_set])
wavelengths_adj_ha1 = ha1_shift + pixel_to_wavelength_index(spectrum_index, *calibration_stats[index_set])
ax[0].plot(wavelengths_adj_ha1, spectrum, marker='.', alpha=.5, label='adjusted')
ax[0].grid(True)
ax[0].legend()

spectrum_label = 'Ha_set2'
index_set = calibration_indices[spectrum_label]
spectrum_index = np.arange(len(normalized_spectra[spectrum_label])) #pixel index
wavelengths = pixel_to_wavelength_index(spectrum_index, *calibration_stats[index_set]) #pixel index --> wavelength inde
spectrum = normalized_spectra[spectrum_label]
peaks, properties = find_peaks(spectrum, height=(.2,.5))  # Set height threshold if needed
print(peaks, pixel_to_wavelength_index(peaks, *calibration_stats[index_set]), properties["peak_heights"])
ax[1].scatter(pixel_to_wavelength_index(peaks, *calibration_stats[index_set]), properties["peak_heights"], marker='*',
ax[1].plot(wavelengths, spectrum, marker='.', alpha=.5, label='initial')
pos = 686
ax[1].scatter(pixel_to_wavelength_index(pos,*calibration_stats[index_set]), spectrum[pos], c='green')
ax[1].set_xlim(685,690)
ax[1].set_ylim(.3,.65)
ax[1].vlines(x=687, ymin=0, ymax=1, color='lightgreen', label='Telluric O2')
ha2_shift = 687 - pixel_to_wavelength_index(pos, *calibration_stats[index_set])
wavelengths_adj_ha2 = pixel_to_wavelength_index(spectrum_index, *calibration_stats[index_set]) + ha2_shift
ax[1].plot(wavelengths_adj_ha2, spectrum, marker='.', alpha=.5,) #label='adjusted')
ax[1].legend()

plt.figure(figsize=(12,3))
plt.plot(wavelengths_adj_ha1, normalized_spectra['Ha_set1'], label='Ha Set 1')
plt.plot(wavelengths_adj_ha2, normalized_spectra['Ha_set2'], label='Ha Set 2')
plt.xlim(685,690)
plt.ylim(.2,.7)
plt.figure(figsize=(12,3))
plt.plot(wavelengths_adj_ha1, normalized_spectra['Ha_set1'], label='Ha Set 1')
plt.plot(wavelengths_adj_ha2, normalized_spectra['Ha_set2'], label='Ha Set 2')
plt.axvline(656.3)
plt.xlim(653, 660)
plt.ylim(0.3,0.8)
plt.subplots_adjust(left=0.1, right=0.9, top=0.99, bottom=0.1)
```

```
[377 694 705] [656.27243555 687.56683781 688.65276344] [0.44020885 0.45279616 0.4975614 ]
```





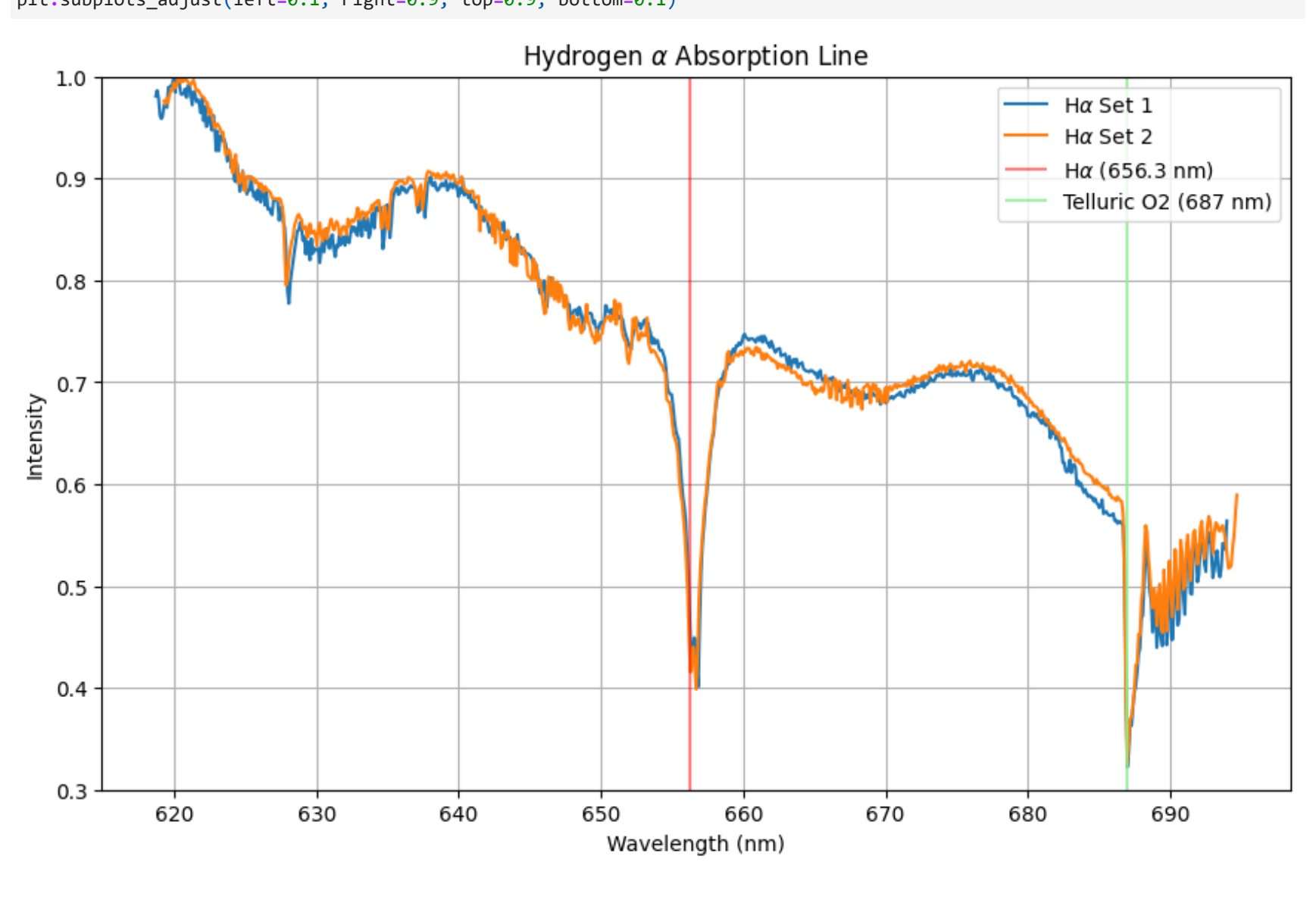


In [40]:

```python
adjusted_wavelengths = {'Ha_set1' : wavelengths_adj_ha1,
                        'Ha_set2' : wavelengths_adj_ha2
                       }
```

In [41]:

```python
#View Stacked Spectra
plot_error = False
spectrum_labels = ['Ha_set1','Ha_set2']
fig, ax = plt.subplots(figsize=(10,6))
for label, (spectrum_key, spectrum), (wavelengths_key, wavelengths) in zip(spectrum_labels, normalized_spectra.items(),
    line, = ax.plot(wavelengths, normalized_spectra[label], label=label.replace('_',' ').replace('a',f'$\\alpha$').repl
    line_color = line.get_color()
    if plot_error:
        ax.errorbar(wavelengths, normalized_spectra[label], normalized_spectra_unc[label],
                fmt='.', label=s_label+' Uncertainty', color=line_color, alpha=.8)
plt.title('Hydrogen $\\alpha$ Absorption Line')
plt.ylabel('Intensity')
plt.xlabel('Wavelength (nm)')
plt.yticks(np.arange(0.0,1.1,0.1))
plt.xticks(np.arange(610,730,10),rotation=0)
#plt.xticks(np.arange(420,500,5),rotation=90)
plt.ylim(.3,1)
#Zoom in to a regiOn
#alpha:
#ax.set_xlim(655.5, 658)
#ax.set_ylim(0.25,0.55)
#beta:
#plt.xlim(484,488)
#plt.ylim(0.3,0.9)
#gamma:
#plt.xlim(433,436.5)
#plt.ylim(0.05,0.7)
#O2:
#plt.xlim(685,690)
#plt.ylim(0.2,0.7)

#Plot Balmer Lines:
plt.vlines(x=656.3, ymin=0, ymax=1, color='r', label='H$\\alpha$ (656.3 nm)', alpha=.5)
#plt.vlines(x=486.1, ymin=0, ymax=1, color='cyan', label='H$\\beta$')
#plt.vlines(x=434, ymin=0, ymax=1, color='blue', label='H$\\gamma$')
plt.vlines(x=687, ymin=0, ymax=1, color='lightgreen', label='Telluric O2 (687 nm)', alpha=.8)

plt.legend(loc='upper right')
plt.grid(True)
plt.subplots_adjust(left=0.1, right=0.9, top=0.9, bottom=0.1)
```



Gaussian Fitting

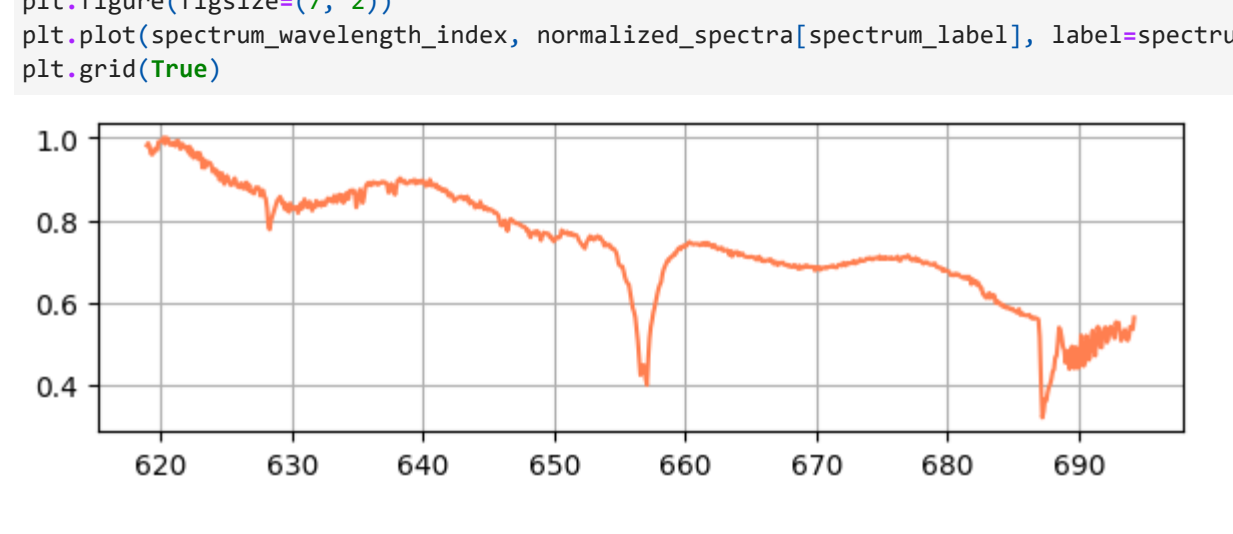
In [42]:

```python
def wavelength_to_pixel_index(wavelength, slope, b):
    return (wavelength - b) / slope
def gaussian(x, a, mu, sigma):
    return a * np.exp(-((x - mu)**2) / (2 * sigma**2))
def double_gaussian(wavelength, A1, mu1, sigma1, A2, mu2, sigma2):
    return (
        A1 * np.exp(-(wavelength - mu1) ** 2 / (2 * sigma1 ** 2)) +
        A2 * np.exp(-(wavelength - mu2) ** 2 / (2 * sigma2 ** 2))
    )
def doppler_shift_function(mu1, mu2, lambda_rest):
    '''
    Rest Wavelength in nm
    mu1 and mu2 in nm
    returns in km/s
    '''
    c = 3e8
    delta_lambda = (mu2 - mu1)
    return (c * delta_lambda) / (lambda_rest * 1e3)
rest_wavelengths = {'Ha' : 656.3,
                    'Hb' : 486.1,
                    'Hg' : 434}
```

Choose Spectrum

In [43]:

```python
spectrum_label = 'Ha_set1'
label = labels[spectrum_label]
lambda_rest = rest_wavelengths['Ha']
index_set = calibration_indices[spectrum_label]
spectrum_index = np.arange(len(normalized_spectra[spectrum_label])) #pixel index

spectrum_wavelength_index = pixel_to_wavelength_index(spectrum_index, *calibration_stats[index_set]) #pixel index --> w

plt.figure(figsize=(7, 2))
plt.plot(spectrum_wavelength_index, normalized_spectra[spectrum_label], label=spectrum_label+' Calibrated Spectrum', ms
plt.grid(True)
```

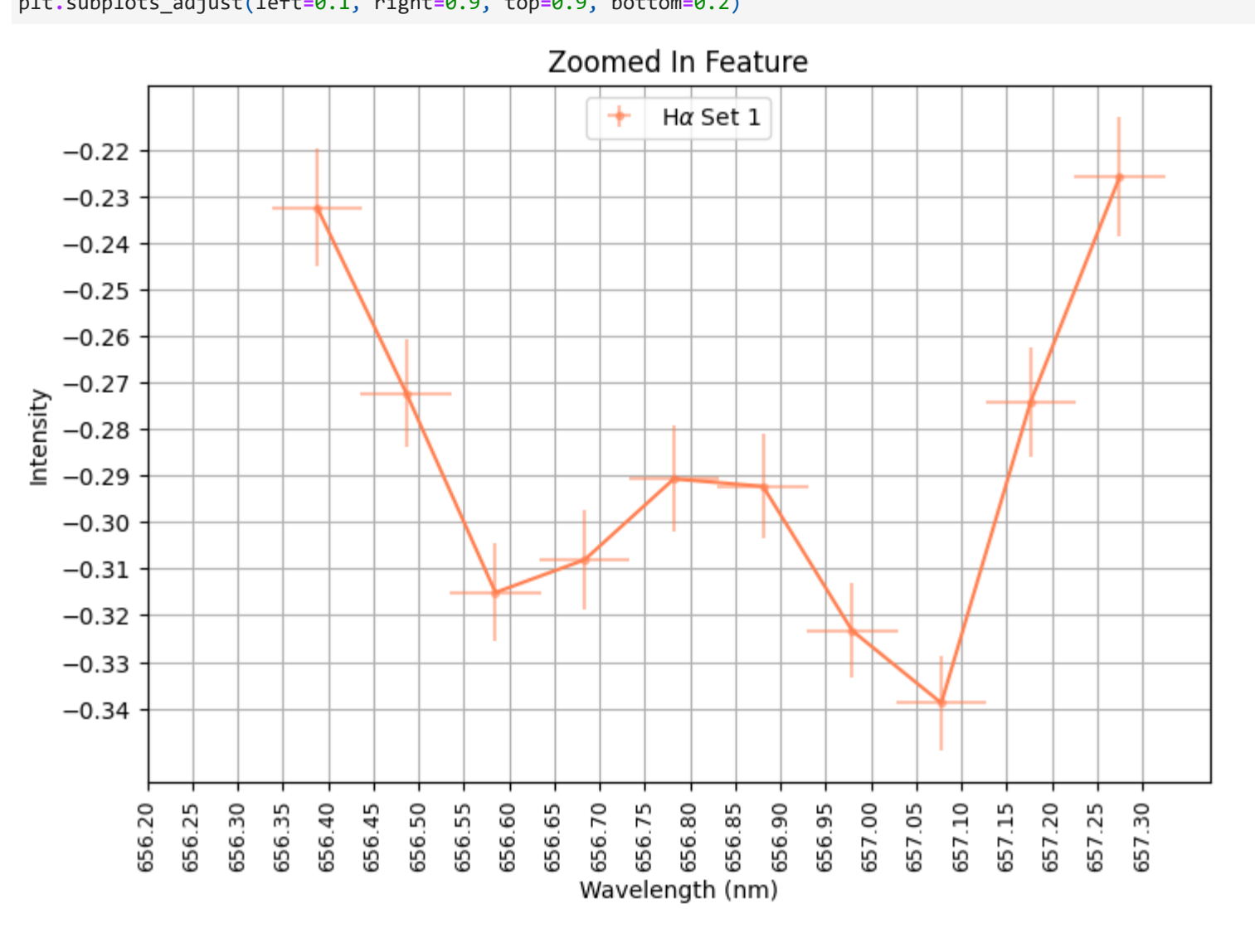


Set Mask and Intensity Offset

Select the wavelength range with the mask, and set the offset so that the base of the gaussian is around zero.

In [44]:

```python
#Set wavelength range with the mask
# adjust offset until the base of the gaussian is around 0.0
wide_view = False # For fining the mask
if spectrum_label == 'Ha_set1':
    wavelength_mask = [656.3,657.3]
    offset = -.74
elif spectrum_label == 'Ha_set2':
    wavelength_mask = [655.6,657]
    offset = -0.54 + .14 -.175
elif spectrum_label == 'Hbg_set1':
    wavelength_mask = [485.5,486.6]
    offset = -.65 +.17
elif spectrum_label == 'Hbg_set2':
    wavelength_mask = [485.5,486.6]
    offset = -.47
## Hgamma set 1:
# wavelength_mask = [433.9,435.05]
# offset = -.32
## Hgamma set 2:
# wavelength_mask = [485.5,486.6]
# offset = -.32

mask = (spectrum_wavelength_index >= wavelength_mask[0]) & (spectrum_wavelength_index <= wavelength_mask[1])
wavelengths = spectrum_wavelength_index[mask]
spectrum = normalized_spectra[spectrum_label][mask] + offset
spectrum_unc = normalized_spectra_unc[spectrum_label][mask]

plt.figure(figsize=(8, 6))
color = 'coral'
plt.plot(wavelengths, spectrum, ms=5, c=color)
plt.errorbar(wavelengths, spectrum, yerr=spectrum_unc, xerr=0.05,
          fmt='.', c=color, label=label, alpha=.5)
plt.title('Zoomed In Feature')
plt.ylabel('Intensity')
plt.xlabel('Wavelength (nm)')
if wide_view:
    tick_separation = .5
    m=math.floor(wavelength_mask[0] / 0.5) * 0.5
    M=math.ceil(wavelength_mask[1] / 0.5) * 0.5
else:
    tick_separation = 0.05
    m=math.floor(wavelength_mask[0] / 0.1) * 0.1
    M=math.ceil(wavelength_mask[1] / 0.1) * 0.1
plt.xticks(np.arange(m,M,tick_separation), rotation=90, size=9)
if not wide_view: plt.yticks(np.arange(round(np.min(spectrum),2),np.max(spectrum)+.01,.01, ))
plt.grid(True)
plt.legend(loc='upper center')
plt.tight_layout
plt.subplots_adjust(left=0.1, right=0.9, top=0.9, bottom=0.2)
```



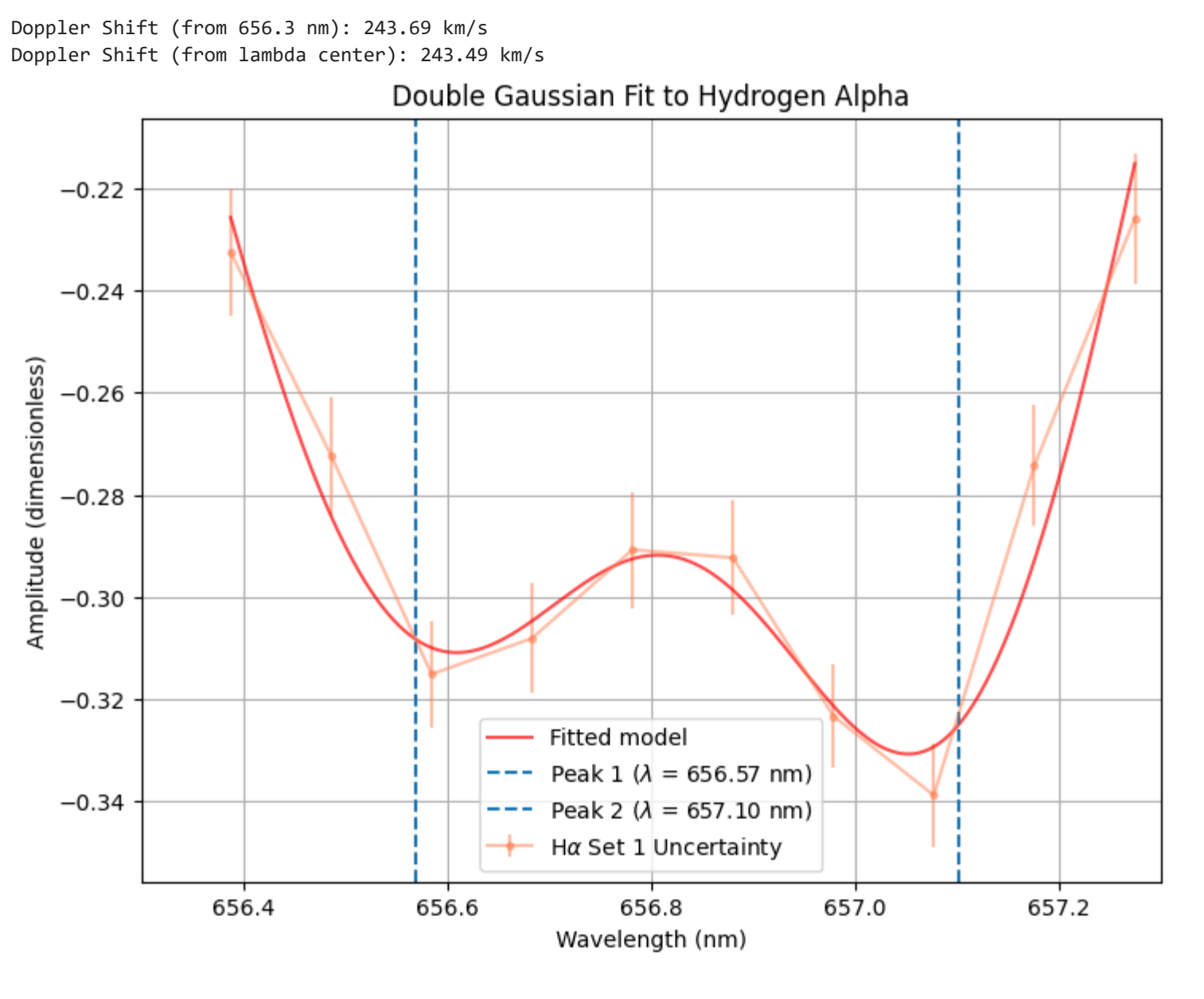
Plot of Double Gaussian Fit

Make good initial guesses by looking at the plot above.

In [45]:

```python
# Initial guesses for fitting (A1, mu1, sigma1, A2, mu2, sigma2)
if spectrum_label == 'Ha_set1':
    initial_guess = [-0.315, 656.55, 0.2, -0.34, 657., 0.2]
elif spectrum_label == 'Ha_set2':
    initial_guess = [-0.16, 656.05, 0.2, -0.175, 656.456, 0.2]
elif spectrum_label == 'Hbg_set1':
    initial_guess = [-0.155, 485.95, 0.2, -0.175, 486.2, 0.2]
elif spectrum_label == 'Hbg_set2':
    initial_guess = [-0.150, 485.95, 0.2, -0.15, 486.2, 0.2]
#Hgamma set 1:
# initial_guess = [-0.12, 434.4, 0.1, -0.12, 434.6, 0.1]

# Fit the double Gaussian model to the data
popt, pcov = curve_fit(double_gaussian, wavelengths, spectrum, p0=initial_guess,
                       sigma=spectrum_unc, absolute_sigma=True)

# Extract the fitted parameters
A1_fit, mu1_fit, sigma1_fit, A2_fit, mu2_fit, sigma2_fit = popt

# Plot the results
plt.figure(figsize=(8, 6))
#plt.plot(wavelengths, spectrum, label=spectrum_label+' Calibrated Spectrum', ms=5, c=color)
plt.errorbar(wavelengths, spectrum, yerr=spectrum_unc, ls='-',
          fmt='.', c=color, label=label+' Uncertainty', alpha=.5)
xrange = np.linspace(np.min(wavelengths), np.max(wavelengths), 10000)
plt.plot(xrange, double_gaussian(xrange, *popt), 'r-', label='Fitted model', alpha=.7)
plt.axvline(mu1_fit, linestyle='--', label=f'Peak 1 ($\lambda$ = {mu1_fit:.2f} nm)')
plt.axvline(mu2_fit, linestyle='--', label=f'Peak 2 ($\lambda$ = {mu2_fit:.2f} nm)')
plt.xlabel('Wavelength (nm)')
plt.ylabel('Amplitude (dimensionless)')
plt.title('Double Gaussian Fit to Hydrogen Alpha')
#plt.axvline(mu1_fit-sigma1_fit, linestyle='--', label=f'$\mu1 - \sigma1$',alpha=.4)
#plt.axvline(mu2_fit+sigma2_fit, linestyle='--', label=f'$\mu2 + \sigma2$',alpha=.4)
#plt.xticks(np.arange(653,661,0.25), rotation=90, size=8)
plt.xlim(wavelength_mask)
plt.legend()
plt.subplots_adjust(left=0.1, right=0.9, top=0.9, bottom=0.1)
plt.grid(True)
print(f"Fitted Parameters:")
print(f"Peak 1: A1 = {A1_fit:.2f}, mu1 = {mu1_fit:.2f}, sigma1 = {sigma1_fit:.2f}")
print(f"Peak 2: A2 = {A2_fit:.2f}, mu2 = {mu2_fit:.2f}, sigma2 = {sigma2_fit:.2f}")
lambda_center = abs(mu1_fit + mu2_fit)/2
dlambda = lambda_center - lambda_rest
print(f"Lambda Center: {lambda_center:.3f} which is {dlambda:.3f} away from rest")

print(f"\nDoppler Shift (from 656.3 nm): {doppler_shift_function(mu1_fit, mu2_fit, lambda_rest=lambda_rest):.2f} km/s")
print(f"Doppler Shift (from lambda center): {doppler_shift_function(mu1_fit, mu2_fit, lambda_rest=lambda_center):.2f} k
```

```
Fitted Parameters:
Peak 1: A1 = -0.30, mu1 = 656.57, sigma1 = 0.24
Peak 2: A2 = -0.30, mu2 = 657.10, sigma2 = 0.21
Lambda Center: 656.836 which is 0.536 away from rest

Doppler Shift (from 656.3 nm): 243.69 km/s
Doppler Shift (from lambda center): 243.49 km/s
```



Chi Squared and Residuals

In [46]:

```python
def residual_statistics(inputs, actual, actual_uncertainty, popt):
    '''
    Returns: residuals, chi_squared, reduced_chi_square, p_value
    '''
    residuals = spectrum - double_gaussian(wavelengths, *popt)
    chi_squared = np.sum((residuals / spectrum_unc)**2)
    nu = (len(wavelengths) - len(popt))
    reduced_chi_square = chi_squared / nu
    p_value = 1 - chi2.cdf(chi_squared, nu)
    return residuals, chi_squared, reduced_chi_square, p_value
```

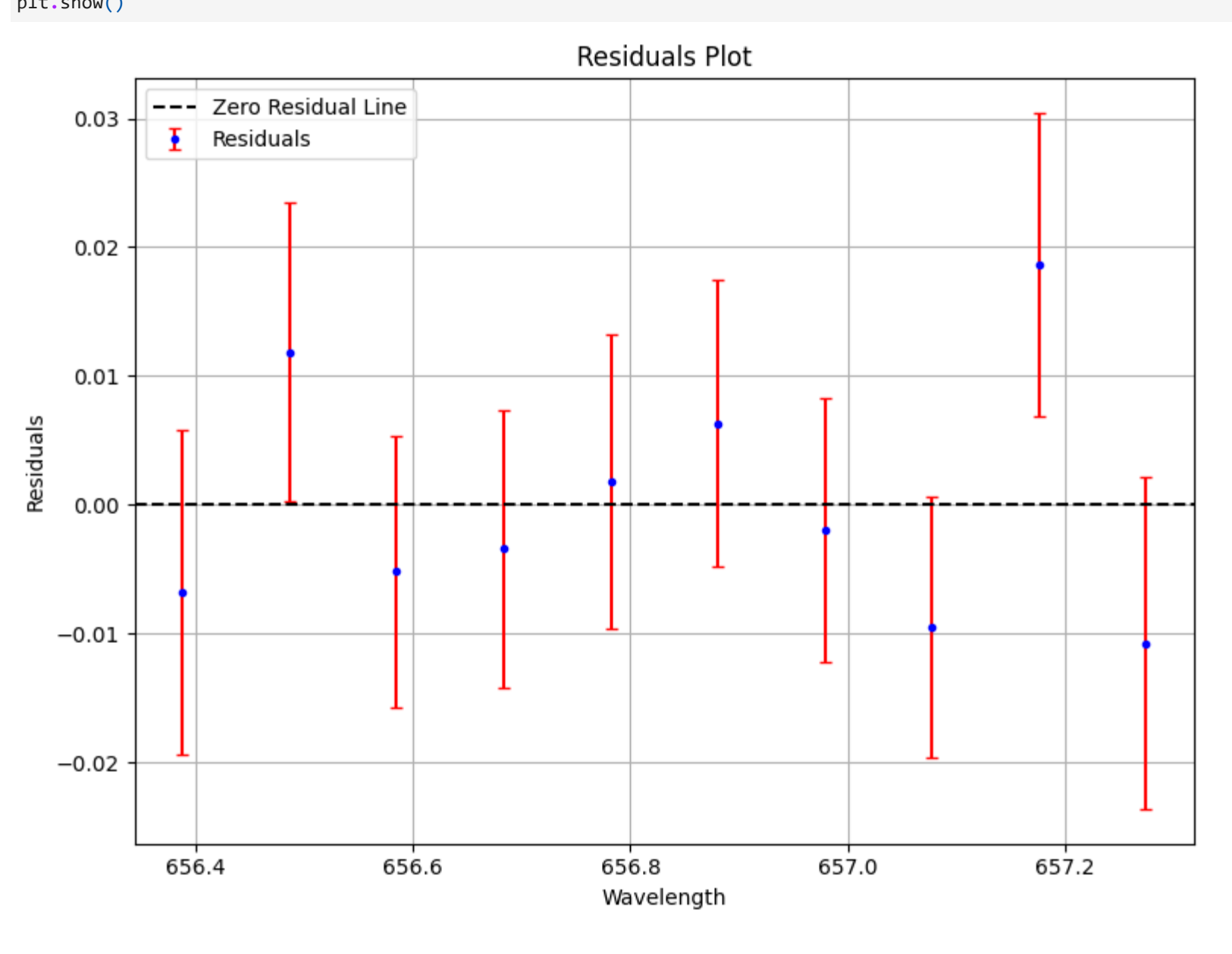
In [47]:

```python
residuals = spectrum - double_gaussian(wavelengths, *popt)
chi_squared = np.sum((residuals / spectrum_unc)**2)
nu = (len(wavelengths) - len(popt))
reduced_chi_square = chi_squared / nu
p_value = 1 - chi2.cdf(chi_squared, nu)
print(f"Chi-Square: {chi_squared:.2f}")
print(f"Reduced chi-square: {reduced_chi_square:.2f}")
print(f"P-Value: {p_value:.2f}")
```

```
Chi-Square: 6.15
Reduced chi-square: 1.54
P-Value: 0.19
```

In [48]:

```python
plt.figure(figsize=(8, 6))
plt.errorbar(wavelengths, residuals, yerr=spectrum_unc, fmt='.', label='Residuals', color='blue', ecolor='red', capsize
plt.axhline(0, color='black', linestyle='--', label='Zero Residual Line')
plt.xlabel('Wavelength')
plt.ylabel('Residuals')
plt.title('Residuals Plot')
plt.legend()
plt.grid()
plt.tight_layout()
plt.show()
```

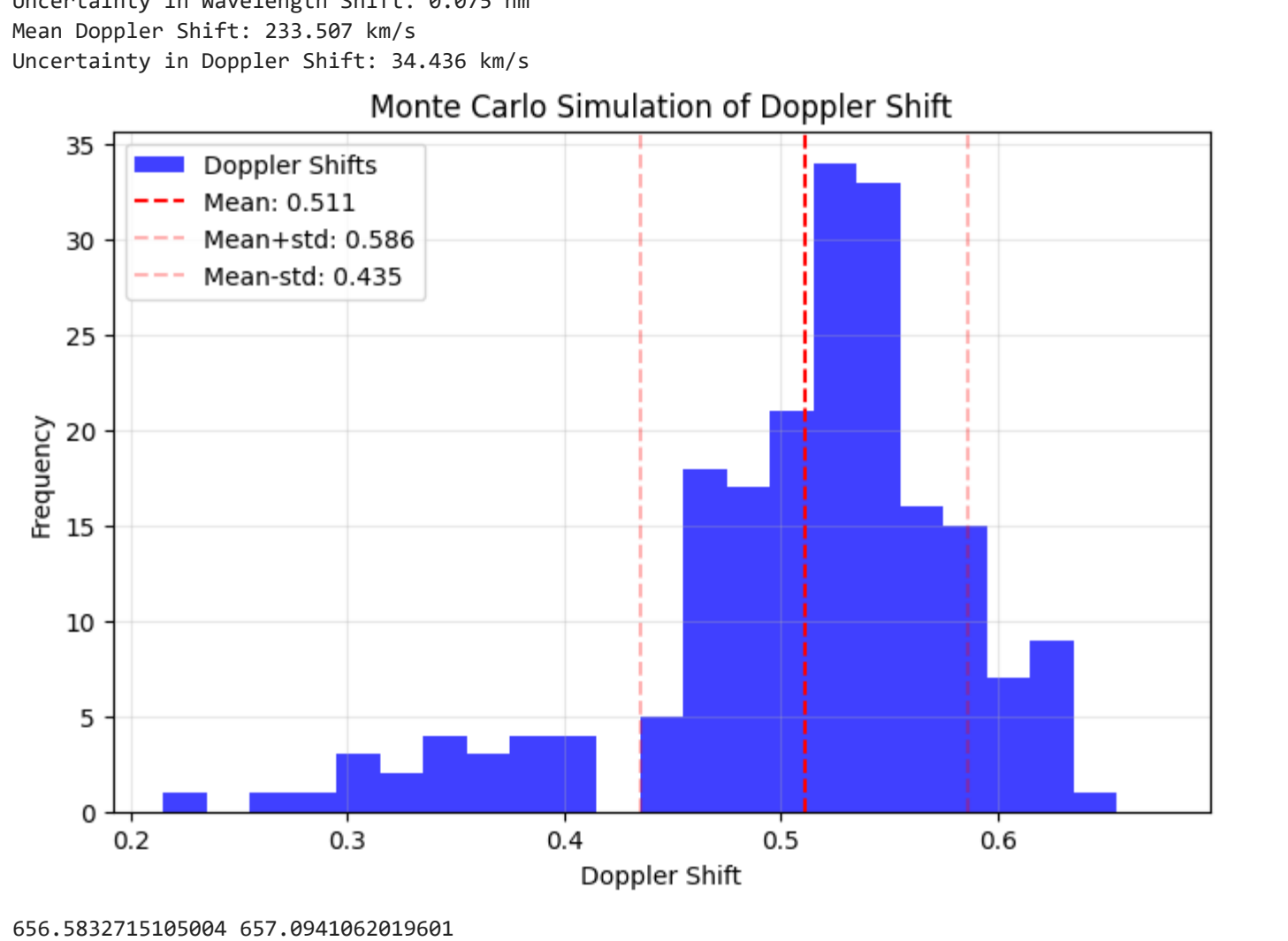


Monte Carlo Simulation

In [49]:

```python
#Monte Carlo
np.random.seed(13)
num_simulations = 200
wavelength_shifts = []
wavelength_unc = np.full_like(wavelengths, 0.05)
popt_array = np.zeros((num_simulations, 6))

for _ in range(num_simulations):
    try:
        #Perturbation
        perturbed_spectrum = spectrum + np.random.normal(0, spectrum_unc)
        perturbed_wavelengths = wavelengths + np.random.normal(0, wavelength_unc)
        #Estimate Params
        popt, pcov = curve_fit(double_gaussian, perturbed_wavelengths, perturbed_spectrum,
                               p0=initial_guess, sigma=spectrum_unc, absolute_sigma=True, maxfev=10000)

        #Find doppler shift
        popt_array[_, :] = popt
        doppler_shift = popt[4] - popt[1]
        wavelength_shifts.append(doppler_shift)
    except Exception as e:
        print(f"Error during fit: {e}")
        continue  # Skip this iteration if the fit fails
print(f"Iterations Ran: {len(wavelength_shifts)}")

# Calculate Results
popt_means = np.mean(popt_array, axis=0)
mu1_mean = popt_means[1]
mu2_mean = popt_means[4]
wavelength_shifts = np.array(wavelength_shifts)
mean_shift = np.mean(wavelength_shifts)
std_shift = np.std(wavelength_shifts)
mean_doppler_shift_velocity = doppler_shift_function(mu2=mean_shift, mu1=0, lambda_rest=lambda_rest)
mean_doppler_shift_uncertainty = doppler_shift_function(mu2=std_shift, mu1=0, lambda_rest=lambda_rest)

print(f"Mean Wavelength Shift: {mean_shift:.3f} nm")
print(f"Uncertainty in Wavelength Shift: {std_shift:.3f} nm")
print(f"Mean Doppler Shift: {mean_doppler_shift_velocity:.3f} km/s")
print(f"Uncertainty in Doppler Shift: {mean_doppler_shift_uncertainty:.3f} km/s")

# Plot distribution of Doppler shifts
bins = np.arange(min(wavelength_shifts), max(wavelength_shifts), 0.02)  # Adjust bin width if necessary
plt.figure(figsize=(8, 5))
plt.hist(wavelength_shifts, bins=bins, alpha=0.75, color='blue', label='Doppler Shifts')
plt.axvline(mean_shift, color='red', linestyle='--', label=f'Mean: {mean_shift:.3f}')
plt.axvline(mean_shift + std_shift, color='red', linestyle='--', label=f'Mean+std: {mean_shift + std_shift:.3f}', alpha
plt.axvline(mean_shift - std_shift, color='red', linestyle='--', label=f'Mean-std: {mean_shift - std_shift:.3f}', alpha
plt.xlabel('Doppler Shift')
plt.ylabel('Frequency')
plt.title('Monte Carlo Simulation of Doppler Shift')
plt.legend()
plt.grid(alpha=0.3)
plt.show()
print(mu1_mean, mu2_mean)
print(popt_means[2],popt_means[5])
```

```
Iterations Ran: 200
Mean Wavelength Shift: 0.511 nm
Uncertainty in Wavelength Shift: 0.075 nm
Mean Doppler Shift: 233.507 km/s
Uncertainty in Doppler Shift: 34.436 km/s
```




```
656.5832715105004 657.0941062019601
0.29002000852903115 0.21709851169088823
```

Monte Carlo Simulation (Streamlined)

In [50]:

```python
initial_guesses = {'Ha_set1': [-0.315, 656.55, 0.2, -0.34, 657., 0.2], #[-0.315, 656.55, 0.2, -0.34, 657., 0.2]
                   'Ha_set2': [-0.16, 656.05, 0.2, -0.175, 656.456, 0.2],
                   'Hbg_set1': [-0.155, 485.95, 0.2, -0.175, 486.2, 0.2],
                   'Hbg_set2': [-0.150, 485.95, 0.2, -0.15, 486.2, 0.2]
                  }
masks = {'Ha_set1' : [656.3,657.3],
         'Ha_set2' : [655.6,657],
         'Hbg_set1' : [485.5,486.6],
         'Hbg_set2' : [485.5,486.6]
        }
offsets = {'Ha_set1' : -0.74,
           'Ha_set2' : -0.54 + .14 -.175,
           'Hbg_set1' : -0.65 +.17,
           'Hbg_set2' : -0.47
          }
rest_wavelengths = {'Ha_set1' : 656.3,
                    'Ha_set2' : 656.3,
                    'Hbg_set1' : 486.1,
                    'Hbg_set2' : 486.1
                   }
```

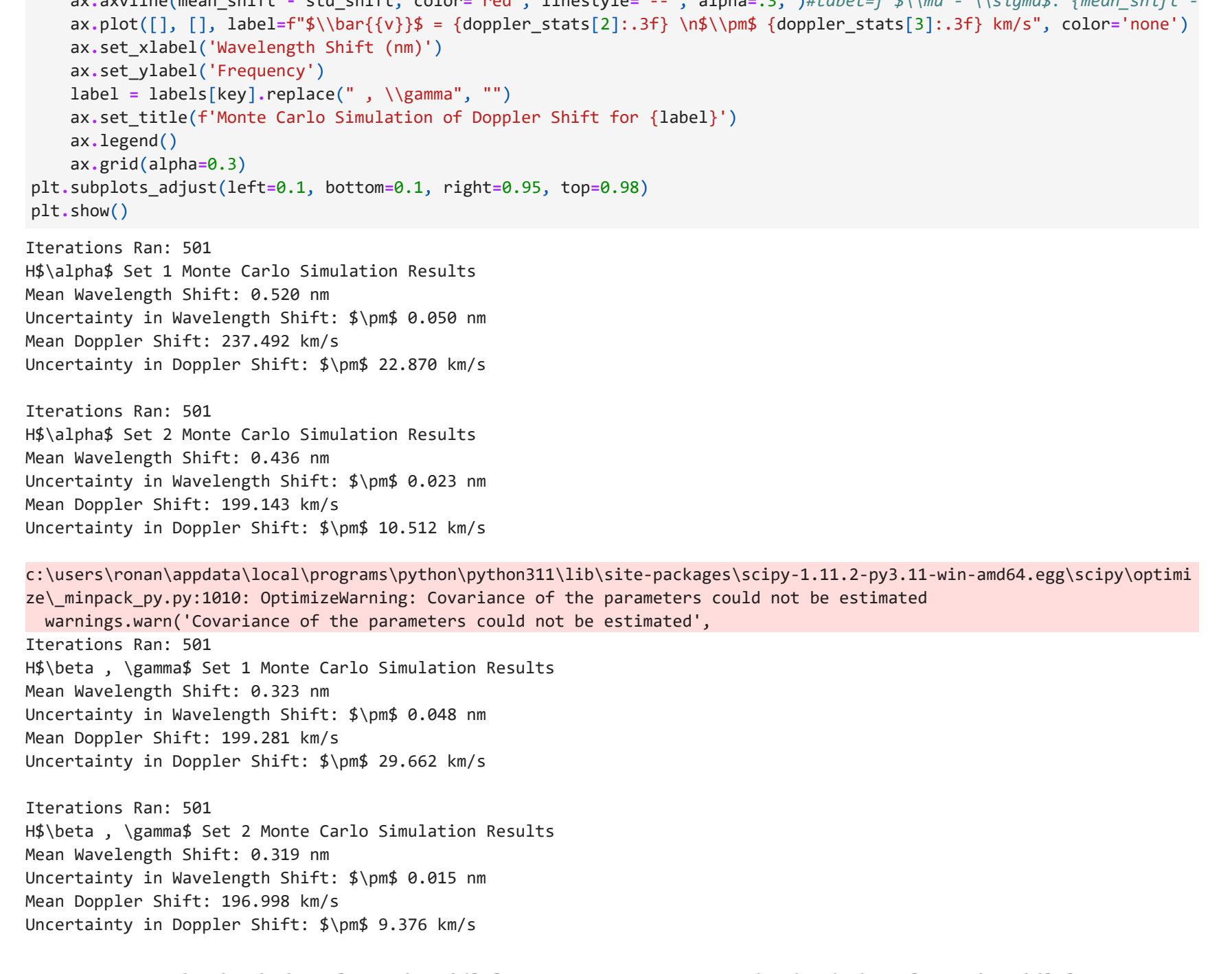
Steps: For each absorption line

1. select proper masks and offsets to apply
2. perturb the spectrum and wavelengths
3. find optimal parameters
4. loop 200x

In [51]:

```python
num_simulations = 20000 # max number
target_number = 500
monte_carlo_stats = {}

for (guess_key, guess), (mask_key, mask), (offset_key, offset), (lambda_key, rest_wavelength) in zip(initial_guesses.it
    np.random.seed(13)

    #Set Spectra Data
    lambda_rest = rest_wavelength
    index_set = calibration_indices[guess_key]
    spectrum_index = np.arange(len(normalized_spectra[guess_key])) #pixel index
    spectrum_wavelength_index = pixel_to_wavelength_index(spectrum_index, *calibration_stats[index_set]) #pixel index -
    spectrum = normalized_spectra[guess_key]
    spectrum_unc = normalized_spectra_unc[guess_key]

    #Adjust Data
    wavelengths_mask = (spectrum_wavelength_index >= mask[0]) & (spectrum_wavelength_index <= mask[1])
    wavelengths = spectrum_wavelength_index[wavelengths_mask]
    spectrum = spectrum[wavelengths_mask] + offset
    spectrum_unc = spectrum_unc[wavelengths_mask]

    #Initialize Monte Carlo Parameters
    wavelength_shifts = []
    wavelength_unc = np.full_like(wavelengths, 0.02)
    popt_array = np.zeros((num_simulations, 6)) #N x 6 parameters array

    #Monte Carlo Simulation
    for i in range(num_simulations):
        if i>target_number: break
        try:
            #Perturbation
            perturbed_spectrum = spectrum + np.random.normal(0, spectrum_unc)
            perturbed_wavelengths = wavelengths + np.random.normal(0, wavelength_unc)
            #Extract Parameters
            popt, pcov = curve_fit(double_gaussian, perturbed_wavelengths, perturbed_spectrum,
                                   p0=guess, sigma=spectrum_unc, absolute_sigma=True, maxfev=10000)
            #Residual Statistics
            statistics = residual_statistics(wavelengths, spectrum, spectrum_unc, popt)
            reduced_chi_squared = statistics[2]
            #if (reduced_chi_squared > 1.7) | (reduced_chi_squared < 0.8): continue
            #Find doppler shift
            doppler_shift = popt[4] - popt[1]
            #if doppler_shift > .45: continue
            wavelength_shifts.append(doppler_shift)
            popt_array[i, :] = popt
        except Exception as e:
            print(f"Error during fit: {e}")
            continue  # Skip this iteration if the fit fails
    print(f"Iterations Ran: {len(wavelength_shifts)}")

    # Calculate Wavelength Centers
    popt_means = np.mean(popt_array, axis=0)
    mu1_mean = popt_means[1]
    mu2_mean = popt_means[4]
    # Calculate Mean and Std of Wavelength (Doppler) Shifts
    wavelength_shifts = np.array(wavelength_shifts)
    mean_shift = np.mean(wavelength_shifts)
    std_shift = np.std(wavelength_shifts)
    # Calculate Doppler Shifted velocity and uncertainty
    mean_doppler_shift_velocity = doppler_shift_function(mu2=mean_shift, mu1=0, lambda_rest=lambda_rest)
    mean_doppler_shift_uncertainty = doppler_shift_function(mu2=std_shift, mu1=0, lambda_rest=lambda_rest)
    #Doppler Stats
    doppler_stats = [mean_shift, std_shift, mean_doppler_shift_velocity, mean_doppler_shift_uncertainty]

    #Add to Dict
    monte_carlo_stats[guess_key] = (doppler_stats, wavelength_shifts, popt_means)
    print(f"{labels[guess_key]} Monte Carlo Simulation Results")
    print(f"Mean Wavelength Shift: {mean_shift:.3f} nm")
    print(f"Uncertainty in Wavelength Shift: $\\pm$ {std_shift:.3f} nm")
    print(f"Mean Doppler Shift: {mean_doppler_shift_velocity:.3f} km/s")
    print(f"Uncertainty in Doppler Shift: $\\pm$ {mean_doppler_shift_uncertainty:.3f} km/s\n")


# Plot distribution of Doppler shifts
fig, axes = plt.subplots(2,2, figsize=(10,10))
axes = axes.flatten()
for ax, (key, value) in zip(axes, monte_carlo_stats.items()):
    doppler_stats = value[0]
    wavelength_shifts = value[1]
    mean_shift = doppler_stats[0]
    std_shift = doppler_stats[1]
    bins = np.arange(min(wavelength_shifts), max(wavelength_shifts), 0.02)  # Adjust bin width if necessary
    ax.hist(wavelength_shifts, bins=bins, alpha=0.75, color='blue', label='Doppler Shifts')
    ax.axvline(mean_shift, color='red', linestyle='--', label=f"$\\mu$: {mean_shift:.3f} $\\pm$ {std_shift:.3f} nm")
    ax.axvline(mean_shift + std_shift, color='red', linestyle='--', alpha=.3, )#label=f'$\\mu + \\sigma$: {mean_shift +
    ax.axvline(mean_shift - std_shift, color='red', linestyle='--', alpha=.3, )#label=f'$\\mu - \\sigma$: {mean_shift -
    ax.plot([], [], label=f"$\\bar{{v}}$ = {doppler_stats[2]:.3f} \n$\\pm$ {doppler_stats[3]:.3f} km/s", color='none')
    ax.set_xlabel('Wavelength Shift (nm)')
    ax.set_ylabel('Frequency')
    label = labels[key].replace(" , \\gamma", "")
    ax.set_title(f'Monte Carlo Simulation of Doppler Shift for {label}')
    ax.legend()
    ax.grid(alpha=0.3)
plt.subplots_adjust(left=0.1, bottom=0.1, right=0.95, top=0.98)
plt.show()
```

```
Iterations Ran: 501
H$\alpha$ Set 1 Monte Carlo Simulation Results
Mean Wavelength Shift: 0.520 nm
Uncertainty in Wavelength Shift: $\pm$ 0.050 nm
Mean Doppler Shift: 237.492 km/s
Uncertainty in Doppler Shift: $\pm$ 22.870 km/s

Iterations Ran: 501
H$\alpha$ Set 2 Monte Carlo Simulation Results
Mean Wavelength Shift: 0.436 nm
Uncertainty in Wavelength Shift: $\pm$ 0.023 nm
Mean Doppler Shift: 199.143 km/s
Uncertainty in Doppler Shift: $\pm$ 10.512 km/s

c:\users\ronan\appdata\local\programs\python\python311\lib\site-packages\scipy-1.11.2-py3.11-win-amd64.egg\scipy\optimi
ze\_minpack_py.py:1010: OptimizeWarning: Covariance of the parameters could not be estimated
  warnings.warn('Covariance of the parameters could not be estimated',
Iterations Ran: 501
H$\beta , \gamma$ Set 1 Monte Carlo Simulation Results
Mean Wavelength Shift: 0.323 nm
Uncertainty in Wavelength Shift: $\pm$ 0.048 nm
Mean Doppler Shift: 199.281 km/s
Uncertainty in Doppler Shift: $\pm$ 29.662 km/s

Iterations Ran: 501
H$\beta , \gamma$ Set 2 Monte Carlo Simulation Results
Mean Wavelength Shift: 0.319 nm
Uncertainty in Wavelength Shift: $\pm$ 0.015 nm
Mean Doppler Shift: 196.998 km/s
Uncertainty in Doppler Shift: $\pm$ 9.376 km/s
```

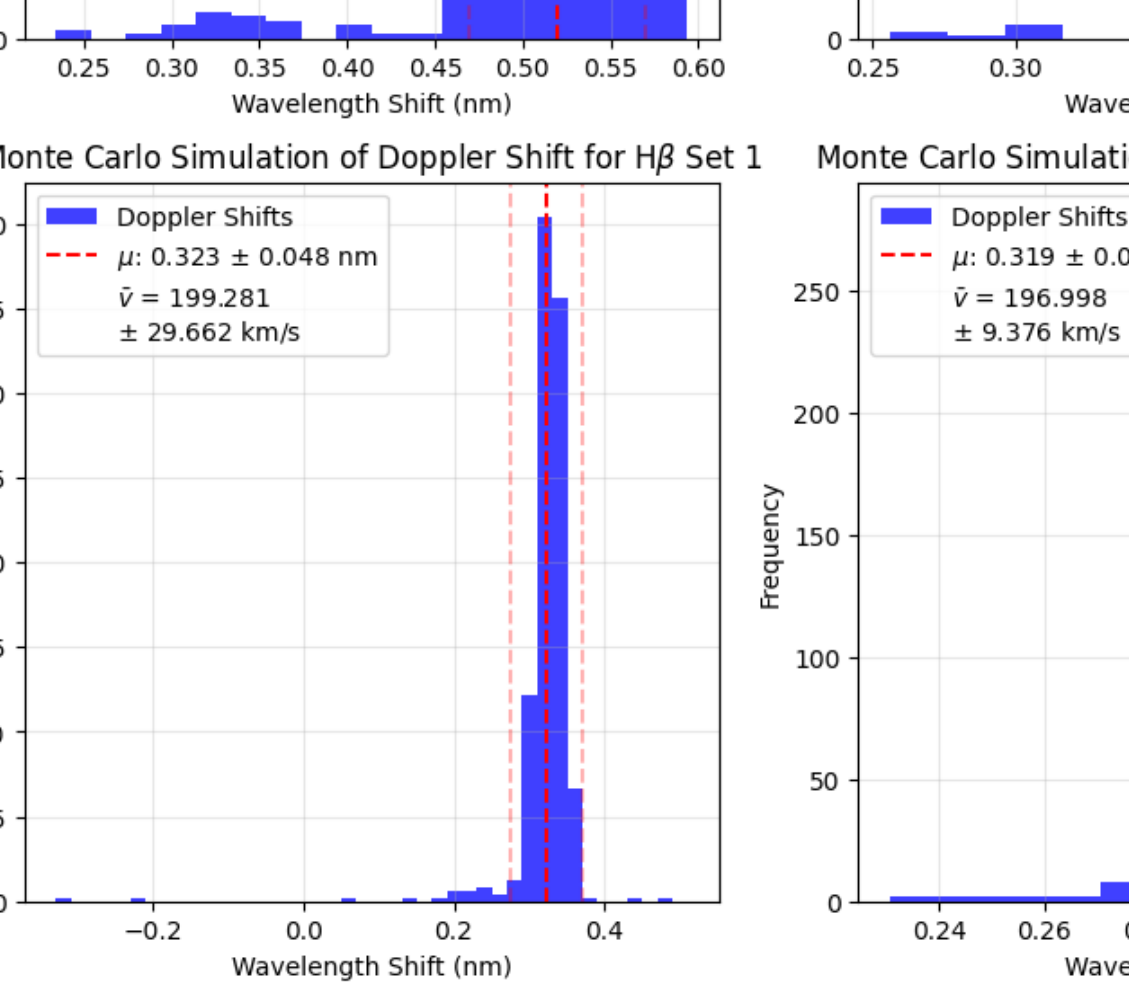


Radial Velocity Curve (Theoretical)

In [52]:

```python
Period = 3.96004753
Period_hrs = Period*24
def radial_velocity_curve(t):
    w = (np.pi*2) / Period_hrs
    A = 219
    return A*np.cos(w*t)
times = np.linspace(-Period_hrs,Period_hrs,10000)
plt.plot(times, radial_velocity_curve(times))
plt.axhline(y=0,c='black',lw=1)
plt.axvline(x=0,c='black',lw=1)

def wavelength_shift(wavelength, velocity):
    '''
    wavelength in nm
    velocity in km/s
    returns wavelength shift in angstroms
    '''
    wavelengths = {'alpha':656.3,
                   'beta':486.1,
                   'gamma':434.0}
    wavelength = wavelengths[wavelength]
    c = 3.0e8
    return ((1e-6*wavelength*velocity)/c) * 1e10

#delta1 = #-Period_hrs/2 + 61.5
#delta2 = #-Period_hrs/2 + 64

'''These are the times when the radial velocities
reach the limit of what we can see on the spectrograph:'''
tmx1 = 16.5300020187
tmx2 = 30.9905683413
tmx3 = 64.0505723787
tmx4 = 78.5111387013
plt.scatter(x=tmx1, y=radial_velocity_curve(tmx1),c='red',s=20)
plt.scatter(x=tmx2, y=radial_velocity_curve(tmx2),c='red',s=20)
plt.scatter(x=tmx3, y=radial_velocity_curve(tmx3),c='red',s=20)
plt.scatter(x=tmx4, y=radial_velocity_curve(tmx4),c='red',s=20)

'''Here is for visualizing a theoretical start and end time.
Choose t1 and t2. The area under the graph between these two points
will be filled in.
'''
#t1 = Period_hrs/4 + 4 + (16.0/60.0)
#t2 = Period_hrs/4 + 4 + (26.0/60.0)
#t1 = Period_hrs/4 + 4
#t2 = Period_hrs/4 + 7
t1 = 4
t2 = 8

tvalues=np.linspace(t1,t2,100)
plt.fill_between(tvalues,radial_velocity_curve(tvalues))
plt.plot(tvalues,radial_velocity_curve(tvalues),color='black')
plt.scatter(x=t1, y=radial_velocity_curve(t1),c='black',s=20)
plt.text(t1+.01,radial_velocity_curve(t1)+7,f'{radial_velocity_curve(t1):.2f}',size=10, c='r')
plt.scatter(x=t2, y=radial_velocity_curve(t2),c='black',s=20)
plt.text(t2+.01,radial_velocity_curve(t2)+7,f'{radial_velocity_curve(t2):.2f}',size=10, c='r')
print(f'Radial Velocity Difference Between t1 and t2: \
{(radial_velocity_curve(t1) - radial_velocity_curve(t2)):.3f} km/s \nTime Duration: {(t2-t1):.3f} hrs')
print(f'v(t=t1)={abs(radial_velocity_curve(t1)):.3f} km/s \nv(t=t2)={abs(radial_velocity_curve(t2)):.3f} km/s')
print(f"Pixel shift for v1: {wavelength_shift('alpha',abs(radial_velocity_curve(t1))):.3f} angstroms(pixels)")
print(f"Pixel shift for v2: {wavelength_shift('alpha',abs(radial_velocity_curve(t2))):.3f} angstroms(pixels)")

plt.xlabel('Time (hrs)')
plt.ylabel('Radial Velocity (km/s)')
plt.title('Radial Velocity vs Time')
plt.plot()

#peak at 19:34
time_obs_ha_set1 = 4 + (26/60) + 52/60#ha set 1 (00:50)
time_obs_ha_set2 = 4 + (26/60) +  3.5#ha set 2
time_obs_hbg_set1 = 4 + (26/60) + 2 + (20/60)#ha set 1
time_obs_hbg_set2 = 4 + (26/60) +  2 + (36/60)  #ha set 2
print(f"Expected velocity for Ha set 1 ({time_obs_ha_set1:.3f} hrs): {radial_velocity_curve(time_obs_ha_set1):.3f}")
print(f"Expected velocity for Ha set2 ({time_obs_ha_set2:.3f} hrs): {radial_velocity_curve(time_obs_ha_set2):.3f}")
print(f"Expected velocity for Hbg set 1 ({time_obs_hbg_set1:.3f} hrs): {radial_velocity_curve(time_obs_hbg_set1):.3f}")
print(f"Expected velocity for Hbg set2 ({time_obs_hbg_set2:.3f} hrs): {radial_velocity_curve(time_obs_hbg_set2):.3f}")
```

```
Radial Velocity Difference Between t1 and t2: 22.309 km/s
Time Duration: 4.000 hrs
v(t=t1)=211.387 km/s
v(t=t2)=189.078 km/s
Pixel shift for v1: 4.624 angstroms(pixels)
Pixel shift for v2: 4.136 angstroms(pixels)
Expected velocity for Ha set 1 (5.300 hrs): 205.694
Expected velocity for Ha set2 (7.933 hrs): 189.564
Expected velocity for Hbg set 1 (6.767 hrs): 197.450
Expected velocity for Hbg set2 (7.033 hrs): 195.749
```



In [ ]: